LIBERIA CHRISTIAN COLLEGE

E-Portal System Phase 1 Development Planning Document

Engineering blueprint for the first delivery

PLANNING BLUEPRINT — NOT AN IMPLEMENTATION

Document version	1.0
Date	20 August 2026
Project phase	Phase 1 — Pre-development planning
Prepared by	AI-assisted Senior Development Planning
Source of truth	LCC E-Portal System Overview & Phase 1 Build Scope v4.0
Planning status	Complete — pending approvals
Approval status	Awaiting VPAA & Project Owner sign-off

This document is a planning artefact. It contains architecture, data models, workflows, pseudocode fragments, test scenarios and acceptance criteria. It contains no application source code, no migration files and no configuration. No repository has been created and no software has been built. Implementation must not begin until the blocking decisions listed in Section 38 have been approved.

Table of Contents

1. Document Control

Field	Value
Project name	Liberia Christian College E-Portal System
Client	Liberia Christian College — Office of the Vice President for Academic Affairs (VPAA)
Delivery organisation	Corex Digital Solutions
Phase	Phase 1 — First delivery (grades, GPA/CGPA, course & schedule planning, historical records)
Document title	Phase 1 Development Planning Document
Document type	Engineering blueprint / development baseline
Version	**2.1 — Development Baseline**
Date	25 August 2026
Prepared by	AI-assisted Senior Development Planning
Intended audience	Project owner; College management / VPAA; implementing developer; AI coding agent; future maintainers
Planning status	**Complete.** All Phase 1 requirements analysed; architecture, data model, workflows, stages, gates and tests defined.
Approval status	**Decisions received and closed.** All nine blocking decisions have been answered (Section 1.6), and the two minor items outstanding at v2.0 — the Incomplete resolution deadline and academic standing — are now settled. **No academic-policy decision remains open.** Six configuration and go-live confirmations remain, none of them design decisions.
Implementation status	**Nothing built.** No repository, no database, no code, no dependencies, no deployment. **Stage 1 is cleared to begin.**

1.1 Source documents reviewed

#	Document	Version	Treatment in this plan
1	*Liberia Christian College E-Portal System — System Overview & Phase 1 Build Scope* File: `Liberia_Christian_College_E-Portal_System_Overview_v4_1.pdf` (7 pages)	v4.0 15 Aug 2026	Original source of truth for scope, roles, workflows and phase boundaries. **Superseded on ten specific points** by the College's decisions of 25 August 2026 — every one recorded in Section 1.6.
2	*College decisions and clarifications* — grading scale, credit rules, repeat policy, Student ID format, approval model, import approach, backup frequency, transcript signatories	25 Aug 2026	**Authoritative where it differs from v4.0.** These answers close all nine blocking decisions and constitute the Registrar's grading policy of record until a formal policy document supersedes them.

1.2 What this document is, and is not

This document IS	This document is NOT
The engineering baseline Phase 1 will be built against, with every business rule now specified numerically	A restatement or replacement of the College's requirements
An architecture and data-model blueprint sufficient to begin implementation	An implementation. No code, migrations, components, APIs or configuration are produced here

This document IS	This document is NOT
A dependency-ordered build plan with acceptance criteria and exit gates per stage	A schedule with committed dates or hour estimates — the input data does not support that precision
A closed register of the decisions the College has now made, with each answer's consequences worked through	An open question list. Version 1.0 was that; this version is the answer to it

1.3 Classification vocabulary used throughout

Label	Meaning	Who resolves it
RECOMMENDATION	An engineering position the delivery team proposes. Implementation proceeds on it unless the College overrules.	Project owner (engineering)
ASSUMPTION	Something taken as true so that planning can be completed. If wrong, rework follows — the size of that rework is stated.	Validated by College; tracked in Section 35
REQUIRES APPROVAL	A point materially affecting requirements, academic policy or the student-facing record. **None remains open.**	VPAA / Registrar
BLOCKING DECISION	Work in the affected area cannot start without this answer. **All nine are now closed.**	VPAA / Registrar / Project owner

1.4 Revision history

Version	Date	Author	Change
1.0	20 Aug 2026	AI-assisted Senior Development Planning	Initial issue. Full Phase 1 planning baseline derived from System Overview v4.0. Nine blocking decisions raised.
2.0	25 Aug 2026	AI-assisted Senior Development Planning	**Development baseline.** All nine blocking decisions answered by the College. Grading scale replaced with the nine-point plus/minus scale; GPA precision changed to three decimal places; credit rules, repeat policy, Student ID format, grade-approval model, import approach, backup frequency and transcript signatories all specified. Section 16 rewritten; every worked example and all Appendix B fixtures recomputed against the new scale. Grade approval redesigned to support individual as well as batch decisions. Two new requirements added (graduation credit total; admission forfeiture). Withdrawal (W) removed from scope; Incomplete (I) added. Readiness assessment moved to **cleared to proceed**.
2.1	**25 Aug 2026**	AI-assisted Senior Development Planning	**Final open academic decisions closed.** Incomplete resolution deadline set at one semester, with expiry handled by a semester-close worklist rather than automatic conversion — and confirmed to run through the existing two-key correction workflow rather than a new Admin power. Academic standing switched on with conventional CGPA bands (probation below 2.000, good standing 2.000–3.499, honours 3.500+), suppressed while provisional and compared on the displayed three-decimal value. Sections 16.4.5 and 16.7 rewritten; four fixtures added; semester-close review screen specified.

1.5 Approval block

This document is the controlling engineering baseline for Phase 1 once the signatures below are complete. The decisions in Section 1.6 have been received and implemented in this revision; the signatures confirm them formally.

Role	Name	Signature	Date
Vice President for Academic Affairs			
Registrar (grading policy authority)			
Project Owner / Lead Developer			

1.6 Change register — amendments to System Overview v4.0

The College's decisions of 25 August 2026 change ten points of the source document and add four requirements it did not contain. These are recorded here rather than silently overwritten, so that anyone reading v4.0 alongside this plan can see exactly where the two differ and which governs.

CR	v4.0 §	What v4.0 said	What now governs	Impact on this plan
CR-01	§10	Five-letter scale: A 90–100 = 4.0; B 80–89 = 3.0; C 70–79 = 2.0; D 60–69 = 1.0; F below 60 = 0.0.	**Nine-letter plus/minus scale** with two-decimal grade points: A+ 4.00, A− 3.70, B+ 3.30, B− 2.70, C+ 2.30, C− 1.70, D+ 1.30, D− 0.70, F 0.00. Bands are 5 marks wide from 60 upward; F is below 60.	**Substantial.** Section 16 rewritten; every worked example and all Appendix B fixtures recomputed; grade-point column widened to `NUMERIC(3,2)`.
CR-02	§10.2, §15 Q1	Incomplete, Withdrawal, audit/non-credit and transfer credit all listed as unresolved.	**Incomplete (I) is in use** and appears on both the grade sheet and the transcript, excluded from GPA. **Withdrawal (W) is not used** and must appear on neither. Audit/non-credit and transfer credit are not used in Phase 1.	Grade scale carries ten entries: nine letters plus I. W, AU and TR are removed from every table, screen and fixture in this document.
CR-03	§10.2, §15	GPA rounding — decimal places and direction unconfirmed.	**Three decimal places** for GPA and CGPA. Half-up, applied once at presentation.	Storage widened to six decimal places so the displayed figure is always re-derivable; every fixture states a three-decimal expected value.
CR-04	§8, §15 Q2	Minimum and maximum credit hours per semester unconfirmed.	**Maximum 21 credit hours per semester.** No meaningful minimum — one credit hour is acceptable. **132 credit hours minimum for a BSc.**	Validation V2 now enforces an upper bound only. A new graduation-progress figure is added to the student record.
CR-05	§8, §10.1	"Passed" undefined; repeat policy stated only as a CGPA carry-over rule.	**Minimum passing grade is D** (D− 0.70 and above earns credit). **An F must be repeated. Any D — D+ or D− — earned in a major course must be repeated.** A D in a non-major course stands.	Introduces the concept of a *major course* (a course owned by the student's own department) and a **mandatory-repeat obligation** tracked on the student record. Validation V3 and the student record both change.
CR-06	§9, §15 Q8	Whether Super Admin reviews each grade individually or approves a class as a batch was open.	**Both.** A Super Admin may approve the whole class batch in one action, or approve and reject individual grades within it.	**Substantial.** The grade-approval model is redesigned: decisions are recorded per grade, a submission may be partially decided, and an individually rejected grade returns to the Admin with a reason while its classmates publish.
CR-07	§9, §15	Whether an Admin enters a numeric score or a letter grade was not stated. §10's bands were whole numbers.	**Numeric scores, decimals permitted.** Lecturers submit numeric marks; the Admin enters the number and the system derives the letter. 96 must produce A+.	Score column accepts one decimal place; a documented score-rounding rule now precedes letter derivation.

CR	v4.0 §	What v4.0 said	What now governs	Impact on this plan
CR-08	§15 Q4	Student ID format and reuse open.	**Digits only. First four digits are the admission year** (e.g. 202634). Existing IDs are reused unchanged. Total length 6 to 8 digits, so an intake larger than 99 students is representable.	Login, validation, seeding and import keying all specified. Pattern `^(19|20)\d{2}\d{2,4}$`.
CR-09	*new*	Not in v4.0.	**A student who is admitted and fails to register for two consecutive semesters forfeits the admission.**	New student status and a semester-close report identifying candidates for forfeiture. Specified in Section 12.5.
CR-10	§4, §15	Only "First Semester" exemplified; number of terms per year unstated.	**Exactly two semesters per academic year.** No summer or vacation school. Academic year label format `2026/2027`.	Calendar model unchanged (it already supported any number); seeds and tests fixed at two.
CR-11	§6.2, §15 Q3	How far back records are entered, and by whom, was open.	**The Admin office enters historical records, gradually, after the system is live.**	**Significant operational consequence:** the system goes live with essentially every continuing student's CGPA provisional, and stays that way for months. The provisional marking becomes the most user-visible feature of the first year, not an edge case. Section 17.8 and risk R-02 rewritten accordingly.
CR-12	§11, §14, §15 Q5/Q7	Transcript signatory, fee and electronic-signature questions open. Backup frequency beyond semester-end open.	**Transcripts are signed and sealed by hand** by the VPAA together with the Dean of Admission and Records; no electronic signature; **no fee or request form** — a transcript is valid only when signed and sealed. **Automatic weekly database backup** is adopted.	Transcript items are Phase 2 and are recorded for that phase. The weekly backup is configured in Stage 11 and narrows the data-loss window from one semester to one week.
CR-13	§10.2, §15 Q1	Treatment of Incomplete unresolved; no deadline stated.	**An Incomplete must be resolved within one semester**, per College policy. At the deadline the system produces a worklist; **nothing converts automatically**. Closure runs through the ordinary correction workflow — Admin requests, Super Admin approves.	Section 16.4.5 rewritten. The deadline is shown to the student. Conversion to F recomputes GPA, credits and creates a mandatory repeat obligation. **No new permission is introduced** — an expired Incomplete is a locked grade like any other.
CR-14	§10.2	Standing thresholds offered as optional; none supplied.	**Academic standing is displayed**, derived from CGPA: probation below 2.000, good standing 2.000–3.499, honours 3.500 and above.	Section 16.7 rewritten. Label only — gates nothing. **Suppressed entirely while a CGPA is provisional**, which in year one is most students. Compared on the displayed three-decimal value so the label can never contradict the number beside it. The College selected conventional bands knowing that on this scale a student passing every course sits on probation.

PRECEDENCE RULE

Where this document and System Overview v4.0 disagree, **this document governs**, and the disagreement is listed above. Where this document is silent, v4.0 governs. Nothing in v4.0 has been changed except the twelve points recorded here — scope, roles, workflows, phase boundaries and the out-of-scope list are all unchanged.

2. Executive Summary

The Liberia Christian College E-Portal will become the College's first digital system of record for academic information. There is no existing codebase, database, application, API or authentication system. All grades exist on paper only. Every architectural decision in this plan is made from first principles, and the plan is written so that a competent developer — or an AI coding agent working under that developer's direction — can begin implementation without rediscovering what the system is supposed to do.

This is version 2.0. Version 1.0 raised nine blocking decisions; the College has answered all nine, and this revision implements those answers. The grading rules are now fully specified numerically, which removes the single largest risk the project carried.

2.1 What Phase 1 delivers

Nine components, taken directly from §5.1 of the source document: authentication and role-based access; academic setup (colleges, departments, courses, credit hours, prerequisites, academic years, semesters); student registration and academic profiles; manual historical record import; grade management with entry, validation, publishing and locking; GPA/CGPA calculation; semester and course schedules; course planning with Admin approval; and audit logging from day one. Transcripts, announcements, dashboards and reports remain Phase 2, though the Phase 1 data and audit design deliberately makes room for them.

2.2 The three findings that shape this plan

Finding 1 — Historical import now dominates the *live* system, not just the schedule

The College has decided that the Admin office will enter historical records **gradually, after go-live**. That is a reasonable operational choice, and it has one consequence the plan must confront directly: the system will go live with essentially every continuing student's CGPA computed from an incomplete record, and will stay that way for months. **The provisional marker is therefore not an edge case — it is the normal state of the system in year one.** It must be unmissable on every figure, understood by every user, and honoured by every downstream rule. The plan responds with a single unified academic-record table carrying an explicit `origin` flag, a per-student import status, and a provisional flag returned alongside every computed figure rather than looked up separately (Section 17).

Finding 2 — The grading rules are now specified, and they are more demanding than v4.0's

The five-letter scale in the source document has been replaced by a nine-point plus/minus scale with two-decimal grade points, GPA reported to three decimal places, decimal score entry, a 132-credit graduation requirement, a 21-credit semester ceiling, and a mandatory-repeat rule for any F and for any D earned in a major course. Every one of those is now a number the engine can be tested against, and every worked example and test fixture in this document has been recomputed accordingly (Section 16, Appendix B). The risk has moved from *unknown rules* to *precise rules implemented precisely* — which is a risk automated tests can hold.

Finding 3 — Grade approval is now per-grade as well as per-batch, and that raises the stakes on the transaction

The College wants a Super Admin to be able to approve a whole class at once *or* pick through it grade by grade, rejecting individual entries. That is more useful than either option alone, and it makes the approval operation more complex: a submission can now be partially decided, some grades publishing while others return to the Admin. The segregation of duties is unchanged and remains the point of the whole design — entry and approval stay in different hands, enforced in the service layer, in database policies and by check constraints, never by hiding a button.

2.3 Recommended stack, in one line

Next.js (App Router, TypeScript) on Vercel, with Supabase Postgres as the system of record, Supabase Auth for sessions, Row-Level Security as the second line of defence, and all business logic executed server-side. The full first-principles evaluation, alternatives and trade-offs are in Section 7. The single most important constraint that falls out of this choice: the browser must never talk to the database directly for anything that writes academic data.

2.4 Build sequence

Phase 1 is broken into eleven dependency-ordered stages, each with concrete deliverables, acceptance criteria and a formal exit gate. The critical path runs: platform foundation → identity and RBAC → academic structure → academic calendar and semester state machine → students → historical import → GPA engine → offerings and schedules → course planning → grade lifecycle → hardening and go-live. Historical import (Stage 6) is placed before the GPA engine because its table is the engine's input; the Admin office's data entry, by the College's decision, begins after go-live rather than at Stage 6, so the import screen must be good enough to sustain months of daily use.

2.5 Readiness verdict

Verdict area	Status	Summary
Architecture & stack	**READY**	Requirements sufficient to design and build platform, identity, academic structure and audit foundations.
Academic structure & calendar	**READY**	Two semesters per year confirmed; hierarchy and state machine specified.
Student identity	**READY**	ID format confirmed: digits only, first four the admission year, 6–8 digits, existing IDs reused.
GPA / CGPA engine	**READY**	Nine-point scale, three-decimal precision, passing grade, repeat rules and graduation total all specified. 30 fixtures recomputed.
Course planning	**READY**	21-credit ceiling, no meaningful floor, mandatory-repeat obligations defined.
Grade workflow	**READY**	Numeric entry with decimals; individual and batch approval; students see published grades only.
Historical import	**READY, with a caveat**	Design complete and owner named. The gradual post-live approach means provisional figures persist for months — an operational reality the College should communicate to students, not a software defect.
Scope	**READY**	Announcements and reports confirmed as Phase 2; the two source contradictions are closed.
Incomplete handling	**READY**	One-semester deadline; expiry produces a worklist, never an automatic grade. Closure runs through the existing two-key correction workflow.
Academic standing	**READY**	Conventional CGPA bands, label only, suppressed while provisional — so most students will see no standing during year one.

Overall: every academic-policy decision is now closed and Stages 1 through 11 are cleared to proceed. What remains is configuration and confirmation — hosting tier, region, sample documents, UAT sign-off — none of it a design decision. Section 37 records the position and Section 38 shows what the College still needs to sign.

3. Project Understanding

3.1 The problem in the College's own terms

Liberia Christian College keeps its academic record on paper. Lecturers write grades on grade sheets and hand them to the Admin office. There is no spreadsheet and no software to migrate from. Consequently a student's GPA, CGPA and transcript can only be produced by someone manually reading and adding up paper records, and prerequisite checking at registration time depends on whoever is at the counter remembering what the student has already passed.

The E-Portal replaces that with a controlled digital record. Phase 1 is deliberately narrow: **get grades and course/schedule planning correct.** Everything else — transcripts, dashboards, announcements, lecturer accounts, fees, attendance, admissions — waits until that foundation is in real use and trusted.

3.2 The operating model the software must fit

This is not a system that digitises a workflow the College wants to change. It is a system that must fit the workflow the College already runs, which has three defining properties:

Property	What the source document says	What it means for the build
Paper is upstream of the system	"Lecturers will continue to submit grades to the Admin office on paper. There is no lecturer account in the system." (§2.1)	The primary grade-entry interface is a transcription tool, not a data-capture form. It must be optimised for one person working down a physical sheet: keyboard-first, ordered to match the sheet, resistant to off-by-one row errors.
Paper stays downstream too	"A printable class grade sheet and class list, so the office can still work on paper where needed and file a signed copy." (§9); "Signed paper grade sheets remain the physical fallback." (§14)	Print output is a Phase 1 requirement, not a nicety. The system does not replace the paper file; it becomes the authoritative index over it.
Two people, two keys	"Grade entry (Admin) and grade approval (Super Admin) are kept as two separate actors so no single account can enter and publish a grade unchecked." (§3.1)	Segregation of duties is a first-class requirement. It must survive at the database layer, in every code path, and in the audit trail — not merely in the navigation menu.

3.3 The deliberate role inversion

The source document flags this explicitly as "a deliberate change from a typical 'Super Admin does everything' model." It is worth restating because it inverts the default assumption an implementer would otherwise carry:

Conventional SIS assumption	LCC E-Portal reality
Super Admin is a superset of Admin	**Super Admin is not a superset.** Super Admin cannot register students, cannot create colleges, departments, courses or semesters, and cannot enter or correct grades.
Admin is the operational layer, Super Admin the escalation path	Admin carries essentially the entire operational workload. Super Admin exists to approve grades, manage Admin accounts, and read the audit log.
Elevated role can unstick any workflow	Super Admin can unstick exactly one thing: moving a semester backwards or reopening a closed semester (§7). Everything else routes back to Admin.

> **IMPLEMENTATION CONSEQUENCE**
>
> Permission checks must be written as explicit allow-lists per action per role. A hierarchical model (`role_level >= 2`) will produce a system that violates the College's stated intent within the first month. This is captured as a hard constraint in Section 11 and tested as a forbidden-action suite in Section 23.

3.4 Environmental constraints that affect design

Constraint	Source	Design response
Unreliable connectivity	"Pages are kept light so they load quickly on a slow connection… must behave sensibly when a connection drops mid-action: no half-saved grade records and no duplicate submissions." (§12)	Server-rendered pages with minimal client JavaScript; every multi-row write wrapped in a database transaction; every mutating request carries an idempotency key; optimistic-concurrency tokens on edit forms.
Mixed device profile	"Responsive… Admin work such as grade entry, scheduling and historical import is designed around a full-size screen." (§12)	Two distinct layout strategies: student surfaces are mobile-first; admin surfaces are desktop-first with dense tables and keyboard navigation, degrading to a usable but not optimised mobile view.
Single backup point per semester	"The College will take a full backup at the end of every semester… up to one semester of entry work is at risk between backups." (§14)	Managed Postgres with continuous point-in-time recovery is recommended precisely because it removes this risk at no development cost; the semester-end export remains a separate, human-readable safety net (Section 29).
Small operator team	Implied throughout: one Admin office, one Super Admin oversight role	No feature may depend on a full-time system administrator. Deployment, backup and monitoring must be managed services with sensible defaults.

3.5 Delivery context

Factor	Value and effect on the plan
Team	**One developer directing an AI coding agent.** This plan is therefore written as an executable specification: acceptance criteria are phrased as testable statements, stages are strictly dependency-ordered, and every stage has an exit gate that a human must sign before the next begins. Parallelism is minimal by design; the plan notes where a second developer would unlock a parallel track.
Skill base	JavaScript/TypeScript full-stack. Weighted heavily in the Section 7 evaluation, because on a solo build the cost of unfamiliarity is paid in defects in exactly the areas — transactions, authorisation, numeric accuracy — where this project cannot afford them.
Hosting	Vercel (application) and Supabase (database, auth, storage). Confirmed by the project owner. Section 7 records what this decision buys and what it costs, and Section 30 records the residual risks it creates.
Greenfield	No legacy system, no data migration, no integration contracts. The only "migration" is manual transcription from paper, which is a business process, not a technical one.

4. Requirements Audit

This section separates what the College actually asked for from what must additionally be true for the system to work, from what is still missing. Nothing is silently added and nothing is silently dropped. Every identifier assigned here is used again in the traceability matrix (Section 36).

Prefix	Category	Definition
`REQ-`	Explicit requirement	Stated directly in System Overview v4.0. Not negotiable without a change request.
`DER-`	Derived technical requirement	Not stated, but technically necessary to deliver a REQ. Introduced by the engineering team.
`GAP-`	Missing requirement	Must be defined before the affected code can be written correctly.
`OBD-`	Open business decision	Requires an answer from the College / VPAA / Registrar.
`TEC-`	Technical decision	The engineering team may decide, subject to the project owner's review.
`OOS-`	Out of scope	Explicitly excluded from Phase 1.

4.1 Explicit requirements

4.1.1 Authentication, users and access control

ID	Requirement	Source	Notes
REQ-A01	Students log in with their Student ID and a password.	§13, §5.1 #1	Not an email address. Drives TEC-04.
REQ-A02	Admins issue and reset passwords for users.	§13	No self-service reset is specified.
REQ-A03	A first-login password change is required.	§13	Applies after issue and after every Admin reset.
REQ-A04	Three roles exist: Student, Admin, Super Admin.	§3	No lecturer role in Phase 1.
REQ-A05	Permissions are enforced in the backend and database policies, not only by hiding buttons in the interface.	§3.1, §13	Hard architectural constraint. See Section 11.
REQ-A06	Super Admin manages Admin accounts.	§3, §3.1	Sole account-management authority for Admins.

4.1.2 Roles and permissions

ID	Requirement	Source	Notes
REQ-R01	A Student views only their own academic information and submits/edits only their own course plan.	§3, §3.1	Isolation must be enforced at the data layer.
REQ-R02	Admin views, edits and submits student records; registers students; creates and manages colleges, departments, courses, semesters and class schedules; enters grades; approves/rejects course plans.	§3, §3.1	The operational role.
REQ-R03	Super Admin approves grades before publication, manages Admin accounts, and views institution-wide records and the audit log.	§3, §3.1	Governance role.
REQ-R04	Super Admin does **not** register students, does not create colleges/departments/courses/semesters, and does not perform grade entry or correction.	§3	Explicit denial. Must be tested as forbidden actions.
REQ-R05	Grade entry (Admin) and grade approval (Super Admin) are separate actors; no single account can enter and publish a grade unchecked.	§3.1	Segregation of duties. Non-negotiable.
REQ-R06	Correction of a locked grade is *requested* by Admin and *approved* by Super Admin.	§3.1, §9	Same two-key principle applied to corrections.

ID	Requirement	Source	Notes
REQ-R07	Only Super Admin may move a semester backwards or reopen a closed semester.	§7	Sole Super Admin operational power.
REQ-R08	Only Super Admin views the audit log.	§3.1, §13	

4.1.3 Academic structure

ID	Requirement	Source	Notes
REQ-S01	Academic information is organised in the fixed hierarchy: Academic Year → Semester → College → Department → Course → Course Offering/Class.	§4	See Section 12 for the correct relational reading of this list.
REQ-S02	The College's terms **College** (e.g. College of Christian Education) and **Department** (e.g. Department of Theology) are used in place of generic "department"/"program".	§4	Terminology is preserved verbatim in schema, UI and documentation.
REQ-S03	A Course Offering / Class is distinct from a Course (e.g. CSC 201 vs. CSC 201 – Section A).	§4	Central modelling decision. Section 12.
REQ-S04	Admin creates and manages colleges, departments, courses, credit hours, prerequisites, academic years and semesters.	§5.1 #2, §3	
REQ-S05	Admin creates class schedules; students view them in the portal.	§5.1 #7, §3	

4.1.4 Students and academic profiles

ID	Requirement	Source	Notes
REQ-T01	A student record holds department, status and enrolment year.	§5.1 #3	Minimum stated attribute set.
REQ-T02	Admin registers students.	§3.1, §5.1 #3	"Registers" here means creating the student record — see GAP-16 on terminology.
REQ-T03	A student views their grade sheets by semester, GPA/CGPA and schedules.	§3	Announcements also listed — see GAP-19 (scope contradiction, resolved to Phase 2).
REQ-T04 **NEW**	A student who is granted admission and fails to register for two consecutive semesters **forfeits the admission**.	College 25 Aug 2026	CR-09. The system identifies candidates at semester close and reports them; an Admin confirms each forfeiture with a reason. No status changes automatically (Section 12.6).
REQ-T05 **NEW**	A graduated student retains **read-only** access to their academic history.	College 25 Aug 2026	Closes DEC-22. The account stays active; the ability to build a course plan does not.
REQ-A07 **NEW**	The Student ID is digits only; the **first four digits are the admission year** (e.g. 202634). Existing IDs are reused unchanged.	College 25 Aug 2026	CR-08. Total length 6–8 digits, so an intake larger than 99 students is representable. Pattern `^(19|20)\d{2}\d{2,4}$`.

4.1.5 Historical records import

ID	Requirement	Source	Notes
REQ-H01	Historical academic records are entered manually by an Admin, student by student.	§2.1, §6	No bulk import file is specified.
REQ-H02	A dedicated historical-entry screen, separate from normal grade entry, where an Admin selects a student and a past semester and enters course code, credit hours and grade.	§6.1	Note: credit hours are entered per record, not only inherited from the course definition.

ID	Requirement	Source	Notes
REQ-H03	Historical entries are flagged as imported records so it is always clear which data came from paper.	§6.1	Drives the `origin` column in Section 9.
REQ-H04	A per-student import status: Not started, In progress, Complete.	§6.1	Exactly three states.
REQ-H05	GPA/CGPA and any transcript view are labelled **provisional** until the student's import status is Complete.	§6.1	Propagates to every derived figure.
REQ-H06	A progress report showing how many students are fully entered.	§6.1	Institution-level operational view.
REQ-H07	The system holds a conflicting/unreadable record; it does not resolve the conflict.	§6.2	Explicitly a human responsibility.

4.1.6 Semester workflow

ID	Requirement	Source	Notes
REQ-W01	The semester lifecycle has six states: Draft → Open → Registration → In Progress → Grade Submission → Closed.	§7	Not a boolean flag.
REQ-W02	The state controls when students may register, when Admins may approve course plans, when grades may be entered, and when records become locked.	§7	State is the gate for four separate capabilities.
REQ-W03	Only a Super Admin can move a semester backwards or reopen a closed semester.	§7	Same as REQ-R07.
REQ-W04	Every backward or reopening transition is written to the audit log.	§7	Section 19 extends this to all transitions.

4.1.7 Course planning and approval

ID	Requirement	Source	Notes
REQ-P01	A student selects courses for the open semester and submits a course plan.	§8	
REQ-P02	The plan can be edited, and courses removed, until it is approved.	§8	Approval is the lock point.
REQ-P03	Validation runs **before submission and again before approval**.	§8	Two enforcement points, not one.
REQ-P04	Validation: prerequisites satisfied by completed, passed courses.	§8	"Passed" is undefined — GAP-02.
REQ-P05	Validation: minimum and maximum credit hours for the semester.	§8	Values undefined — OBD-02.
REQ-P06	Validation: courses already completed and passed are not repeatable unless flagged as a retake.	§8	Retake is an explicit flag on the plan item.
REQ-P07	Validation: no duplicate courses within the same plan.	§8	
REQ-P08	Validation: course availability in the current semester.	§8	An offering must exist and be open.
REQ-P09	Validation: schedule conflicts between selected classes.	§8	Requires meeting-time data — DER-14.
REQ-P10	Admin approves or rejects the plan; a rejection requires a reason, which the student sees in the portal.	§8	Reason is mandatory and student-visible.
REQ-P11	During the first semester of use, an Admin can override a failed prerequisite check with a recorded reason.	§8	Override is auditable and time-bounded — see GAP-11.

4.1.8 Grade management

ID	Requirement	Source	Notes
REQ-G01	An Admin enters grades from lecturer paper sheets.	§9, §5.1 #5	

ID	Requirement	Source	Notes
REQ-G02	A class grade-entry screen lists every registered student for the course offering, in an order that lets the Admin work down the paper sheet.	§9	Ordering is a stated requirement, not cosmetic — GAP-08 asks which order.
REQ-G03	Validation of student IDs and course codes; duplicate grade records rejected.	§9	Duplicate prevention is explicit.
REQ-G04	A printable class grade sheet and class list.	§9	Phase 1 deliverable.
REQ-G05	Draft grades are entered and edited by Admin only and are not visible to students.	§9	
REQ-G06	Once a class's grades are complete, Admin submits them for approval.	§9	Submission is per class, not per student.
REQ-G07	A Super Admin reviews and approves the submission, which publishes the grades to students and locks them.	§9	Approve = publish + lock, one atomic act.
REQ-G08	A correction to a locked grade is requested by Admin with a reason and takes effect only once a Super Admin approves it.	§9	
REQ-G09	The audit log keeps the old and new value, both actors, and the reason for every correction.	§9	

4.1.9 Grading scale, GPA and CGPA

ID	Requirement	Source	Notes
REQ-C01 **AMENDED**	**Grading scale (nine letters):** A+ 95–100 = 4.00; A− 90–94 = 3.70; B+ 85–89 = 3.30; B− 80–84 = 2.70; C+ 75–79 = 2.30; C− 70–74 = 1.70; D+ 65–69 = 1.30; D− 60–64 = 0.70; F below 60 = 0.00.	College 25 Aug 2026	**Supersedes the five-letter scale in v4.0 §10** (CR-01). Bands are five marks wide from 60 upward. Grade points carry two decimal places.
REQ-C02	The system calculates semester GPA, CGPA, credits attempted and credits earned **from approved grades**.	§10	Unapproved grades never enter a calculation.
REQ-C03	The Registrar's written grading policy is the authority for all GPA/CGPA rules.	§2.1	Policy document not yet supplied — DEC-01.
REQ-C04	When a course is repeated, both attempts are kept on the academic record.	§10.1	
REQ-C05	The most recent attempt is counted in CGPA; the earlier attempt is automatically dropped from CGPA.	§10.1	"Most recent", not "best" — see GAP-06.
REQ-C06	The earlier, dropped attempt is marked **R (Repeated)** on the student's record so it stays visible without affecting CGPA.	§10.1	R is a display marker, not a grade.
REQ-C07	A repeated course's credit hours count only once toward graduation requirements, however many times it was attempted.	§10.1	Affects credits earned, not credits attempted — GAP-07.
REQ-C08	Both attempts remain on the transcript; only the CGPA contribution changes.	§10.1	Transcript itself is Phase 2; the data rule is Phase 1.
REQ-C09 **NEW**	Grades are entered as numeric marks, decimals permitted. The system rounds and derives the letter — a mark of 96 must produce A+.	College 25 Aug 2026	CR-07. Half-up to a whole number once, before band lookup. The entered mark is stored verbatim.
REQ-C10 **NEW**	GPA and CGPA are reported to **three decimal places**.	College 25 Aug 2026	CR-03. Half-up, applied once at presentation.
REQ-C11 **NEW**	The minimum passing grade is **D**. An **F must be repeated**. **Any D earned in a major course must be repeated.**	College 25 Aug 2026	CR-05. Introduces the *major course* concept (Section 12.5) and a tracked mandatory-repeat obligation. A D still earns credit and still counts in GPA.
REQ-C12 **NEW**	**132 credit hours** are required for a BSc. A semester plan may not exceed **21 credit hours**; there is no meaningful minimum.	College 25 Aug 2026	CR-04. The graduation total is a displayed progress figure only — it gates nothing in Phase 1.

ID	Requirement	Source	Notes
REQ-C13 **NEW**	**Incomplete (I)** is used and appears on both the grade sheet and the transcript, carrying no academic weight until resolved. **Withdrawal (W) is not used** and must appear on neither.	College 25 Aug 2026	CR-02. Audit/non-credit and transfer credit are also not used in Phase 1.
REQ-C14 **NEW**	An Incomplete must be resolved **within one semester**. At the deadline the system reports; it does not convert the grade.	College policy 25 Aug 2026	CR-13. The deadline is shown to the student. Closure runs through the existing correction workflow — Admin requests, Super Admin approves — so no new permission is created.
REQ-C15 **NEW**	**Academic standing** is displayed, derived from CGPA: probation below 2.000, good standing 2.000–3.499, honours 3.500 and above.	College 25 Aug 2026	CR-14. Label only — gates nothing. Suppressed entirely while a CGPA is provisional.

4.1.10 Responsive design, security, audit, backup

ID	Requirement	Source	Notes
REQ-D01	The portal works on phones, tablets, laptops and desktops, with the layout adapting to screen size.	§2.1, §12	
REQ-D02	Grade sheets and schedules stay readable on a narrow screen without horizontal scrolling.	§12	Specific, testable.
REQ-D03	Pages are kept light so they load quickly on a slow connection.	§12	Needs a numeric budget — DER-25.
REQ-D04	Admin work (grade entry, scheduling, historical import) is designed around a full-size screen.	§12	
REQ-D05	The system must behave sensibly when a connection drops mid-action: no half-saved grade records and no duplicate submissions.	§12	Transactions + idempotency. Section 22.
REQ-E01	Every important action on grades and academic records is logged: who, what changed, which record, old and new value, and when.	§13	Six mandatory audit fields.
REQ-E02	The audit log is append-only and visible to the Super Admin.	§13	Append-only must be enforced technically — DER-20.
REQ-E03	Audit logging is recorded from day one for grades and academic records.	§5.1 #9	Not retrofitted at the end. Stage 1 deliverable.
REQ-B01	A full backup is taken at the end of every semester.	§14	
REQ-B02	A restore is tested at least once so the backup is known to work.	§14	Explicit verification requirement.
REQ-B03	End-of-semester full export of all academic data in a plain readable format (spreadsheet or CSV) openable without the system.	§14	Phase 1 deliverable.
REQ-B04	Signed paper grade sheets remain the physical fallback and continue to be filed.	§14	Process requirement; no software work.

Total explicit requirements captured: 72 — 62 from System Overview v4.0, plus ten added or amended by the College's decisions of 25 August 2026 (Section 1.6). Every one is traced to a stage, a component and a test in Section 36.

4.2 Derived technical requirements

These are not in the source document. They are introduced by the engineering team because an explicit requirement cannot be delivered correctly without them. Each states the requirement it serves, so the College can see that nothing here is decoration.

ID	Derived requirement	Serves	Why it is necessary
DER-01	An `app_user` identity record exists independently of role-specific profiles, holding role, account status, and a `must_change_password` flag.	REQ-A01…A04	Students, Admins and Super Admins authenticate through one mechanism but carry different profile data. A shared identity anchor prevents three parallel auth paths.
DER-02	Login accepts a Student ID (students) or a username (staff) and resolves it server-side to the internal identity before authenticating.	REQ-A01	Students have no institutional email. The identifier the user types is not the identifier the auth provider uses.
DER-03	Passwords are never stored or verified by application code; hashing is delegated to the managed auth provider (bcrypt/scrypt-class, per-user salt).	REQ-A02	Hand-rolled credential storage is the single most common critical defect in first-time systems of record.
DER-04	An Admin-issued or Admin-reset password is a single-use temporary credential; the session is restricted to the password-change screen until changed.	REQ-A03	"Required" must mean unbypassable, not "prompted on the dashboard".
DER-05	The user's role is carried in a signed session claim and is also the subject of a database policy lookup; it is never read from a client-supplied value.	REQ-A05	Otherwise authorisation can be defeated by editing a request body.
DER-06	All academic results — whether produced in-system or transcribed from paper — are stored in a **single unified academic-record table** with an `origin` discriminator.	REQ-H03, REQ-C02, REQ-P04	GPA, CGPA, prerequisite checking and the future transcript must all read one place. Two parallel tables guarantees two divergent answers to "what has this student passed?".
DER-07	The academic record stores the **credit hours and grade point in force at the time the result was recorded**, not a live reference to the course definition.	REQ-C01, REQ-H02	Course credit hours change over time. A historical GPA must not silently move when an Admin edits a course in 2029.
DER-08	All grade points, GPA and CGPA values use exact decimal arithmetic (Postgres `NUMERIC`), never binary floating point, at every layer including serialisation.	REQ-C02	Floating point produces 3.6699999 and disputes. This is a correctness requirement, not a style preference.
DER-09	Semester GPA, CGPA, credits attempted and credits earned are computed by a single pure function with no I/O, invoked both for on-demand display and for stored snapshots.	REQ-C02	One implementation means one place to test and one place to fix when the Registrar's policy is clarified.
DER-10	A derived `student_semester_summary` record is written transactionally whenever the underlying academic records for that student-semester change, and is treated as a cache, never as the source of truth.	REQ-C02, REQ-T03	Recomputing a full CGPA on every page view is acceptable at LCC's scale, but a stored snapshot makes the "provisional" state and its timestamp explicit and auditable.
DER-11	Approving a class grade submission writes the published grades into the academic record and the semester summary **inside one database transaction**.	REQ-G07, REQ-D05	A partial publish would show some students a grade and leave others' CGPA wrong.
DER-12	Approving a course plan creates the corresponding registration rows inside one transaction; the plan and the registrations can never disagree.	REQ-P10, REQ-G02	The grade-entry class list is generated from registrations; if approval half-succeeds, students disappear from grade sheets.
DER-13	Every mutating request carries a client-generated idempotency key; a repeated key returns the original result rather than performing the action twice.	REQ-D05	Directly answers "no duplicate submissions" on an unreliable connection.
DER-14	A course offering has zero or more structured meeting times (day of week, start time, end time, room).	REQ-P09, REQ-S05	Schedule-conflict detection is impossible against a free-text schedule string.
DER-15	A course offering carries an **instructor name as free text**, with no user account and no login.	REQ-G04, REQ-S05	A printable class grade sheet and a student timetable both need to name the lecturer. This does not create a lecturer role (OOS-01 is preserved).
DER-16	Registration is a first-class record linking a student to a course offering, created on plan approval, and is the object a grade attaches to.	REQ-G02, REQ-P10	"Every registered student for the course offering" presupposes this entity.

ID	Derived requirement	Serves	Why it is necessary
DER-17	Grade records progress through an explicit status: `draft → submitted → published → locked`, with correction handled as a separate request object.	REQ-G05…G08	The lifecycle in §9 is a state machine; modelling it as boolean flags produces unreachable and contradictory combinations.
DER-18	A class grade submission is a first-class object (one per course offering per attempt) that Super Admin approves or rejects as a unit, with a rejection reason.	REQ-G06, REQ-G07	Submission is per class; the reviewable artefact must therefore exist per class.
DER-19	Optimistic concurrency: every editable record carries a version token; a save against a stale token is rejected with a clear conflict message.	REQ-D05, REQ-G03	Two Admins entering the same class simultaneously must not silently overwrite each other.
DER-20	The audit log is protected by database privileges and policy: `INSERT` only; no `UPDATE` or `DELETE` grant exists for any application role.	REQ-E02	"Append-only" enforced by convention is not append-only.
DER-21	Audit entries are written in the same transaction as the change they describe.	REQ-E01	An audit written afterwards can be lost precisely when it matters — during a failure.
DER-22	Reference data (colleges, departments, courses, semesters, students) is **archived/deactivated, never hard-deleted**, once any academic record references it.	REQ-C08, REQ-E01	Deleting a course would orphan or rewrite history.
DER-23	Every list view that can exceed ~50 rows is paginated or virtualised server-side.	REQ-D03	Institution-wide student and audit views will otherwise become unusable on a slow connection within one semester.
DER-24	Print output (class grade sheet, class list) is produced by a dedicated print stylesheet against server-rendered HTML, not by a client-side PDF library.	REQ-G04	Keeps the page weight low and works on any device that can print.
DER-25	Performance budget: student-facing pages ≤ 150 KB of JavaScript transferred and interactive within 3 s on a 3G-class connection; admin pages ≤ 300 KB.	REQ-D03	Makes "kept light" a testable acceptance criterion rather than an aspiration.
DER-26	A bootstrap procedure creates the first Super Admin account outside the normal UI, once, and is recorded in the audit log.	REQ-A06	Super Admin manages Admins; nothing in the source creates the first Super Admin.
DER-27	Time is stored in UTC with an explicit institutional display timezone (Africa/Monrovia, UTC+0).	REQ-E01, REQ-W02	Audit timestamps and semester deadlines must be unambiguous.
DER-28	The academic record links to a semester, not to a free-text term label, including for imported records.	REQ-H02, REQ-C05	"Most recent attempt" requires a reliable chronological ordering of semesters.

4.3 Missing requirements

These must be defined before the affected code is written. Where the team can reasonably decide, a recommendation is given and the item is also carried into Section 34 as a decision with an owner.

ID	What is missing	Blocks	Recommended treatment and classification
GAP-01	**Score vs letter at entry.** §10 gives score ranges and §9 describes transcribing lecturer sheets, but never says whether the Admin types a numeric score (0–100) or a letter grade.	Stage 10	Enter the **numeric score**; the system derives and stores the letter and grade point. Preserves the lecturer's actual mark, makes boundary disputes auditable, and lets a policy change be re-derived. Must still accept letter-only entry for historical records where the paper shows only a letter. BLOCKING DECISION DEC-04
GAP-02	**Minimum passing grade.** §8 requires prerequisites "satisfied by completed, passed courses" and §10.1 refers to repeats, but "passed" is never defined. D = 1.0 = 60–69 may or may not satisfy a prerequisite.	Stage 7, 9	Recommend **D (60) is the minimum pass** for credit earned, with a separate, optionally higher **prerequisite threshold per course** (default C). Without this the prerequisite engine cannot be written. BLOCKING DECISION DEC-05

ID	What is missing	Blocks	Recommended treatment and classification
GAP-03	**Score boundary handling.** Ranges are integers (90–100, 80–89). Behaviour for a decimal mark such as 89.5 is undefined.	Stage 7	Recommend the entered score is a whole number 0–100; reject decimals at entry. If the College marks with decimals, define rounding *before* letter derivation. REQUIRES APPROVAL DEC-06
GAP-04	**GPA rounding.** §10.2 lists it as unconfirmed: decimal places and direction.	Stage 7	Recommend compute at full precision, display and store to **2 decimal places, half-up**. Never round intermediate values. REQUIRES APPROVAL DEC-07
GAP-05	**Incomplete (I), Withdrawal (W), audit/non-credit, transfer credit.** §10.2 lists all four as unconfirmed, including whether W counts as credits attempted and the deadline for resolving an I.	Stage 7	Each becomes a named policy flag on the grade scale (`counts_in_gpa`, `counts_in_attempted`, `counts_in_earned`) so the answer is configuration, not a rewrite. Defaults proposed in Section 16. BLOCKING DECISION DEC-08
GAP-06	**Repeat rule when the later attempt is worse.** §10.1 says the most recent attempt counts. If a student earns A then retakes and earns F, the stated rule lowers the CGPA.	Stage 7	Implement exactly as written (most recent counts) — but the College should confirm it understands this consequence, because "best attempt" is the more common policy and switching later requires recomputing every affected CGPA. REQUIRES APPROVAL DEC-09
GAP-07	**Repeats and credits attempted.** §10.1 fixes credits *earned* at once per course, but is silent on whether the dropped attempt still counts in credits *attempted* and in the semester GPA of the semester it occurred in.	Stage 7	Recommend: the dropped attempt **remains in that earlier semester's GPA and credits attempted** (semester GPA is a historical fact and should not change retroactively), and is excluded only from CGPA and credits earned. REQUIRES APPROVAL DEC-10
GAP-08	**Class list ordering.** §9 requires the entry screen to match the paper sheet order, but the paper sheet's order is not stated.	Stage 10	Recommend default alphabetical by surname, with the printed class sheet generated from the *same* ordering so screen and paper always agree; allow the Admin to switch to Student-ID order if that is how lecturers receive the sheet. RECOMMENDATION
GAP-09	**Student ID format.** §15 Q4 asks it as an open question. Nothing specifies length, character set, embedded year/cohort, or reuse of existing IDs.	Stage 2, 5, 6	Cannot be assumed. Affects login, validation, uniqueness, seeding and every historical import keystroke. BLOCKING DECISION DEC-02
GAP-10	**Which semesters exist per academic year.** §4 exemplifies "First Semester"; the source never states whether there is a second, a summer/intersession term, or how many.	Stage 4	Recommend modelling semesters as ordered rows within an academic year with a configurable sequence number and name, so two, three or more terms are supported without schema change. Confirm the actual calendar. REQUIRES APPROVAL DEC-11
GAP-11	**Duration of the prerequisite override.** §8 permits Admin override "during the first semester of use" but gives no mechanism for ending it.	Stage 9	Recommend an explicit, Super-Admin-controlled institution setting `prerequisite_override_enabled` with an expiry date, defaulting to the end of the first semester, so it cannot become permanent by inertia. REQUIRES APPROVAL DEC-12
GAP-12	**Scope of a semester's state.** §7 defines states but not whether the state is institution-wide or can differ per College/Department.	Stage 4	Recommend **institution-wide** state per semester for Phase 1 — simplest, matches a single Admin office, and avoids a combinatorial permission matrix. RECOMMENDATION DEC-13
GAP-13	**Definition of the "Open" state.** The list Draft → Open → Registration implies Open and Registration are different, but the difference is not described.	Stage 4	Recommend: **Open** = calendar and offerings published and visible, registration not yet permitted; **Registration** = students may build and submit plans. Section 13 specifies both fully. RECOMMENDATION
GAP-14	**Course plan vs. registration.** The source never states explicitly that approving a plan registers the student in the offerings, yet §9 refers to "every registered student for the course offering".	Stage 9, 10	Recommend plan approval is the sole automatic path to registration, plus an explicit auditable Admin direct-registration action for exceptions. Documented as DER-16 / DER-12. RECOMMENDATION
GAP-15	**Late registration, add/drop and withdrawal after approval.** §8 locks the plan at approval; nothing describes changing a registration afterwards.	Stage 9	Recommend Phase 1 supports an Admin-performed, audited **drop** and **add** against an approved plan while the semester is in Registration or In Progress, with a reason. Without it the office has no legitimate route and will resort to database edits. REQUIRES APPROVAL DEC-14

ID	What is missing	Blocks	Recommended treatment and classification
GAP-16	**Terminology collision on "registration".** The source uses "registers students" (creating a student record, §3) and "Registration" (a semester state for course enrolment, §7) for two different things.	All	Adopt and use consistently: **Student Enrolment** = creating the student record; **Course Registration** = enrolling a student in offerings. Glossary in Appendix A. No requirement change. RECOMMENDATION
GAP-17	**Who manages Super Admin accounts.** §3 says Super Admin manages Admin accounts. Nothing says who creates, disables or resets a Super Admin.	Stage 2	Recommend a Super Admin may manage other Super Admins, with a hard invariant that at least one active Super Admin always exists; the first is created by the bootstrap procedure (DER-26). REQUIRES APPROVAL DEC-15
GAP-18	**Student account lifecycle.** §5.1 mentions student "status" but does not enumerate the values or their effects on login and course planning.	Stage 5	Recommend: Active, Inactive/Withdrawn, Suspended, Graduated. Only Active may build a course plan; all non-deleted students retain read access to their own history. REQUIRES APPROVAL DEC-16
GAP-19	**Scope contradiction — announcements.** §3 lists announcements as a Student capability and "publishes announcements" as an Admin responsibility, but §5.2 places announcements in Phase 2.	Stage 11	Resolve in favour of §5.2 (the explicit phase table): **announcements are Phase 2.** Recorded so it cannot re-enter Phase 1 by reference to §3. Requires the College's confirmation. REQUIRES APPROVAL DEC-17
GAP-20	**Scope contradiction — reports.** §3 says Super Admin "views institution-wide records, reports and the audit log", but §5.2 places academic reports and dashboards in Phase 2.	Stage 11	Resolve as: Phase 1 gives Super Admin **institution-wide read access and the audit log**, plus the two operational reports the source explicitly requires elsewhere (historical-import progress §6.1, and the semester-end export §14). No dashboards, no analytics. REQUIRES APPROVAL DEC-18
GAP-21	**Grade visibility timing.** §15 Q6 asks whether students see grades when entered or only after approval — but §9 already states draft grades are not visible to students.	Stage 10	§9 is unambiguous and is treated as the requirement: **only after approval and publication.** Q6 is treated as a confirmation, not an open design choice, because answering it the other way would contradict REQ-G05 and defeat REQ-R05. REQUIRES APPROVAL DEC-19
GAP-22	**Batch vs individual grade approval.** §15 Q8. §9 describes class-level submission, which implies batch, but does not settle whether Super Admin must review each grade.	Stage 10	Recommend **batch approval of a class submission**, with the review screen showing every grade, a distribution summary and outlier flags, and the ability to reject the whole batch with a reason. Individual-grade approval would make the role unworkable at scale. BLOCKING DECISION DEC-20
GAP-23	**Minimum/maximum credit hours.** §15 Q2, including whether limits differ per department.	Stage 9	Model as an institution default with an optional per-department override, so either answer is configuration. Values required before plan validation can be tested. BLOCKING DECISION DEC-03
GAP-24	**Student personal data set.** Beyond department, status and enrolment year, no attribute list is given (name parts, contact, photograph).	Stage 5	Recommend a minimal Phase 1 set: legal name parts, Student ID, department, enrolment year, status, and one contact field. Any additional personal data should be justified against a use, not collected by default. REQUIRES APPROVAL DEC-21
GAP-25	**Data retention and account closure.** Nothing states how long records are kept or what happens to a graduated student's login.	Stage 11	Recommend academic records are retained indefinitely (they are the institutional record); graduated students retain read-only login. Confirm with the College. REQUIRES APPROVAL DEC-22
GAP-26	**Course offering capacity.** Not mentioned anywhere. Course planning validation lists availability but not seat limits.	Stage 8, 9	Recommend including a nullable capacity field, enforced only when set, so the College can adopt it without a schema change. Not a Phase 1 workflow requirement. RECOMMENDATION
GAP-27	**Language, locale and academic year label format.** "2026/2027" is exemplified but not constrained.	Stage 4	Recommend English (en-LR), `YYYY/YYYY` label validated by pattern, dates displayed `DD Mon YYYY`, times 24-hour, timezone Africa/Monrovia. RECOMMENDATION

4.4 Open business decisions

Every open question from the source document is carried forward here without loss. §15 lists eight; §6.2 adds three; §10.2 adds six rules; §11 and §14 add two more. All are reproduced, mapped to the decision register, and none is resolved unilaterally.

ID	Question (as posed by the source)	Source	Register	Effect if unanswered
OBD-01	Treatment of Incomplete and Withdrawal grades in GPA.	§15 Q1, §10.2	DEC-08	GPA engine cannot be finalised or tested. Blocks Stage 7 exit.
OBD-02	Minimum and maximum credit hours per semester, and per department if different.	§15 Q2	DEC-03	Course-plan validation has no thresholds. Blocks Stage 9.
OBD-03	How far back historical records will be entered, and who will enter them.	§15 Q3, §6.2	DEC-23	Import volume, staffing and go-live date are unknown. Blocks operational scheduling, not the screen.
OBD-04	Student ID format, and whether existing IDs will be reused.	§15 Q4	DEC-02	Login, validation and seeding cannot be built. Blocks Stage 2.
OBD-05	Transcript issuance process: who signs, and whether a fee or request form applies.	§15 Q5	DEC-24	Phase 2 concern. Non-blocking for Phase 1; recorded so it is not lost.
OBD-06	Whether students may see grades as soon as they are entered, or only after Super Admin approval and publishing.	§15 Q6	DEC-19	Contradicts §9 if answered "as entered". Requires explicit confirmation.
OBD-07	Whether transcripts should carry an electronic signature, or be signed by hand on the printed copy.	§15 Q7, §11	DEC-25	Phase 2. Only architectural implication is keeping a document-issuance record model open.
OBD-08	Whether Super Admin's grade approval reviews every grade individually or approves a class submission as a batch.	§15 Q8	DEC-20	Determines the approval UI and the grade-record status model. Blocks Stage 10.
OBD-09	What happens when a paper record is missing, unreadable, or contradicts another record.	§6.2	DEC-26	The system will hold the record; the College must define who adjudicates and how the outcome is recorded.
OBD-10	Who does the historical data entry, and how many hours per week are available.	§6.2	DEC-23	Determines whether import completes before or long after go-live.
OBD-11	Audit / non-credit courses: shown on records but excluded from GPA.	§10.2	DEC-08	Grade-scale configuration. Same blocking effect as OBD-01.
OBD-12	Transfer credits: counted toward credits earned; included in or excluded from CGPA.	§10.2	DEC-08	Also determines whether a transfer record needs a source-institution field.
OBD-13	Rounding: decimal places and direction for GPA and CGPA.	§10.2	DEC-07	Two systems can disagree on a student's honours standing over one hundredth of a point.
OBD-14	Standing: thresholds for probation, good standing and honours, if the College wants them displayed.	§10.2	DEC-27	Optional. If declined, no standing is displayed anywhere in Phase 1.
OBD-15	Whether to add an automatic weekly database backup in addition to the semester-end backup.	§14	DEC-28	Offered at no additional development cost. Strongly recommended — see Section 29.

4.5 Technical decisions

These the engineering team can reasonably make. They are listed so that they are visible and reviewable, not so that they are debated.

ID	Decision	Position
TEC-01	Application framework and hosting	Next.js (App Router, TypeScript) on Vercel. Section 7.
TEC-02	Database	Supabase-managed PostgreSQL as the sole system of record. Section 7.

ID	Decision	Position
TEC-03	Data access pattern	All writes and all academic reads go through server-side code. The browser holds no database credential capable of writing academic data. Row-Level Security is the second line of defence, not the first. Section 8.
TEC-04	Student-ID login mechanism	Supabase Auth with a deterministic internal identifier derived from the Student ID; the login form accepts the Student ID and the server resolves it. Section 18.3.
TEC-05	Schema migration tool	Versioned, checked-in SQL migrations applied through a single forward-only pipeline; every migration reviewed as code. Section 30.
TEC-06	Validation	One schema definition per input, shared between client and server, with the server always re-validating. Section 21.
TEC-07	UI system	Utility-first CSS with a small, accessible headless component library; no heavy design framework. Section 7.
TEC-08	Test framework layering	Unit (GPA engine, validators), integration (database + policies), end-to-end (workflows and forbidden actions). Section 23.
TEC-09	Numeric representation	`NUMERIC` in the database; a decimal library in application code; no floats anywhere in the grade path. DER-08.
TEC-10	Error monitoring	A hosted error tracker with source maps and release tagging; alerts routed to the developer. Section 30.
TEC-11	Version control	Git, trunk-based with short-lived branches, one branch per stage, protected main, mandatory review before merge. Section 30.
TEC-12	Audit implementation site	Written by the server-side service layer in the same transaction, not by database triggers alone — because the actor, reason and business intent are only known to the application. Section 19.

4.6 Out-of-scope requirements

Reproduced from §2.2 and §5.3. These must not appear in any Phase 1 stage, deliverable or acceptance criterion. Section 31 explains how the architecture stays open to them without building them.

ID	Excluded from Phase 1	Source	Guard against re-entry
OOS-01	Lecturer accounts and lecturer grade submission	§2.2, §5.3	DER-15 provides an instructor *name* only. No lecturer role, no lecturer login, no lecturer-facing screen appears in Section 20.
OOS-02	Fees, payments, financial clearance and online clearance	§2.2, §5.3	No money concept in the domain model. Registration is never gated on payment in Phase 1.
OOS-03	Attendance and examination management	§2.2, §5.3	Grade records hold a final result only; no component/assessment breakdown is modelled.
OOS-04	Admissions and online application	§2.2, §5.3	Students enter the system only by Admin enrolment.
OOS-05	Digital or QR-based signature and verification	§2.2, §5.3, §11	No signing, no verification endpoint. Print output carries no machine-verifiable mark.
OOS-06	SMS and email notifications	§2.2, §5.3	No outbound messaging service is provisioned. Rejection reasons and grade publication are seen in-portal only.
OOS-07	Transcript generation and PDF download	§5.2, §11	Phase 2. Phase 1 builds the academic record the transcript will read, and nothing more.
OOS-08	Announcements, events and in-portal notifications	§5.2	Phase 2 — see GAP-19 for the source contradiction and its resolution.
OOS-09	Role-specific dashboards and academic reports	§5.2	Phase 2 — see GAP-20. Phase 1 landing pages are navigation, not analytics.
OOS-10	Document management	§5.3	No file upload or document store in Phase 1.

ID	Excluded from Phase 1	Source	Guard against re-entry
OOS-11	Financial clearance as a registration precondition	§2.1	Remains a manual Business Office process, off-system.

5. Phase 1 Scope Boundary

The scope below is the contract for Phase 1. Anything not listed as In Scope or Supporting Infrastructure does not get built, even if it is small, even if it seems obvious, and even if a user asks for it mid-build. Section 26 makes scope confirmation part of every stage gate.

5.1 In scope

#	Component	What is included	Why it belongs in Phase 1
1	Authentication, users and role-based access	Student-ID + password login for students; credentialled login for Admin and Super Admin; Admin-issued and Admin-reset passwords; forced first-login password change; three roles with backend- and database-enforced permissions; Super Admin management of Admin accounts.	Nothing else can exist without identity. Every other requirement is expressed in terms of "who may do this". Source §5.1 #1.
2	Academic setup	Colleges, Departments, Courses (with credit hours and prerequisites), Academic Years, Semesters.	These are the reference data every academic record points at. Grades and plans are meaningless without them. Source §5.1 #2.
3	Student enrolment and academic profiles	Creating and editing the student record: Student ID, name, department, status, enrolment year, contact.	The subject of every academic record. Source §5.1 #3.
4	Historical record import	Dedicated per-student, per-past-semester entry screen; imported-record flag; per-student import status (Not started / In progress / Complete); provisional labelling of derived figures; institution-wide progress report.	Named in the source as the largest work item and the main schedule risk. GPA, CGPA, prerequisite checking and future transcripts all depend on it. Source §5.1 #4, §6.
5	Grade management	Class grade-entry screen; validation and duplicate rejection; draft grades invisible to students; class submission for approval; Super Admin approval that publishes and locks; correction request and approval; printable class grade sheet and class list.	The core purpose of Phase 1. Source §5.1 #5, §9.
6	GPA / CGPA calculation	Semester GPA, CGPA, credits attempted, credits earned, computed from approved grades using the College's scale and carry-over rule; provisional marking where history is incomplete.	The reason grades are digitised at all. Source §5.1 #6, §10.
7	Semester and course schedules	Course offerings/classes per semester with section, instructor name and structured meeting times; Admin creates them; students view their timetable.	Required for schedule-conflict validation in course planning and for the student's semester view. Source §5.1 #7.
8	Course planning and Admin approval	Student builds and submits a plan; six validation rules run before submission and again before approval; Admin approves or rejects with a mandatory, student-visible reason; approval locks the plan and creates registrations; time-bounded prerequisite override.	The second of the two things the College asked Phase 1 to get right. Source §5.1 #8, §8.
9	Audit logging	Append-only log of every important action on grades and academic records, capturing actor, action, entity, record, old value, new value, reason and timestamp; Super Admin view.	Explicitly required "from day one", not retrofitted. It is also the only evidence that segregation of duties held. Source §5.1 #9, §13.
10	Semester lifecycle control	Six-state semester machine governing registration, plan approval, grade entry and record locking; Super-Admin-only backward and reopening transitions; all transitions audited.	Source §7 defines it as a requirement in its own right; every other workflow reads its state as a precondition.
11	Semester-end academic data export	Full export of academic data as CSV/spreadsheet, openable without the system.	Explicit requirement in §14 and the College's stated safety net. Small work, high value.

#	Component	What is included	Why it belongs in Phase 1
12	Responsive delivery	Mobile-first student surfaces; desktop-optimised admin surfaces; readable grade sheets and schedules on narrow screens; light pages; no half-saved records or duplicate submissions on connection loss.	Stated requirements in §2.1 and §12 with concrete, testable consequences.

5.2 Supporting infrastructure

Technical capabilities Phase 1 cannot work without, but which no user would describe as a feature. They are listed separately so they are neither forgotten in estimation nor mistaken for scope creep.

Capability	Content	Supports
Environment topology	Local development, a shared testing/staging environment with its own database, and production. Separate credentials and separate data in each.	Prevents testing against live academic records. Section 30.
Schema migration pipeline	Versioned, forward-only, checked-in SQL migrations applied identically in every environment.	Database integrity across eleven stages of schema evolution.
Authorisation kernel	A single server-side permission-check module plus database row-level policies, both derived from one permission table.	REQ-A05, REQ-R01–R08. Prevents divergence between the two enforcement layers.
Transaction and idempotency layer	A standard wrapper for multi-row writes: transaction, idempotency key, optimistic-concurrency check, audit write.	REQ-D05, DER-11, DER-12, DER-13, DER-19, DER-21.
Audit service	One function every mutating service calls, in-transaction, with a typed action vocabulary.	REQ-E01–E03. Ensures no code path can change a grade without logging it.
Reference-data seeding	Repeatable seed of the grade scale, roles and permission table; a separate, clearly marked demo dataset for the test environment only.	Reproducible environments and test fixtures.
Backup and restore procedure	Automated database backup, a written restore runbook, and at least one rehearsed restore.	REQ-B01, REQ-B02. Section 29.
Error monitoring and logging	Hosted error tracking with release tagging; structured server logs that never contain credentials or full grade payloads.	Operability with no dedicated system administrator.
Automated test harness and CI	Test runner, database fixtures, and a pipeline that runs unit, integration and end-to-end suites on every change.	Section 23. Without it, eleven stages of change will silently break Stage 7's arithmetic.
Print stylesheet	Server-rendered print layouts for the class grade sheet and class list.	REQ-G04.
Bootstrap procedure	One-time creation of the first Super Admin, audited.	DER-26. Without it the system cannot be started.

5.3 Out of scope

ID	Excluded	Deferred to	Why it is excluded
OOS-07	Transcript generation and PDF download	Phase 2	The source makes it conditional on the historical import being complete. Building it before the data exists would produce a document that is confidently wrong.
OOS-08	Announcements, events, in-portal notifications	Phase 2	Explicit in §5.2. Not needed for grades or course planning to be correct. See GAP-19 for the source contradiction.
OOS-09	Role-specific dashboards and academic reports	Phase 2	Explicit in §5.2. Phase 1 landing pages provide navigation and status only. See GAP-20.

ID	Excluded	Deferred to	Why it is excluded
OOS-01	Lecturer accounts and lecturer grade submission	Later	The College's process keeps lecturers on paper. Adding accounts would change the operating model, not just the software.
OOS-03	Attendance and examination management	Later	Requires a component-level assessment model that Phase 1 deliberately does not have.
OOS-04	Admissions and online application	Later	A separate business process with its own approvals; students enter Phase 1 by Admin enrolment.
OOS-02 / OOS-11	Fees, payments, financial clearance, online clearance	Later	Remains with the Business Office off-system per §2.1. Introducing money would make registration dependent on a system the College has not asked for.
OOS-10	Document management	Later	No file storage need in Phase 1; adding it introduces a whole class of security and retention obligations.
OOS-06	SMS and email notifications	Later	Requires a provider, deliverability handling and a contact-data quality programme. In-portal visibility satisfies Phase 1.
OOS-05	Digital / QR-based signature and verification	Later	Open question for the College (§15 Q7) and only meaningful once transcripts exist.

SCOPE GUARD

Three items are at high risk of drifting into Phase 1 because they feel small: a "quick" announcements banner, a "simple" dashboard chart, and a "just a print button" transcript. Each is explicitly excluded above. The Stage exit gate in Section 26 requires the developer to confirm that no out-of-scope item was implemented, and Section 36's traceability matrix contains no row that would justify one.

5.4 Scope boundaries that are easy to misread

In Phase 1	Not in Phase 1	The line between them
Printable class grade sheet and class list	Printable transcript	Both are print output, but the transcript is the student's whole cross-semester history and depends on the import being complete. The class sheet is one class, one semester, and the office needs it immediately.
Storing both attempts of a repeated course and applying the carry-over rule to CGPA	Displaying a transcript that shows both attempts	The *rule* is Phase 1 because CGPA is Phase 1. The *document* is Phase 2.
Super Admin sees institution-wide academic records and the audit log	Super Admin dashboards, charts, analytics	Read access to records is a permission. Reporting is a feature. Only the permission is in Phase 1.
Instructor name on a course offering	Lecturer accounts	A name is a data field on an offering. An account is an identity, a role, a permission set and a set of screens.
Historical-import progress report	Academic reports	The progress report is named as a requirement in §6.1 and is an operational necessity for running the import. It reports on data-entry state, not on academic outcomes.
Semester-end CSV export of academic data	Ad-hoc report builder	The export is one fixed, documented output required by §14. Anything parameterised is Phase 2.

6. Risks and Critical Success Factors

This section identifies where the project is most likely to fail *before* any architecture is proposed, so that the architecture can be judged against it. The full quantified register — with prevention, detection, mitigation, owner and approval requirement — is Section 28.

6.1 The four critical success factors

#	Factor	Why the project fails without it
1	**The confirmed grading rules are implemented exactly, not approximately** — was: obtain the rules	The rules are now supplied in full (Section 1.6). The risk has moved: **every one of them differs from the industry default** — nine letters not five, five-mark bands, two-decimal grade points, three-decimal GPA, most-recent rather than best repeats, mandatory repeats on a major-course D. Code written from habit rather than from this document will be wrong in at least five places and will look entirely plausible. The forty-seven fixtures in Appendix B exist to catch exactly that.
2	**Incompleteness stays visible for as long as it lasts — which is most of year one**	The College will enter historical records gradually after go-live (CR-11), so nearly every continuing student's CGPA will be provisional for months. The danger is not the missing data; it is a confident number computed from missing data. Provisional marking is therefore a primary feature of this system, not a corner case — and the College telling students what it means is as important as the software displaying it.
3	**Segregation of grade entry and grade approval holds at the data layer**	It is the only structural control against an unchecked grade change. If it exists only in the UI, the College's stated reason for having a Super Admin evaporates.
4	**Phase 1 stays Phase 1**	A solo developer with an AI agent can produce a great deal of code quickly. That is precisely the condition under which Phase 2 features arrive early, half-finished, untested, and entangled with the core.

6.2 Highest-risk areas, ranked before architecture

#	Area	Prob.	Impact	Severity	Primary mitigation designed into this plan
1	GPA / CGPA accuracy	High	Critical	CRITICAL	GPA is a pure, isolated domain service (DER-09); exact decimal arithmetic, now carrying two-decimal grade points and three-decimal results (DER-08); every rule is versioned configuration rather than a hard-coded assumption; **40 worked fixtures** in Appendix B become the regression suite; Stage 7 cannot exit until every fixture passes exactly and the Registrar countersigns the plain-language rules document.
2	Historical academic record entry	High	Critical	CRITICAL	Single unified academic-record table with `origin` flag (DER-06); frozen credit hours and grade points per record (DER-07); three-state per-student import status; provisional propagation to every derived figure (REQ-H05); duplicate detection on (student, semester, course); conflicts held, never auto-resolved (REQ-H07); progress report as a first-class screen.
3	Grade approval and locking integrity	Medium	Critical	CRITICAL	Explicit grade status machine (DER-17); approval publishes and locks in one transaction (DER-11); locked grades are immutable except through an approved correction request; role checks in both the service layer and RLS; forbidden-action test suite that asserts a Super Admin session *cannot* write a grade.
4	Role permission failure	Medium	Critical	CRITICAL	Non-hierarchical, explicit allow-list permission model (Section 11); one permission table drives both the service kernel and the database policies; every "No" cell in the permission matrix becomes an automated negative test.

#	Area	Prob.	Impact	Severity	Primary mitigation designed into this plan
5	Ambiguous requirements resolved by silent assumption	Low was High	High	MEDIUM	All nine blocking decisions are answered and implemented. Three assumptions still carry real consequence and are named as such: ASM-20 (what "major course" means), ASM-21 (how "two inactive semesters" is counted) and ASM-19 (most-recent repeats). Each has a stated validation route and a small fix if caught early.
6	Grade correction abuse or loss of history	Medium	High	HIGH	Correction is a request object, never an edit; old and new values, both actors and the reason are captured in-transaction; audit log is INSERT-only at the privilege level (DER-20).
7	Course prerequisite errors	Medium	High	HIGH	Prerequisites evaluated only against the unified academic record; explicit passing threshold of 0.70, now confirmed (CR-05); time-bounded, audited override rather than a permanent bypass (GAP-11); validation at both submission and approval (REQ-P03).
8	Semester state transition errors	Medium	High	HIGH	State machine with an explicit legal-transition table (Section 13); every capability check reads the state; backward transitions restricted to Super Admin and audited; illegal transitions rejected at the service layer and constrained in the database.
9	Data integrity — duplicates and partial writes	Medium	High	HIGH	Unique constraints as the last line of defence, not the only one; transactions around every multi-row operation; idempotency keys; optimistic concurrency tokens. Section 22.
10	Backup failure / data loss	Low	Critical	HIGH	Managed Postgres with point-in-time recovery; a rehearsed restore before go-live (REQ-B02); semester-end human-readable export (REQ-B03); recommendation to accept the no-cost weekly backup offered in §14 (DEC-28).
11	Scope creep into Phase 2	High	Medium	HIGH	Explicit out-of-scope register; scope confirmation in every stage exit gate; a traceability matrix with no row that could justify a Phase 2 feature.
12	Audit log incompleteness	Medium	High	HIGH	Audit written by a single service in the same transaction as the change (DER-21, TEC-12); a typed action vocabulary so a new action cannot be added without registering it; audit assertions inside workflow tests, not as a separate afterthought.
13	Connection loss producing duplicate or half-saved grades	Medium	Medium	MEDIUM	Idempotency keys on every mutation; class grade entry saved as an all-or-nothing batch; explicit conflict messages rather than silent overwrite.
14	Deployment / environment failure	Low	High	MEDIUM	Three separated environments; forward-only reviewed migrations; documented rollback that distinguishes reverting code from reverting schema; go-live checklist in Section 30.
15	Solo-delivery key-person risk	Medium	High	HIGH	This document itself; everything in version control; no undocumented environment steps; runbooks for backup, restore and bootstrap. A second person must be able to continue from the repository and this plan alone.

6.3 Risks specific to an AI-assisted solo build

The delivery model — one developer directing an AI coding agent — introduces failure modes that a conventional plan would not address. They are called out here because the mitigations shaped the structure of Sections 24 to 27.

Failure mode	How it shows up on this project	Mitigation built into the plan
Plausible-but-wrong domain logic	An agent will implement the *common* rules rather than LCC's, because the common rules dominate its training data. LCC differs on five points at once: nine letters not five, five-mark bands, two-decimal grade points, three-decimal GPA, and most-recent rather than best-attempt repeats.	Section 16 states the rules explicitly and Appendix B gives numeric fixtures with expected outputs. The fixtures are written before the engine and are the acceptance criterion for Stage 7.

Failure mode	How it shows up on this project	Mitigation built into the plan
Silent scope expansion	Asked for a student grade page, an agent adds a chart, an announcements panel and a "download transcript" button because those are what such pages usually contain.	Section 20 specifies each screen's exact contents and actions; the stage exit gate requires an explicit scope check; Section 5.3 names the three most likely intruders.
Authorisation by convention	Generated code checks the role in the page component and omits it in the server action, because the page "already" checks.	Section 11 mandates a single permission kernel that every mutation calls; Section 23 requires a forbidden-action test per "No" cell before a stage can close.
Floating-point arithmetic	Default numeric handling in JavaScript is IEEE-754. A GPA of 3.67 becomes 3.6699999999999995 in a JSON payload.	DER-08 and TEC-09 make exact decimal arithmetic a hard rule across database, application and serialisation, with a test that asserts exact string equality.
Migration drift	Schema changed directly in the hosted database console during debugging; the checked-in migrations no longer describe production.	TEC-05: forward-only migrations are the only permitted change path; the stage gate includes verifying that a fresh database built from migrations matches the deployed schema.
Review fatigue	A large volume of generated code arrives faster than one person can read it, so review degrades to "does it run".	Stages are sized so that each produces a reviewable increment; the Definition of Done (Section 27) is a checklist, not a judgement call; the highest-risk logic (GPA, authorisation, transactions) is deliberately concentrated in small, isolated modules that are worth reading line by line.

7. Technology Stack Recommendation

Technology is chosen against this project's actual requirements, not against popularity. Six of those requirements do most of the work in the evaluation, so they are stated first and every recommendation below is judged against them.

7.1 The requirements that decide the stack

#	Requirement	What it eliminates or demands
1	Permissions enforced at backend **and database** level (REQ-A05)	Demands a database with real row-level authorisation, or a backend that is the only path to data. Eliminates any architecture where the browser queries the database directly with a user-scoped key and no policy layer.
2	Correct decimal arithmetic for GPA (DER-08)	Demands exact numeric types end to end. Eliminates document stores that serialise numbers as doubles.
3	Multi-row atomic operations — publish a class, approve a plan (DER-11, DER-12)	Demands genuine ACID transactions across related tables. Eliminates eventually-consistent stores.
4	Relational academic data with referential integrity (Section 22)	Demands foreign keys, unique constraints and check constraints. A relational database is not a preference here; it is the requirement.
5	Light pages on a slow connection (REQ-D03, DER-25)	Demands server rendering with minimal client JavaScript. Eliminates heavy single-page-application shells.
6	Solo developer, JS/TS skill base, no dedicated sysadmin (Section 3.5)	Demands one language across the stack and managed infrastructure. Eliminates polyglot architectures and self-managed servers.

7.2 Recommendation summary

Layer	Recommended	Approval required?	One-line rationale
Frontend	Next.js (App Router) + React + TypeScript	Confirmed by owner	Server-rendered by default; one language with the backend; minimal client JS.
Backend	Next.js Server Actions and Route Handlers (Node runtime) — one deployable	Confirmed by owner	A separate API service would add operational surface a solo team cannot justify.
Database	PostgreSQL, managed by Supabase	Confirmed by owner	ACID, exact NUMERIC, constraints, row-level security — every hard requirement in one engine.
Authentication	Supabase Auth (email/password primitive) with server-side Student-ID resolution	YES — TEC-04 mechanism	Delegates password hashing and session issuance; the Student-ID mapping is the only custom part.
Authorisation	Server-side permission kernel (primary) + Postgres RLS (defence in depth)	REVIEW	Directly implements REQ-A05's "backend *and* database policies".
Data access	Typed SQL query builder with checked-in SQL migrations (Drizzle or equivalent)	REVIEW	Keeps SQL visible and reviewable — essential when an AI agent writes the queries.
API architecture	RPC-style server actions organised as a service layer; no public REST/GraphQL surface in Phase 1	REVIEW	No third-party consumer exists; a public API would be scope and attack surface without a user.
Validation	One schema per input (Zod or equivalent), shared client/server, server always authoritative	Team decision	Single definition prevents client and server rules drifting apart.

Layer	Recommended	Approval required?	One-line rationale
Styling / UI	Tailwind CSS + a headless accessible component primitive set	Team decision	No runtime CSS-in-JS cost; dense admin tables and mobile student views from one system.
Testing	Vitest (unit/integration) + Playwright (end-to-end) + a disposable Postgres for integration tests	Team decision	Policy and transaction behaviour must be tested against real Postgres, not a mock.
Deployment	Vercel, three environments, Git-triggered	Confirmed by owner	Zero-maintenance hosting; preview deployments per branch aid solo review.
Database hosting	Supabase — one project per environment	`YES` — cost/plan	Separate projects, not separate schemas, so a test mistake cannot touch production data.
Error monitoring	Sentry (or equivalent) with release tagging and source maps	Team decision	No dedicated sysadmin; failures must find the developer rather than wait to be found.
Version control	Git + GitHub, protected main, one branch per stage, mandatory review	Team decision	Also the mechanism for the stage gates in Section 26.
Backup	Managed daily backup + point-in-time recovery + semester-end CSV export	`YES` — DEC-28, plan tier	Removes the one-semester data-loss window the source accepts in §14.

7.3 Major decisions in full

7.3.1 Database engine

Field	Detail
Recommended	**PostgreSQL** (Supabase-managed)
Why	Every one of the six deciding requirements maps to a native Postgres capability: ACID transactions across tables; `NUMERIC` exact decimals for grade points and GPA; foreign keys, unique and check constraints for the integrity rules in Section 22; row-level security policies to satisfy the "database policies" half of REQ-A05; and mature point-in-time recovery for REQ-B01/B02. It also carries no per-seat cost and is portable — if the College ever wants to self-host on campus, the data moves with a standard dump.
Alternatives considered	**MySQL/MariaDB** — capable, but no first-class row-level security, so the database half of REQ-A05 would have to be simulated with views and application-managed session variables. **SQLite** — excellent for a single-node deployment but incompatible with Vercel's stateless functions and offers no concurrent multi-writer story. **MongoDB** — eliminated on requirements 2, 3 and 4: no exact decimals by default, weaker relational integrity, and the academic domain is intrinsically relational. **SQL Server / Oracle** — licensing and operational weight unjustifiable at this scale.
Trade-offs	Supabase's managed Postgres ties the project to one vendor's console, connection pooler and auth schema. Mitigated by keeping all schema in checked-in SQL migrations and all business logic in application code rather than vendor-specific features, so the database itself remains standard Postgres and portable.
Approval	Confirmed by project owner. No further approval required, other than the plan tier (see 7.5).

7.3.2 Application framework and topology

Field	Detail
Recommended	**A single Next.js application** containing both the rendered UI and the server-side service layer, deployed to Vercel.
Why	One language, one repository, one deployment, one set of types shared between the form and the database row. For a solo developer, every additional service is a permanent tax on every change. Server components render the heavy admin tables on the server and ship almost no JavaScript, which is the direct answer to REQ-D03. Server Actions give a natural place to put the transaction/idempotency/audit wrapper described in Section 21.

Field	Detail
Alternatives considered	**Separate Node API (NestJS/Express) + SPA** — cleaner service boundary and easier to reuse later for a mobile app, at the cost of two deployables, CORS, duplicated types and roughly 30–40% more integration work. Rejected for Phase 1; Section 31 explains how the service layer is structured so this split remains possible later without rewriting business logic. **Laravel** or **Django** — both are strong fits for this domain and have excellent admin ergonomics, but both are outside the stated skill base, and on a solo build unfamiliarity converts directly into defects in transactions and authorisation. **Remix/SvelteKit** — comparable technically; Next.js chosen for ecosystem depth and first-class Vercel support.
Trade-offs	Vercel's serverless model means no long-running background workers and cold starts on infrequently used routes. Neither matters in Phase 1: there is no scheduled processing, and the semester-end export can run as a request-scoped streaming download. It also means database connections must go through a pooler — a configuration detail, but one that must be got right before Stage 1 exits.
Approval	Confirmed by project owner.

7.3.3 Authentication mechanism

Field	Detail
Recommended	**Supabase Auth** for credential storage and session issuance, with a **server-side Student-ID → identity resolution step** in front of it. The login form asks a student for their Student ID; the server looks up the corresponding identity and completes authentication. Students never see or need an email address.
Why	REQ-A01 requires Student-ID login; REQ-A02/A03 require Admin-issued, Admin-resettable credentials with a forced first change. Delegating hashing, session tokens and rotation to the managed provider removes the highest-consequence category of hand-written security code (DER-03). The only bespoke element is a lookup, which is small, testable and auditable.
Alternatives considered	**Fully custom auth** (own users table, Argon2, own session cookies) — maximum control and no synthetic identifiers, but places password hashing, session fixation, timing attacks and reset-token handling in the hands of a solo developer. Rejected on risk. **Auth.js / NextAuth with a credentials provider** — a reasonable middle path and worth revisiting if Supabase Auth's email-shaped identifier proves awkward; noted as the fallback. **Third-party identity provider (Google/Microsoft)** — eliminated because students do not have institutional accounts.
Trade-offs	Supabase Auth models an identifier in email form. A deterministic non-routable identifier derived from the Student ID (for example `<student-id>@students.lcc.invalid`) satisfies this without implying that mail can be sent there. Two consequences must be designed for deliberately: (a) it must be impossible for that address to be used as a real contact channel or password-reset route, and (b) if the College ever changes a Student ID, the identity mapping must be updated in one place. Both are addressed in Section 18.3.
Approval	REQUIRES APPROVAL — DEC-29. The College should be told plainly that a synthetic, non-deliverable identifier will exist internally, and confirm that students will only ever be asked for their Student ID.

7.3.4 Where authorisation is enforced

Field	Detail
Recommended	**Two layers, one source.** A server-side permission kernel is the primary gate: every service call passes through `assertCan(actor, action, resource)` before any data is touched. Postgres row-level security is the second gate, so that even a defective or forgotten check cannot expose or alter another student's record. Both are generated from the single permission table in Section 11.
Why	REQ-A05 explicitly requires enforcement in "the backend and database policies". Two independent layers derived from one definition gives real defence in depth without the classic failure of two hand-maintained rule sets drifting apart.
Alternatives considered	**RLS only**, with the browser holding a user-scoped key and querying Supabase directly — fewer moving parts, but pushes complex academic rules (semester state, plan lock, grade status) into SQL policies where they are hard to read and harder to test, and makes every future rule change a database migration. Rejected. **Backend only**, with the database accessed by a single privileged role — simpler, but leaves one forgotten check as a total compromise, and does not satisfy the letter of REQ-A05. Rejected.
Trade-offs	Writing and testing policies costs real time in Stage 1 and Stage 2, and policies must be kept in step with schema changes. This is accepted: it is the difference between a system that has permissions and a system that appears to.

Field	Detail
Approval	RECOMMENDATION — project owner review.

7.3.5 Data access layer

Field	Detail
Recommended	**A typed SQL-first query builder** (Drizzle or equivalent) with hand-written, checked-in SQL migrations.
Why	Three project-specific reasons. First, an AI agent writes much of the query code, and SQL-shaped queries are far easier for a human reviewer to check than a deeply nested ORM expression. Second, row-level security, check constraints and partial unique indexes are first-class in SQL and awkward in ORM migration DSLs. Third, the GPA and prerequisite queries are the correctness-critical part of the system and deserve to be read as SQL.
Alternatives considered	**Prisma** — better developer ergonomics and a strong schema language, but its migration layer is less comfortable with RLS policies and custom constraints, and generated queries are harder to reason about at the point where accuracy matters most. **Raw `pg` driver** — maximum transparency, no type safety, more boilerplate per query. **Supabase client SDK for all access** — convenient but blurs the line between client-safe and server-only data paths, which is exactly the line this project must keep sharp.
Trade-offs	More SQL to write by hand, particularly for the reporting-shaped queries in the historical-import progress view.
Approval	RECOMMENDATION — project owner review.

7.3.6 API architecture

Field	Detail
Recommended	**Internal RPC-style server actions over an explicit service layer.** No public REST or GraphQL API in Phase 1. Route handlers exist only where a non-form consumer genuinely needs one — file downloads (CSV export), print views, and health checks.
Why	There is no third-party consumer in Phase 1 and none in Phase 2's stated list. A public API would be surface area to secure, document, version and rate-limit for no user. The service layer, however, is designed as if an API existed — each operation is a named, self-contained function that validates, authorises, transacts and audits — so exposing it later is a thin adapter, not a rewrite.
Alternatives considered	**REST API from day one** — deferred, not rejected; Section 31 keeps the option open. **GraphQL** — rejected: authorisation per field is the wrong complexity for a system whose central requirement is provably correct permissions.
Trade-offs	A future mobile app would need the adapter layer built. Accepted; it is not in any stated phase.
Approval	RECOMMENDATION — project owner review.

7.4 What this stack does *not* give us, and what we do about it

Gap	Consequence	Response in this plan
No background job runner on Vercel's serverless functions	Long-running work (a full-institution recalculation, a very large export) cannot run as a normal request.	Phase 1 has no such workload. The semester-end export streams. If a bulk recalculation is ever needed, it runs as an operator-invoked, chunked, resumable job — noted as a Phase 2 architectural consideration in Section 31.
Serverless functions cannot hold a persistent database connection	Connection exhaustion under load, or unexpected latency.	Use the connection pooler in transaction mode; verify pooled transaction semantics explicitly in Stage 1, because multi-statement transactions behave differently across pooler modes and this project depends on them.

Gap	Consequence	Response in this plan
Vendor concentration: application, database, auth and file storage with two providers	An outage at either provider takes the portal offline; a pricing or policy change affects the College directly.	Accepted for Phase 1 given the alternative is unmanaged infrastructure with no administrator. Mitigated by keeping schema and logic portable (7.3.1), by the semester-end offline export (REQ-B03), and by the paper grade sheets that remain the physical fallback (REQ-B04). Recorded as a risk in Section 28.
Data residency is outside Liberia	Student academic data will be stored in whichever region the Supabase project is created in.	REQUIRES APPROVAL — DEC-30. The College should choose the region consciously and confirm it is acceptable. Recommend the region with the lowest latency to Liberia that the provider offers, and that the choice be recorded.
No offline capability	Admin cannot enter grades while the campus connection is down.	Out of scope for Phase 1 and not requested. The paper sheet is the fallback and entry resumes when connectivity returns. Idempotency keys ensure a retried submission after a drop does not double-write.

7.5 Cost and plan considerations requiring approval

Two hosting decisions have a cost dimension that only the College can settle. Both are recorded rather than assumed.

Item	Register	What must be decided
Database plan tier	DEC-31	Free-tier managed Postgres typically offers limited backup retention and may pause inactive projects — both unacceptable for a system of record. A paid tier with daily backups and point-in-time recovery is required for production. The test environment may remain on a free tier.
Number of environments	DEC-31	Three environments (development, testing/staging, production) means at least two hosted database projects. This plan treats that as a minimum, not an optimisation.

RECOMMENDATION

Do not run the production academic record on a free tier. The College's stated backup posture in §14 already accepts a one-semester exposure window; a tier without point-in-time recovery would widen rather than close it, and the difference in cost is small against the cost of re-entering a semester of paper records.

8. System Architecture

8.1 Architectural principles

Five principles govern every structural decision that follows. They are stated first because they are the criteria against which any later change to the architecture should be judged.

#	Principle	Consequence
P1	**The browser is untrusted.**	No client holds a database credential capable of writing academic data. Every mutation crosses a server boundary where it is validated, authorised, transacted and audited. Client-side checks exist only to give fast feedback.
P2	**One rule, one place.**	Each business rule — the grading scale, the passing threshold, the credit-hour limits, the semester-state gate — is defined once, on the server, and consumed everywhere. A rule expressed twice is a rule that will disagree with itself.
P3	**Critical logic is pure and isolated.**	The GPA/CGPA engine, the course-plan validator and the semester transition table are pure functions with no database access. They take data in and return results, which makes them exhaustively testable and impossible to break by accident from a UI change.
P4	**Defence in depth on authorisation.**	The service layer is the primary gate; Postgres row-level security is the second. A forgotten check in one layer is contained by the other.
P5	**Nothing important happens outside a transaction.**	Publishing a class, approving a plan, applying a correction and completing an import each write several tables plus an audit entry. All of it commits together or none of it does.

8.2 Layered architecture

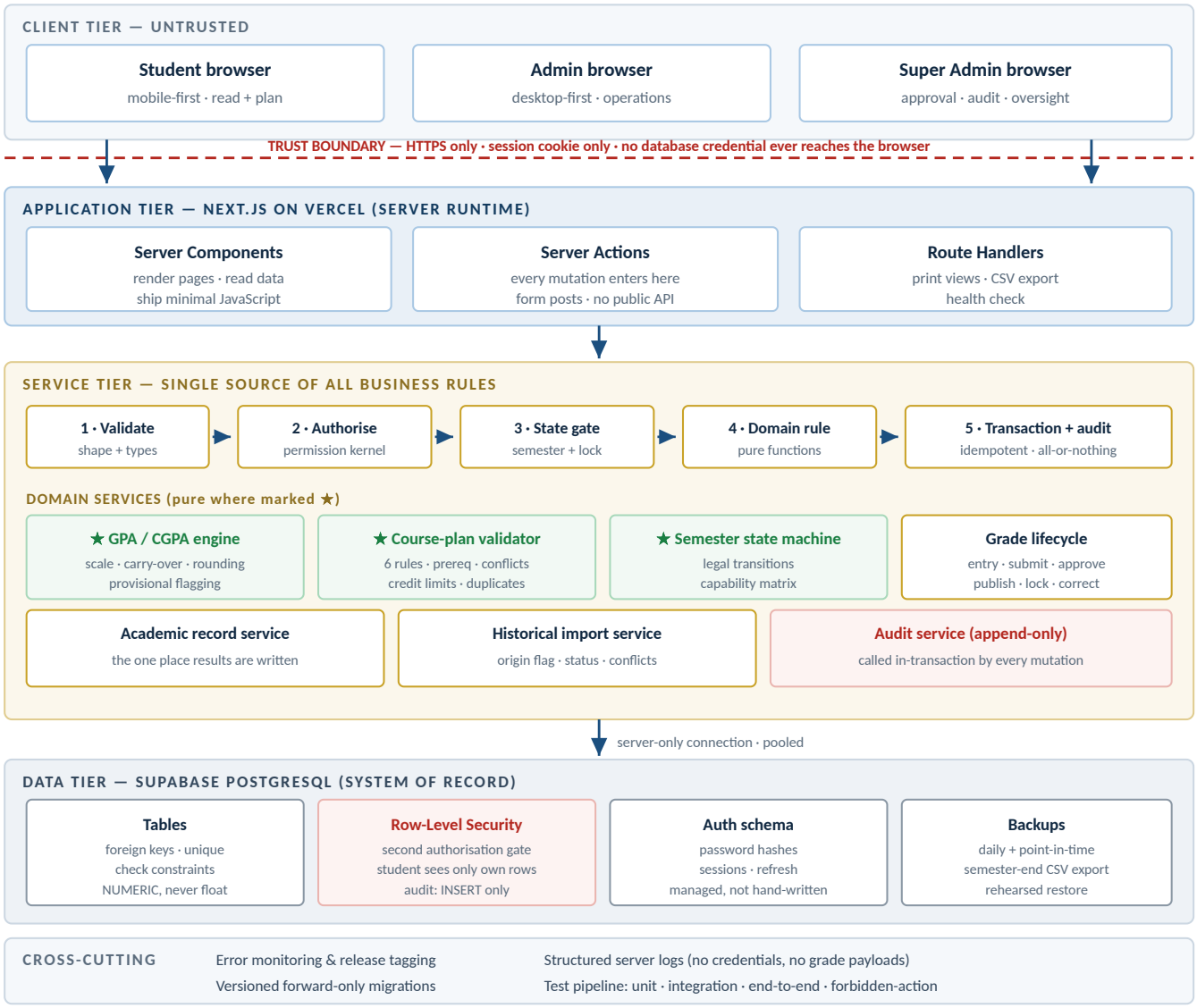


Figure 8.1 — Phase 1 layered architecture. The trust boundary is the defining feature: everything above it is a rendering surface, everything below it enforces rules.

8.3 How data moves through the system

Two representative paths, one read and one write, describe every other interaction in the system.

8.3.1 Read path — a student opens their semester grade sheet

```
1. Browser requests /portal/results/{semesterId} with a session cookie.
2. Server Component resolves the session → actor { userId, role: STUDENT, studentId }.
3. assertCan(actor, "student.results.view", { studentId: actor.studentId })
     → allowed only for the actor's own studentId.
4. Query academic_record JOIN semester
     WHERE student_id = actor.studentId
       AND semester_id = {semesterId}
       AND status IN ('published')            -- draft/submitted are never selected
   Row-level security independently restricts the same rows. Both must agree.
5. gpaEngine.computeSemester(records, policy) → { gpa, creditsAttempted, creditsEarned }
6. importStatus = student.historical_import_status
   provisional = (importStatus != 'COMPLETE')
7. Render server-side. Response carries HTML + ~0 KB of business logic.
   No grade data, no policy, no role is present in client JavaScript.
```

Note step 4: unpublished grades are excluded by the query *and* by policy. Note step 6: the provisional flag travels with the number, not as a separate lookup the UI might forget to make.

8.3.2 Write path — Super Admin approves a class grade submission

```
1. Browser posts the approval form to a Server Action with an idempotency key.
2. Validate input shape: { submissionId, decision: APPROVE|REJECT, reason? }.
3. assertCan(actor, "grade.submission.approve", { submissionId })
     → role must be SUPER_ADMIN. An ADMIN actor is refused here, not in the UI.
4. Idempotency check: has this key already been processed? If yes, return the stored result.
5. BEGIN TRANSACTION
   a. Lock the submission row (SELECT ... FOR UPDATE) and re-read its status.
      If status != 'SUBMITTED', abort with a conflict error.
   b. Re-assert that the actor is not the same user who submitted it   -- REQ-R05
   c. Re-assert semester state permits publication                     -- REQ-W02
   d. For each grade in the submission:
        - set status = PUBLISHED, locked_at = now()
        - upsert the corresponding academic_record row
          (frozen credit_hours, frozen grade_point, origin = 'SYSTEM')
   e. Recompute and upsert student_semester_summary for each affected student
   f. Recompute and upsert cumulative summary for each affected student
   g. Set submission.status = APPROVED, approved_by, approved_at
   h. audit.write(actor, "GRADE_SUBMISSION_APPROVED", submission, old, new, reason)
      + one audit row per grade published
   COMMIT
6. If any step fails, the whole transaction rolls back: no student sees a grade,
   no CGPA moves, and the submission remains awaiting approval.
7. Store the idempotency result. Revalidate affected pages.
```

WHY STEP 5B MATTERS

The permission check in step 3 establishes that the actor holds the Super Admin role. Step 5b establishes something different and equally required by REQ-R05: that the approver is not the submitter. If a person ever holds both roles — which the College should avoid but the software cannot prevent — this check is what preserves the two-key principle. It is re-asserted inside the transaction because role assignments can change between page load and submit.

8.4 Where each critical rule lives

This table exists to prevent the single most damaging architectural mistake available to this project: a business rule implemented in the browser. Every rule below has exactly one authoritative home.

Rule	Authoritative home	Also enforced at	Client's role
Grading scale and grade points	Server: grade-scale configuration table + pure engine	Database check constraint on allowed letters	Displays the scale. Never computes a grade point.
Semester GPA / CGPA / credits	Server: pure GPA engine (DER-09)	—	Displays the returned figures verbatim.
Repeat carry-over rule	Server: inside the GPA engine	—	Displays the R marker the server supplies.
Prerequisite satisfaction	Server: course-plan validator against academic_record	—	May show a provisional warning; never authorises submission.
Credit-hour minimum / maximum	Server: validator, reading institution/department configuration	—	Shows a running total for feedback only.
Duplicate course in a plan	Server: validator	Unique index on (plan_id, course_id)	Disables an already-selected row for usability.
Schedule conflict	Server: validator over structured meeting times	—	May highlight a clash as the student selects.
Who may enter a grade	Server: permission kernel	RLS policy on grade tables	Hides the control. This is cosmetic only.
Who may approve a grade	Server: permission kernel + submitter/approver separation check	RLS policy	Hides the control.
Locked-grade immutability	Server: grade lifecycle service	Database: no UPDATE path for locked rows outside an approved correction	Renders the field read-only.
Semester-state gating	Server: state machine consulted by every affected service	Check constraint on legal state values	Shows the current state and why an action is unavailable.
Student data isolation	Server: permission kernel scoping every query by actor	RLS policy: `student_id = current student`	None. The client never receives another student's row to hide.
Audit completeness	Server: audit service invoked in-transaction	Database: INSERT-only privilege	None.
Provisional marking	Server: attached to the computed figure	—	Renders the badge it is given.

NON-NEGOTIABLE

If any row in the table above ever acquires a second implementation in client code, that is a defect, not an optimisation — including "just for faster feedback". Fast feedback is achieved by calling the server rule, not by copying it.

8.5 Environment topology

Environment	Application	Database	Data and purpose
Development	Local machine	Local Postgres or a dedicated hosted project	Synthetic seed data only. Never a copy of production. Free to reset.
Testing / Staging	Vercel preview or a fixed staging deployment	Separate hosted project	Realistic but fabricated data, including deliberately incomplete historical records. Where UAT (Section 23.9) happens.

Environment	Application	Database	Data and purpose
Production	Vercel production	Separate hosted project, paid tier with point-in-time recovery	The College's real academic record. Access limited to the deploying developer and one named administrator; every access recorded.

The three environments never share a database, and production data is never copied downward. If realistic volume is needed for testing, it is generated, not exported. This is a hard rule: a copied student record in a test environment is a data breach waiting for a misconfiguration.

9. Domain Model

The model below is derived from the requirements, not copied from a generic student-information-system template. Three modelling decisions do most of the work and are stated before the diagram, because the rest of the model only makes sense once they are understood.

9.1 The three decisions that shape the model

Decision 1 — One academic record table, not two

The obvious reading of the requirements produces two stores of results: grades created in the system, and historical records transcribed from paper. That is the wrong shape. Four different features must answer the question *"what has this student completed and passed?"*: CGPA, prerequisite checking, retake detection, and the Phase 2 transcript. If the answer lives in two tables, those four features will eventually give three different answers.

Instead there is a single `academic_record` table. Every completed course result — however it arrived — is one row. An `origin` column distinguishes `SYSTEM` (published through the grade workflow) from `IMPORTED` (transcribed from paper), which satisfies REQ-H03 without fragmenting the data. Publication of a grade writes into this table; historical entry writes into it directly.

Decision 2 — Academic records freeze their own arithmetic inputs

An `academic_record` row stores the credit hours and the grade point **as they were when the result was recorded**, rather than looking them up from the course definition at calculation time. Courses change: a three-credit course becomes four, a grading scale is revised. Without frozen values, a 2024 GPA silently changes when an Admin edits a course in 2029, and a student who printed a record last year holds a document the system no longer agrees with. The course reference is retained for reporting and prerequisite matching; the numbers used in arithmetic are the frozen ones.

Decision 3 — Course, Offering and Registration are three different things

The source's hierarchy (§4) ends "Course → Course Offering / Class", and §9 refers to "every registered student for the course offering". That implies three distinct entities that are frequently conflated:

Entity	What it is	Lifetime	Example
`course`	A reusable catalogue definition: code, title, credit hours, prerequisites, owning department.	Years. Survives every semester.	CSC 201 — Data Structures, 3 credits
`course_offering`	That course, actually taught, in one semester, as one section, at particular times.	One semester.	CSC 201 – Section A, First Semester 2026/2027, Mon & Wed 09:00–10:30, Room B4
`registration`	One student's enrolment in one offering. The thing a grade attaches to.	One semester, then becomes history.	Student 20240117 in CSC 201 – Section A

9.2 Conceptual entity map

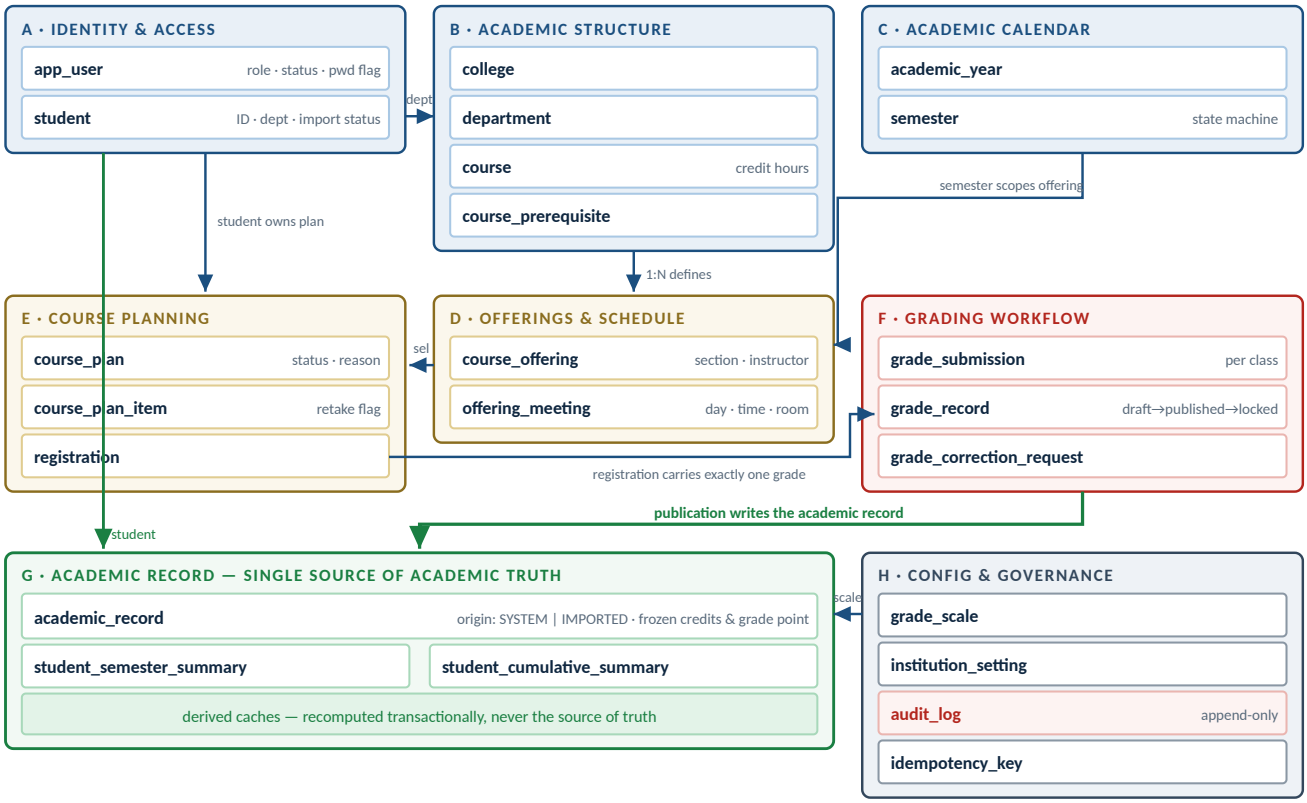


Figure 9.1 — Conceptual entity map, clustered by domain. Green indicates the academic-truth path: everything that determines a GPA, a prerequisite decision or a future transcript converges on one table. The complete relationship list with cardinalities is Table 9.3.

9.3 Relationship catalogue

From	Card.	To	Meaning and integrity note
app_user	1 : 0..1	student	A student profile always has an identity; Admin and Super Admin identities have no student profile. Deleting is not permitted — see 9.5.
college	1 : N	department	A department belongs to exactly one college. Restrict delete once departments exist.
department	1 : N	course	Course ownership. A course is offered by one department, though students from any department may take it.
department	1 : N	student	A student's home department. Changing it is an audited event and does not alter past records.
course	N : M	course (via course_prerequisite)	Self-referencing. Must be acyclic — enforced by an application-level cycle check on write.
academic_year	1 : N	semester	Ordered by sequence number within the year.
semester + course	1 : N	course_offering	Unique on (semester, course, section). This is where a catalogue course becomes a taught class.
course_offering	1 : N	offering_meeting	Structured meeting times. Zero meetings is legal (an offering with no timetable yet) but blocks conflict checking.
student + semester	1 : 0..1	course_plan	At most one active plan per student per semester. A rejected plan is revised, not duplicated.

From	Card.	To	Meaning and integrity note
course_plan	1 : N	course_plan_item	Unique on (plan, course) — this is REQ-P07 expressed as a constraint.
course_plan_item	N : 1	course_offering	The student selects a specific section, not just a course, because schedule conflicts are section-level.
course_plan_item	1 : 0..1	registration	Created on plan approval. A registration may also be created directly by Admin, in which case it has no plan item.
student + course_offering	1 : 0..1	registration	Unique. A student cannot be registered twice in the same offering.
course_offering	1 : N	grade_submission	One submission per attempt. A rejected submission is followed by a new one; history is preserved.
registration	1 : 0..1	grade_record	Exactly one grade per registration. Unique constraint — this is REQ-G03's duplicate rejection at the data layer.
grade_submission	1 : N	grade_record	The batch a Super Admin approves or rejects as a unit.
grade_record	1 : N	grade_correction_request	A locked grade may be corrected more than once over its life; every request is retained.
grade_record	1 : 0..1	academic_record	Populated at publication. The link is what lets a correction find and update the record it produced.
student + semester + course	1 : N	academic_record	Not unique on its own — a student may legitimately have two records for the same course in different semesters (a repeat). Unique on (student, semester, course, attempt).
student + semester	1 : 0..1	student_semester_summary	Derived cache. Recomputed in the same transaction as any change to its inputs.
student	1 : 0..1	student_cumulative_summary	Derived cache holding CGPA, total credits and the provisional flag.
grade_scale	1 : N	academic_record	By letter code. The record stores the frozen grade point, so a later scale change does not rewrite history.
app_user	1 : N	audit_log	Actor reference. Never cascades — an audit entry must survive any change to the actor's account.

9.4 Entity specifications

9.4.1 app_user

Purpose	The single authentication and authorisation anchor for every person who can log in. Exists so that students, Admins and Super Admins share one identity mechanism rather than three.
Key attributes	`id` (matches the auth provider's user id), `login_identifier` (Student ID for students, username for staff), `display_name`, `role` ∈ {STUDENT, ADMIN, SUPER_ADMIN}, `status` ∈ {ACTIVE, DISABLED}, `must_change_password`, `last_login_at`, `created_by`, `created_at`.
Relationships	1:0..1 student; 1:N audit_log (as actor); referenced as creator/approver on many records.
Ownership	Admin creates and resets student accounts. Super Admin creates and manages Admin accounts (REQ-A06). Super Admin accounts are managed per DEC-15; the first is created by the bootstrap procedure (DER-26).
Lifecycle	Created → temporary password issued (`must_change_password = true`) → active after first change → may be disabled → never deleted.
Constraints	Role is immutable after creation (changing a role means creating a new account, so that historical actor attribution stays truthful). At least one ACTIVE SUPER_ADMIN must exist at all times — enforced by the service layer and a database check on the disable path.
Uniqueness	`login_identifier` is globally unique, case-insensitive.
Deletion	**Never hard-deleted.** Disabling revokes access; the record persists so audit entries and approvals remain attributable.

Audit	Creation, disable/enable, role assignment, password reset by an Admin, and forced-change completion are all logged. The password itself is never logged in any form.

9.4.2 student

Purpose	The academic profile of a person enrolled at the College. The subject of every academic record.
Key attributes	`student_number` (the College's Student ID — digits only, first four the admission year, 6–8 digits; CR-08), name parts, `department_id`, `enrolment_year`, `status` ∈ {ACTIVE, INACTIVE, SUSPENDED, GRADUATED, ADMISSION_FORFEITED}, `historical_import_status` ∈ {NOT_STARTED, IN_PROGRESS, COMPLETE}, `import_completed_by`, `import_completed_at`, one contact field.
Relationships	1:1 app_user; N:1 department; 1:N course_plan, registration, academic_record.
Ownership	Admin only (REQ-T02). Super Admin may read but not create or edit (REQ-R04).
Lifecycle	Enrolled by Admin → historical import progresses through its three states → active study → graduated or withdrawn. Status changes never alter existing academic records.
Constraints	Only an ACTIVE student may submit a course plan. `historical_import_status` may only move NOT_STARTED → IN_PROGRESS → COMPLETE, and moving back to IN_PROGRESS is an audited action requiring a reason.
Uniqueness	`student_number` unique across the institution, including graduated and withdrawn students. Reuse of a retired ID is prohibited regardless of how DEC-02 is answered.
Deletion	**Never.** A student enrolled in error is marked INACTIVE with a reason. Deleting would orphan academic records and break audit attribution.
Audit	Enrolment, every field change, department change, status change and every import-status transition.

9.4.3 college / department

Purpose	The College's own organisational vocabulary (REQ-S02): **College** is the faculty-level unit, **Department** the programme-level unit within it. These terms are used verbatim in the schema, the interface and all documentation.
Key attributes	college: `code`, `name`, `is_active`. department: `college_id`, `code`, `name`, `is_active`, optional `max_credits` override — a department may set a ceiling below the institution's 21, never above it (CR-04).
Relationships	college 1:N department; department 1:N course; department 1:N student.
Ownership	Admin only. Explicitly denied to Super Admin (REQ-R04).
Lifecycle	Created → used → deactivated when no longer offered. Deactivation hides it from new selections but leaves history intact.
Constraints	A department cannot be deactivated while it has ACTIVE students — the service returns a clear error naming the blocking records rather than failing silently.
Uniqueness	college.code unique; department.code unique within its college.
Deletion	Hard delete permitted **only** while no dependent row of any kind exists. Otherwise deactivate (DER-22).
Audit	Create, rename, code change, activate/deactivate.

9.4.4 course

Purpose	The reusable catalogue definition of a subject. Not a class, not a timetable entry.
Key attributes	`department_id`, `code` (e.g. CSC 201), `title`, `credit_hours` (integer, > 0), `level`, `is_active`, optional `prerequisite_min_grade` (defaults to the institution passing grade of 0.70).
Relationships	N:1 department; N:M course via course_prerequisite; 1:N course_offering; referenced by academic_record.
Ownership	Admin only.
Lifecycle	Created → offered across many semesters → retired (deactivated). A retired course still appears on historical records.
Constraints	Changing `credit_hours` is permitted but does **not** alter any existing academic_record, because those store frozen values (Decision 2). The system warns the Admin of this explicitly at the point of change.

Uniqueness	`code` unique institution-wide, normalised (trimmed, upper-cased, single-spaced) before comparison.
Deletion	Deactivate once any offering or academic record references it.
Audit	Create, edit — with credit-hour changes logged as a distinct, highlighted action because of their academic significance.

9.4.5 course_prerequisite

Purpose	Expresses that one course must be completed and passed before another may be taken (REQ-P04).
Key attributes	`course_id`, `prerequisite_course_id`, `min_grade` (nullable; defaults to the institution passing grade of 0.70 — grade D–).
Relationships	Both columns reference course.
Ownership	Admin only.
Lifecycle	Added and removed as the curriculum changes. A change applies to future validation only; it never invalidates a plan already approved.
Constraints	No self-reference. **No cycles** — checked by traversing the prerequisite graph on every insert, with a clear error naming the cycle. A cycle would make the affected courses permanently unregisterable.
Uniqueness	Unique on (course_id, prerequisite_course_id).
Deletion	Hard delete is safe; the relationship is a rule, not a record of anything that happened.
Audit	Add and remove, both logged — a removed prerequisite changes who may register for what.

9.4.6 academic_year / semester

Purpose	The time skeleton of the academic record. Every result, plan, offering and registration hangs off a semester.
Key attributes	academic_year: `label` (e.g. 2026/2027), `start_date`, `end_date`, `is_current`. semester: `academic_year_id`, `sequence` (1, 2, 3…), `name` (First Semester…), `state` ∈ the six states of REQ-W01, `start_date`, `end_date`, `min_credits` / `max_credits` override.
Relationships	academic_year 1:N semester; semester 1:N course_offering, course_plan, academic_record, student_semester_summary.
Ownership	Admin creates and advances forward. **Only Super Admin moves a semester backwards or reopens it** (REQ-R07, REQ-W03).
Lifecycle	The six-state machine specified in Section 13.
Constraints	Semester dates must fall within the parent academic year. Only one semester may be in each of Registration and Grade Submission at a time — a guard against an Admin accidentally opening two registration periods (recommendation; see Section 13.6). State values are constrained in the database, and transitions are validated against the legal-transition table.
Uniqueness	academic_year.label unique; semester unique on (academic_year_id, sequence).
Deletion	A semester may be deleted only while in Draft with no dependent rows. After that, never.
Audit	**Every state transition**, forward and backward, with actor, from-state, to-state, timestamp, and a mandatory reason for any backward or reopening move (REQ-W04).

9.4.7 course_offering

Purpose	A course actually taught in a specific semester as a specific section. The object students register into and grades are entered against (REQ-S03).
Key attributes	`semester_id`, `course_id`, `section` (e.g. A), `instructor_name` (free text, no account — DER-15), `capacity` (nullable — GAP-26), `status` ∈ {DRAFT, PUBLISHED, CANCELLED}, `frozen_credit_hours` (copied from the course at creation).
Relationships	N:1 semester; N:1 course; 1:N offering_meeting, registration, grade_submission; referenced by course_plan_item.
Ownership	Admin only (REQ-S05).

Lifecycle	Created in DRAFT while the semester is Draft or Open → PUBLISHED so students can select it → registrations accumulate → grades entered → semester closes and the offering becomes historical. CANCELLED only while it has no registrations.
Constraints	Cannot be created against a Closed semester. Cannot be deleted once any registration exists. `frozen_credit_hours` is set once and never edited.
Uniqueness	Unique on (semester_id, course_id, section).
Deletion	Permitted only in DRAFT with no registrations; otherwise CANCELLED.
Audit	Create, publish, cancel, instructor change, capacity change, and any schedule change after publication (students may already have planned around it).

9.4.8 offering_meeting

Purpose	A structured meeting time for an offering. Exists because REQ-P09's schedule-conflict check cannot be performed against free text (DER-14).
Key attributes	`offering_id`, `day_of_week` (1–7), `start_time`, `end_time`, `room` (free text).
Relationships	N:1 course_offering.
Ownership	Admin only.
Lifecycle	Created with or after the offering; edited until the semester reaches In Progress; changes after publication are audited and flagged to affected students in their timetable view.
Constraints	`end_time > start_time` (check constraint). Two meetings of the same offering may not overlap.
Uniqueness	No natural key; overlap is prevented by validation rather than a unique index.
Deletion	Hard delete permitted; it is scheduling data, not a record of an event.
Audit	Changes made after the offering is published.

9.4.9 course_plan / course_plan_item

Purpose	A student's proposed set of courses for one semester, and the Admin's decision on it (REQ-P01, REQ-P10).
Key attributes	course_plan: `student_id`, `semester_id`, `status` ∈ {DRAFT, SUBMITTED, APPROVED, REJECTED}, `total_credits`, `submitted_at`, `reviewed_by`, `reviewed_at`, `rejection_reason`, `version`. course_plan_item: `plan_id`, `offering_id`, `course_id`, `is_retake`, `prereq_override_reason`, `prereq_override_by`.
Relationships	N:1 student; N:1 semester; 1:N course_plan_item; each item N:1 course_offering, 1:0..1 registration.
Ownership	The plan belongs to the student while DRAFT or REJECTED. On SUBMITTED it becomes read-only to the student. Admin owns the approve/reject decision. Super Admin has no role here at all.
Lifecycle	DRAFT → SUBMITTED → APPROVED (terminal, locked) or REJECTED → back to DRAFT for revision. Section 14 gives the full machine.
Constraints	A rejection must carry a reason (REQ-P10) — enforced by a check constraint, not only by the form. An APPROVED plan is immutable. Total credits must sit within the configured minimum and maximum at both submission and approval (REQ-P03, REQ-P05).
Uniqueness	At most one non-superseded plan per (student, semester). Unique on (plan_id, course_id) for items — REQ-P07.
Deletion	A DRAFT plan may be deleted by its student. Once submitted, never — the submission and its outcome are part of the record.
Audit	Submission, approval, rejection with reason, every prerequisite override with its reason and actor, and any Admin-side edit.

9.4.10 registration

Purpose	The fact that a student is enrolled in a specific offering for a specific semester. The anchor a grade attaches to and the source of the class list (REQ-G02, DER-16).

Key attributes	`student_id`, `offering_id`, `semester_id`, `plan_item_id` (nullable), `source` ∈ {PLAN_APPROVAL, ADMIN_DIRECT}, `is_retake`, `status` ∈ {REGISTERED, DROPPED}, `dropped_reason`, `frozen_credit_hours`.
Relationships	N:1 student; N:1 course_offering; 0..1 course_plan_item; 1:0..1 grade_record.
Ownership	Created by the system on plan approval, or directly by an Admin (DEC-14). Never by a student, never by a Super Admin.
Lifecycle	REGISTERED → optionally DROPPED (audited, with reason) → graded → historical.
Constraints	Cannot be dropped once its grade is published. Cannot be created against a semester past Grade Submission. Creation must be atomic with plan approval (DER-12).
Uniqueness	Unique on (student_id, offering_id).
Deletion	Never. A registration created in error is DROPPED with a reason.
Audit	Creation (including its source), drop with reason, and any change of offering.

9.4.11 grade_submission

Purpose	The reviewable unit a Super Admin approves or rejects: one class's grades, submitted together (REQ-G06, DER-18).
Key attributes	`offering_id`, `attempt_no`, `status` ∈ {SUBMITTED, **PARTIALLY_DECIDED**, CLOSED}, `submitted_by`, `submitted_at`, `grade_count`, `undecided_count`. Decisions themselves live on each grade, because a Super Admin may act per grade or on the whole batch (CR-06).
Relationships	N:1 course_offering; 1:N grade_record.
Ownership	Created by Admin. Decided by Super Admin. Neither role can do the other's part.
Lifecycle	SUBMITTED → APPROVED (publishes and locks every grade in it) or REJECTED (returns every grade to draft, with the reason visible to the Admin).
Constraints	`reviewed_by ≠ submitted_by` — a check constraint, so the two-key rule survives even a defect in the service layer. Rejection requires a reason. A submission may only be created when every registered student in the offering has a draft grade, or the Admin explicitly confirms a partial submission with a recorded note.
Uniqueness	Unique on (offering_id, attempt_no). At most one SUBMITTED submission per offering at a time.
Deletion	Never. Rejected submissions are the evidence that review happened.
Audit	Submit, approve, reject — each with actor, timestamp, grade count and reason.

9.4.12 grade_record

Purpose	One student's result for one registration, as it moves through the entry → approval → publication → lock lifecycle.
Key attributes	`registration_id`, `submission_id` (nullable while draft), `score` `NUMERIC(4,1)` (nullable when the grade is a letter such as I), `letter`, `grade_point` `NUMERIC(3,2)`, `status` ∈ {DRAFT, SUBMITTED, PUBLISHED, LOCKED}, `entered_by`, `entered_at`, **`decided_by`**, **`decided_at`**, **`decision_reason`** (per-grade approval, CR-06), `published_at`, `locked_at`, `version`.
Relationships	1:1 registration; N:1 grade_submission; 1:N grade_correction_request; 1:0..1 academic_record.
Ownership	Entered and edited by Admin while DRAFT (REQ-G05). Published by Super Admin approval. Editable by no one once locked.
Lifecycle	DRAFT (invisible to students) → SUBMITTED (still invisible, read-only to Admin) → PUBLISHED + LOCKED on approval → changed only through an approved correction. Section 15 gives the full machine.
Constraints	`letter` must exist in grade_scale. `grade_point` must match the scale entry for that letter at the time of publication. Score, if present, must fall in the band for the letter. No UPDATE path exists for a LOCKED row outside the correction service.
Uniqueness	One grade_record per registration — the database expression of REQ-G03's duplicate rejection.
Deletion	A DRAFT grade may be cleared (set back to null) by the entering Admin. A published grade is never deleted.

Audit	Every entry, every draft edit (old → new), submission, publication, lock, and every correction. This is the most heavily audited entity in the system.

9.4.13 grade_correction_request

Purpose	The only lawful route to changing a locked grade (REQ-G08). Modelled as a request so that the change is proposed by one actor and effected by another.
Key attributes	`grade_record_id`, `old_score`, `old_letter`, `old_grade_point`, `new_score`, `new_letter`, `new_grade_point`, `reason` (mandatory), `requested_by`, `requested_at`, `status` ∈ {PENDING, APPROVED, REJECTED}, `decided_by`, `decided_at`, `decision_note`.
Relationships	N:1 grade_record; two references to app_user.
Ownership	Requested by Admin (REQ-R06). Decided by Super Admin. Never self-approved.
Lifecycle	PENDING → APPROVED (the grade, the academic record and both summaries are updated in one transaction) or REJECTED (nothing changes; the request remains as evidence).
Constraints	`decided_by ≠ requested_by`, as a check constraint. Reason is non-empty. Only one PENDING request per grade at a time. The old values are captured at request time and re-verified at approval time — if the grade changed in between, the request is rejected as stale.
Uniqueness	Partial unique index: one PENDING per grade_record_id.
Deletion	Never.
Audit	Request, approval or rejection, and the resulting grade change — old value, new value, both actors, reason, timestamp (REQ-G09).

9.4.14 academic_record

Purpose	**The single source of academic truth.** One row per completed course result, whatever its origin. Every GPA, every CGPA, every prerequisite decision, every retake detection and every future transcript reads this table and nothing else.
Key attributes	`student_id`, `semester_id`, `course_id`, `course_code_snapshot`, `course_title_snapshot`, `credit_hours` (frozen), `letter`, `grade_point` (frozen, NUMERIC), `score` (nullable), `attempt_no`, `origin` ∈ {SYSTEM, IMPORTED}, `grade_record_id` (nullable — null for imported rows), `counts_in_gpa`, `counts_in_attempted`, `counts_in_earned`, `is_repeat_dropped`, **`was_major_at_record`** (frozen — Section 12.5), `entered_by`, `entered_at`, `source_note` (for imported records: which paper document, any legibility caveat).
Relationships	N:1 student; N:1 semester; N:1 course; 0..1 grade_record.
Ownership	Written by the system on grade publication, or by an Admin through the historical-entry screen. Never edited directly by anyone — a change to a SYSTEM row goes through a correction request; a change to an IMPORTED row goes through the historical-correction path, which is equally audited.
Lifecycle	Created → participates in every calculation → corrected only through a controlled path → never deleted.
Constraints	`credit_hours > 0`. `grade_point` between 0.0 and 4.0. `origin = 'SYSTEM'` implies `grade_record_id IS NOT NULL`; `origin = 'IMPORTED'` implies it is NULL — a check constraint, so the two paths cannot be confused. The three `counts_in_*` flags are populated from grade_scale at write time, freezing the policy alongside the numbers.
Uniqueness	Unique on (student_id, semester_id, course_id, attempt_no). This is the duplicate-prevention rule for both grade publication and historical import, in one constraint.
Deletion	**Never** in normal operation. An imported row entered against the wrong student is voided by an audited correction that records both the error and the fix; the row is marked void rather than removed.
Audit	Every insert and every change, with origin, actor, old and new values and reason. This table and the audit log together are the College's academic record.

9.4.15 student_semester_summary and student_cumulative_summary

Purpose	Derived caches of the GPA engine's output, so that the provisional state and the moment of calculation are explicit and auditable rather than recomputed silently on every page view.
Key attributes	semester: `student_id`, `semester_id`, `gpa`, `credits_attempted`, `credits_earned`, `quality_points`, `is_provisional`, `computed_at`, `policy_version`. cumulative: `student_id`, `cgpa`, `total_credits_attempted`, `total_credits_earned`, `is_provisional`, `computed_at`, `policy_version`.
Relationships	N:1 student; semester summary N:1 semester.
Ownership	The system alone. No user writes these directly; there is no UI that edits them.
Lifecycle	Written or rewritten in the same transaction as any change to their inputs: publication, correction, historical entry, import-status change.
Constraints	`policy_version` records which grading-policy version produced the figure, so a later policy change can be detected and the affected records recomputed deliberately rather than by surprise. `is_provisional` is true whenever the student's import status is not COMPLETE (REQ-H05).
Uniqueness	Unique on (student_id, semester_id) and (student_id) respectively.
Deletion	May be deleted and rebuilt from academic_record at any time without loss — that is the test of whether it is really a cache.
Audit	Not audited individually. The events that cause recomputation are audited, and the values are always reproducible from audited data.

9.4.16 audit_log

Purpose	The append-only record of who did what to which academic record, when, and why (REQ-E01, REQ-E02).
Key attributes	`id`, `occurred_at`, `actor_user_id`, `actor_role_snapshot`, `action` (typed vocabulary — Section 19.3), `entity_type`, `entity_id`, `student_id` (nullable, for filtering by student), `old_value` (JSONB), `new_value` (JSONB), `reason`, `request_id`, `ip_address`, `user_agent`.
Relationships	N:1 app_user (actor), never cascading.
Ownership	Written only by the audit service, only inside the transaction of the change it describes (DER-21). Read only by Super Admin (REQ-R08).
Lifecycle	Insert. That is the whole lifecycle.
Constraints	No UPDATE or DELETE privilege is granted to any application role (DER-20). `actor_role_snapshot` stores the role held at the time, so a later role change cannot rewrite the meaning of a past action. Indexed on (student_id, occurred_at) and (entity_type, entity_id) for practical retrieval.
Uniqueness	None. Duplicates are impossible in practice because entries are written once per transaction.
Deletion	**Never, by anyone, through any application path.**
Audit	Reading the audit log is itself logged, so that oversight of the oversight exists.

9.4.17 Supporting entities

Entity	Purpose and key attributes	Notes
`grade_scale`	The College's grading policy as data: `letter`, `min_score`, `max_score`, `grade_point`, `counts_in_gpa`, `counts_in_attempted`, `counts_in_earned`, `is_passing`, `display_order`, `policy_version`, `effective_from`.	Seeded with the **nine confirmed letters plus Incomplete** (CR-01, CR-02): A+ 4.00, A− 3.70, B+ 3.30, B− 2.70, C+ 2.30, C− 1.70, D+ 1.30, D− 0.70, F 0.00, I excluded from all three totals. **No Withdrawal, audit or transfer-credit entry exists.** `grade_point` is `NUMERIC(3,2)`. Changing the scale creates a new policy version rather than editing rows in place, so historical figures remain reproducible. Admin-editable only with Super Admin approval — RECOMMENDATION, since it is the single most consequential configuration in the system.

Entity	Purpose and key attributes	Notes
`institution_setting`	Named configuration: credit-hour minimum and maximum, GPA rounding, prerequisite override switch and expiry, current academic year, institutional display name.	Every value that is currently an open decision lives here, so answering a decision is a configuration change rather than a code change. Changes are audited.
`idempotency_key`	`key`, `actor_user_id`, `operation`, `request_hash`, `result`, `created_at`.	Implements DER-13. Keys older than a short retention window are purged. A repeated key with a *different* request body is an error, not a replay.

9.5 Deletion policy summary

Deletion is the easiest way to destroy an academic record, so it is governed explicitly rather than left to whatever each screen happens to do.

Class of data	Delete permitted?	Rule
Academic results, registrations, plans after submission, submissions, corrections, audit entries	**Never**	These are records of things that happened. Errors are corrected through an audited path that preserves both the error and the fix.
Users and students	**Never**	Disable or mark inactive. Attribution of past actions must survive.
Reference data (college, department, course) once referenced	**No — deactivate**	Deactivation removes it from new selections and leaves history readable (DER-22).
Reference data never referenced	**Yes**	A typo created and caught immediately may be removed. The service checks for dependents and refuses otherwise.
Draft course plans, draft offerings, unsaved drafts	**Yes**	Nothing has happened yet.
Derived summaries	**Yes**	Rebuildable from academic_record by definition.

10. Database Architecture

This section describes how the domain model of Section 9 is realised in PostgreSQL: table groups, key types, constraint strategy, indexing, security policies and migration discipline. It contains no migration files — those are implementation artefacts produced in the stages of Section 24.

10.1 Schema organisation

Schema	Contents	Access
`app`	All business tables: identity, structure, calendar, offerings, planning, grading, records, configuration.	Application role only, mediated by RLS policies.
`audit`	`audit_log` and nothing else.	INSERT granted to the application role; SELECT granted only through a policy restricted to Super Admin; UPDATE and DELETE granted to no one.
`auth`	Provider-managed credential and session storage.	Managed by the auth provider. The application reads the user id and nothing else.

Separating the audit schema is not cosmetic: it makes the "no UPDATE, no DELETE" grant a schema-level statement that is trivial to verify and hard to weaken by accident during a later migration.

10.2 Table inventory

Group	Table	Primary key	Note
Identity	`app_user`	`id` (uuid, = auth user id)	Role, status, must_change_password.
	`student`	`id` (uuid)	One-to-one with app_user; carries `student_number`.
Structure	`college`	`id` (uuid)	
	`department`	`id` (uuid)	FK college.
	`course`	`id` (uuid)	FK department; unique normalised code.
	`course_prerequisite`	(course_id, prerequisite_course_id)	Composite natural key; acyclicity enforced in the service layer.
Calendar	`academic_year`	`id` (uuid)	
	`semester`	`id` (uuid)	Unique (academic_year_id, sequence); state column constrained.
Offerings	`course_offering`	`id` (uuid)	Unique (semester_id, course_id, section).
	`offering_meeting`	`id` (uuid)	Check end_time > start_time.
Planning	`course_plan`	`id` (uuid)	Partial unique (student_id, semester_id) where not superseded.
	`course_plan_item`	`id` (uuid)	Unique (plan_id, course_id).
	`registration`	`id` (uuid)	Unique (student_id, offering_id).
Grading	`grade_submission`	`id` (uuid)	Unique (offering_id, attempt_no); check reviewed_by ≠ submitted_by.
	`grade_record`	`id` (uuid)	Unique (registration_id) — one grade per registration.
	`grade_correction_request`	`id` (uuid)	Partial unique on grade_record_id where status = PENDING.

Group	Table	Primary key	Note
Records	`academic_record`	`id` (uuid)	Unique (student_id, semester_id, course_id, attempt_no).
	`student_semester_summary`	(student_id, semester_id)	Derived cache.
	`student_cumulative_summary`	(student_id)	Derived cache.
Config	`grade_scale`	(policy_version, letter)	Versioned; never edited in place.
	`institution_setting`	`key`	Typed values; every change audited.
	`idempotency_key`	`key`	Short retention.
Audit	`audit.audit_log`	`id` (bigint identity)	Append-only. Monotonic id aids ordering and gap detection.

Nineteen business tables plus one audit table. That is the whole of Phase 1. If a twentieth business table appears during implementation, it is either a missed requirement (raise it) or scope creep (reject it).

10.3 Key and type conventions

Convention	Rule	Reason
Primary keys	UUID v7 (time-ordered) surrogate keys everywhere except natural composite keys noted above.	Time-ordered UUIDs give good index locality without exposing a sequential count of students or grades in URLs.
Natural keys	Kept as unique constraints, never as primary keys — `student_number`, `course.code`, `academic_year.label`.	Natural keys change. A Student ID format change (DEC-02) must not require rewriting every foreign key in the database.
Money-like numerics	Not applicable — there is no money in Phase 1.	OOS-02.
Grade points and GPA	`NUMERIC(9,6)` for stored GPA/CGPA; `NUMERIC(3,2)` for grade points; `NUMERIC(4,1)` for entered scores; integer for credit hours.	DER-08, CR-01, CR-03. Grade points need two decimals (3.70, 2.30, 0.70) and GPA is **displayed to three**; storing six means the displayed figure is always re-derivable and a later precision change needs no data migration. Scores carry one decimal because the College permits decimal marks (CR-07).
Enumerations	Postgres `CHECK` constraints over `text`, not native enum types.	Adding a value to a native enum is a schema-lock operation; a check constraint is edited freely, which matters because several enumerations (student status, grade letters) are pending decisions.
Timestamps	`timestamptz`, stored UTC, displayed Africa/Monrovia.	DER-27.
Soft state	`is_active` / `status` columns rather than deleted rows.	DER-22 and Section 9.5.
Concurrency	Every user-editable table carries `version integer NOT NULL DEFAULT 1`, incremented on update.	DER-19.
Text normalisation	Codes stored upper-cased and trimmed; unique indexes built on the normalised expression.	Prevents "CSC 201" and "csc 201" becoming two courses.

10.4 Constraint strategy

Constraints are the last line of defence, applied on the principle that **anything that must never be true in the database is expressed in the database**, even though the service layer already prevents it.

Constraint	Table	Protects against
Unique (registration_id)	grade_record	Duplicate grade for one student in one class — REQ-G03.
Unique (student_id, semester_id, course_id, attempt_no)	academic_record	Duplicate result rows from either grade publication or historical import — REQ-H02, REQ-G03.
Unique (student_id, offering_id)	registration	Double registration.
Unique (plan_id, course_id)	course_plan_item	Duplicate course in a plan — REQ-P07.
Unique (semester_id, course_id, section)	course_offering	Two identical sections of the same course.
Check reviewed_by ≠ submitted_by	grade_submission	Self-approval of a grade batch — REQ-R05. Enforced even if the service layer is defective.
Check decided_by ≠ requested_by	grade_correction_request	Self-approval of a correction — REQ-R06.
Check (status = 'REJECTED') → rejection_reason IS NOT NULL	course_plan, grade_submission	Rejection without a reason — REQ-P10.
Check origin/grade_record_id coherence	academic_record	An imported row pretending to be system-generated, or vice versa — REQ-H03.
Check credit_hours > 0, grade_point BETWEEN 0 AND 4	academic_record, course	Arithmetic corruption of GPA.
Check end_time > start_time	offering_meeting	Meetings that break conflict detection.
Check state IN (six values)	semester	An undefined semester state — REQ-W01.
Foreign keys, all ON DELETE RESTRICT	All	Cascading deletes. **No foreign key in this database cascades**; deletion of a referenced row must be refused, not silently propagated. The one deliberate exception is `course_plan_item`, which cascades from a draft `course_plan` the student deletes.
Partial unique: one SUBMITTED submission per offering	grade_submission	Two competing batches awaiting approval for the same class.
Partial unique: one PENDING correction per grade	grade_correction_request	Contradictory concurrent corrections.

10.5 Row-Level Security policies

RLS is enabled on every table in `app` and on `audit.audit_log`. Policies are written against the authenticated user's id and their role, both read from the session — never from a request parameter (DER-05).

Table group	Student	Admin	Super Admin
academic_record, grade_record, student_semester_summary, student_cumulative_summary	SELECT own rows only, and only where the grade is published	SELECT all; INSERT/UPDATE only via the service role on unlocked rows	SELECT all; no INSERT, no UPDATE, no DELETE
student, course_plan, registration	SELECT/UPDATE own rows; UPDATE only while the plan is DRAFT	Full operational access	SELECT only
college, department, course, course_prerequisite, academic_year, semester, course_offering, offering_meeting	SELECT active rows only	Full access	SELECT only (except the semester-state backward transition, which the service performs under an explicitly elevated, audited path)
grade_submission, grade_correction_request	No access	INSERT and SELECT own institution's rows; no UPDATE of the decision fields	SELECT all; UPDATE only the decision fields

Table group	Student	Admin	Super Admin
app_user	SELECT own row	SELECT and manage STUDENT rows only	SELECT all; manage ADMIN and SUPER_ADMIN rows
grade_scale, institution_setting	SELECT	SELECT; UPDATE only with Super Admin approval flow	SELECT; approve changes
audit.audit_log	No access	No access	SELECT only

ON THE RELATIONSHIP BETWEEN THE TWO LAYERS

The RLS policies above are deliberately coarser than the service-layer rules. RLS answers *"could this actor ever legitimately touch this row?"*; the service layer answers *"is this specific action legitimate right now, given the semester state, the record's lock status and who submitted it?"*. Trying to express the second question in RLS would produce policies nobody can read or test. Both are generated from the same permission table so they cannot contradict each other on the first question.

10.6 Indexing plan

Index	Table	Query it serves
(student_id, semester_id)	academic_record	Semester grade sheet; semester GPA computation.
(student_id, course_id, semester_id)	academic_record	Prerequisite satisfaction; retake detection; repeat carry-over resolution.
(student_id) INCLUDE grade columns	academic_record	Full CGPA recomputation for one student.
(offering_id, status)	registration	Class list for grade entry and printing.
(semester_id, status)	course_plan	Admin's queue of plans awaiting approval.
(status) WHERE status = 'SUBMITTED'	grade_submission	Super Admin's approval queue — a small partial index over a table that grows every semester.
(student_id, occurred_at DESC)	audit.audit_log	"Show me everything that happened to this student's record."
(entity_type, entity_id, occurred_at DESC)	audit.audit_log	"Show me the history of this grade."
(occurred_at DESC)	audit.audit_log	Chronological audit browse.
lower(trim(code))	course	Course-code lookup and duplicate prevention.
lower(student_number)	student	Student lookup during grade entry and historical import — the single most frequent admin query.
(historical_import_status)	student	Import progress report — REQ-H06.

Scale check: a college of a few thousand students accumulating roughly ten academic records per student per year produces a table in the low hundreds of thousands of rows after a decade. Every query above is an index seek at that size. No partitioning, no materialised views and no caching layer are required in Phase 1, and adding them speculatively would be over-engineering.

10.7 Migration discipline

Rule	Detail
Forward-only	Migrations are numbered, immutable once merged, and applied in order. A mistake is fixed by a new migration, never by editing an applied one.
Checked in as SQL	Every migration is readable SQL in version control, reviewed as code. No schema change is ever made through a hosted console — that is the drift risk identified in Section 6.3.
Policies and grants are migrations too	RLS policies, grants and constraints live in migrations alongside tables, so a fresh database is a complete database.

Rule	Detail
Verified at every gate	Each stage exit gate includes building a database from scratch from migrations alone and confirming it matches the deployed schema.
Reversibility	Additive migrations (new table, new nullable column, new index) are safely reversible. Destructive migrations (dropping or narrowing a column) require an explicit backup immediately before, and a written rollback note in the pull request. In Phase 1 there should be very few of these; if there are many, the domain model was wrong.
Seeds are separate	Reference seeds (grade scale, permission table) are idempotent and run in every environment. Demo data is a separate script that refuses to run against production.

11. Role and Permission Model

11.1 The model is an allow-list, not a hierarchy

The College has explicitly rejected the "Super Admin does everything" model (§3). The permission system must therefore be built as an **explicit allow-list of (role, action) pairs**. There is no notion of a role being "above" another, no `role_level` integer, and no inheritance. A role either appears in the allow-list for an action or it does not.

> **HARD CONSTRAINT**
>
> Any implementation that tests seniority — `if (role >= ADMIN)`, `isAtLeast()`, `hasRoleOrHigher()` — is incorrect for this system and must be rejected in review. The correct shape is `assertCan(actor, 'grade.enter', resource)` resolving against a table.

11.2 Role definitions

Role	Purpose (source §3)	Explicitly can	Explicitly cannot
Student	Views personal academic information; plans and submits courses for a semester and edits the plan before approval.	View own grade sheets, GPA/CGPA and schedule; build, edit and submit own course plan; change own password.	See any other student's data; see unpublished grades; change any academic record; see the audit log.
Admin	Runs day-to-day academic operations.	Enrol students; create and manage colleges, departments, courses, prerequisites, academic years, semesters, offerings and schedules; enter and edit draft grades; submit grade batches; approve or reject course plans; enter historical records; request grade corrections; advance a semester forward.	Approve grades; approve corrections; move a semester backwards or reopen it; manage Admin or Super Admin accounts; view the audit log.
Super Admin	Oversight focused on grade approval and governance.	Approve or reject grade submissions; approve or reject grade corrections; manage Admin accounts; view all institution-wide records; view the audit log; move a semester backwards or reopen a closed semester.	Enrol students; create colleges, departments, courses or semesters; enter or edit any grade; correct a grade; create or edit offerings and schedules; approve course plans.

11.3 Permission matrix

Every row is an action the system can perform. **Y** = permitted; **N** = forbidden and tested as such; — = not applicable to that role. "Approval" names the second actor required, if any. "Audit" marks actions that must produce an audit entry. "Lock" describes what becomes immutable.

Action	Stu	Adm	Super	Approval by	Audit	Locking / notes
Authentication and accounts						
Log in with Student ID	Y	—	—	—	Y	Failed attempts also logged (count, not credential).
Change own password	Y	Y	Y	—	Y	Forced on first login and after any reset.
Create student account	N	Y	N	—	Y	REQ-R04 denies Super Admin.
Reset a student password	N	Y	N	—	Y	Issues a temporary credential; forces change.
Create / disable an Admin account	N	N	Y	—	Y	REQ-A06.
Create / disable a Super Admin account	N	N	Y	—	Y	Pending DEC-15. At least one active Super Admin must always remain.

Action	Stu	Adm	Super	Approval by	Audit	Locking / notes
Academic structure						
Create / edit College	N	Y	N	—	Y	Deactivate rather than delete once referenced.
Create / edit Department	N	Y	N	—	Y	Cannot deactivate with active students.
Create / edit Course	N	Y	N	—	Y	Credit-hour change logged distinctly; does not alter existing records.
Add / remove prerequisite	N	Y	N	—	Y	Cycle check on write.
Create Academic Year / Semester	N	Y	N	—	Y	REQ-R04 denies Super Admin explicitly.
Create / edit Course Offering and schedule	N	Y	N	—	Y	Post-publication schedule changes flagged to affected students.
View academic structure	Y	Y	Y	—	N	Students see active records only.
Semester lifecycle						
Advance semester forward one state	N	Y	N	—	Y	Only along legal transitions (Section 13).
Move semester backwards	N	N	Y	—	Y	REQ-R07/W03. Reason mandatory.
Reopen a closed semester	N	N	Y	—	Y	Reason mandatory. Unlocks grade entry for that semester only.
Students and course planning						
View own academic record	Y	—	—	—	N	Published grades only.
View any student's record	N	Y	Y	—	N	Super Admin read-only (REQ-R03).
Edit a student record	N	Y	N	—	Y	Field-level old/new values captured.
Confirm an admission forfeiture	N	Y	N	—	Y	REQ-T04 / CR-09. The system only produces a candidate list; a human confirms each case with a reason. **No status ever changes automatically.**
Reverse a forfeiture (readmission)	N	Y	N	—	Y	Reason mandatory. Returns the student to Active.
Create / edit own course plan	Y	—	—	—	Y	Only while DRAFT or REJECTED, and only during Registration state.
Submit own course plan	Y	—	—	Admin	Y	Becomes read-only to the student on submission.
Approve / reject a course plan	N	Y	N	—	Y	Approval **locks** the plan and creates registrations atomically. Rejection requires a reason.
Override a failed prerequisite check	N	Y	N	—	Y	Only while the override window is open (DEC-12). Reason mandatory.
Register a student directly / drop a registration	N	Y	N	—	Y	Pending DEC-14. Blocked once the grade is published.
Grade management						
Enter / edit a draft grade	N	Y	N	—	Y	REQ-G05, REQ-R04. Invisible to students throughout.
View draft grades	N	Y	Y	—	N	Super Admin sees drafts only in the context of a submission under review.
Submit a class grade batch	N	Y	N	Super Admin	Y	Batch becomes read-only to the Admin.
Approve a grade submission **as a batch**	N	N	Y	—	Y	**Publishes and locks** every still-undecided grade in the batch; writes academic records; recomputes GPA/CGPA and repeat obligations. One transaction. Approver must not be the entering Admin.

Action	Stu	Adm	Super	Approval by	Audit	Locking / notes
Approve an **individual grade** within a submission	N	N	Y	—	Y	CR-06. Same transaction applied to the selected subset. Unselected grades stay invisible to students until decided.
Reject a grade submission **as a batch**	N	N	Y	—	Y	Returns every still-undecided grade to DRAFT; reason mandatory and visible to the Admin. Already-published grades are never un-published.
Reject an **individual grade**	N	N	Y	—	Y	That one grade returns to DRAFT with a mandatory reason. Its classmates are unaffected.
Request correction of a locked grade	N	Y	N	Super Admin	Y	REQ-R06. Old value, new value and reason captured at request time.
Approve a grade correction	N	N	Y	—	Y	Applies the change, updates the academic record and both summaries, re-locks. Approver must not be the requester.
Print class grade sheet / class list	N	Y	Y	—	Y	Printing is logged: it produces a document that leaves the system.
Historical records						
Enter a historical record	N	Y	N	—	Y	Written with `origin = IMPORTED`. Duplicate rejected by constraint.
Correct a historical record	N	Y	N	Super Admin REC.	Y	Recommended to follow the same two-key path as a grade correction once a student's import is COMPLETE.
Set import status to Complete	N	Y	N	—	Y	Removes the provisional marker from that student's figures. A consequential act — see Section 17.6.
Reopen a completed import	N	Y	N	Super Admin REC.	Y	Reason mandatory; re-applies the provisional marker.
View import progress report	N	Y	Y	—	N	REQ-H06.
Governance						
View audit log	N	N	Y	—	Y	REQ-R08. The act of reading it is itself logged.
Change the grade scale / grading policy	N	Y	N	Super Admin REC.	Y	Creates a new policy version. Never edits a version in use.
Change institution settings (credit limits, rounding, override window)	N	Y	N	Super Admin REC.	Y	Old and new values recorded.
Run the semester-end export	N	Y	Y	—	Y	Logged: a full copy of academic data leaves the system.

11.4 Enforcement obligations

Obligation	Detail
Single source	The matrix above is encoded once, as data. The service kernel and the RLS policy generator both read it. Adding an action to the system means adding a row; an action with no row is denied by default.
Deny by default	An unrecognised action, an unrecognised role, or a missing matrix row results in refusal, never in permission.
Server-side only	The client is told what to render, not what is permitted. A client that receives an incorrect answer can only render the wrong menu, never perform the wrong action.
Separation of duties re-checked in-transaction	Approver ≠ submitter and decider ≠ requester are verified inside the transaction and enforced by check constraints, not only at request entry.

Obligation	Detail
Every "N" is a test	Section 23 requires an automated test for each forbidden cell: authenticate as that role, attempt the action through the real service path, assert refusal and assert that no data changed. A stage cannot close with an untested "N".
Role immutability	A user's role cannot be changed in place. Changing what someone may do means disabling one account and creating another, so that audit attribution stays truthful.

12. Academic Structure

12.1 Reading the hierarchy correctly

The source document (§4) presents the academic structure as a single ordered list:

```
Academic Year → Semester → College → Department → Course → Course Offering / Class
```

Read literally as a containment chain, this would mean a College belongs to a Semester, which is wrong: the College of Christian Education does not cease to exist between terms. The list is in fact **two independent hierarchies that meet at the course offering**, and modelling it as one chain would produce a database that has to duplicate the entire organisational structure every semester.

Hierarchy	Chain	Nature
Time	Academic Year → Semester	Recurring. New rows every year. Carries the state machine of Section 13.
Organisation	College → Department → Course	Durable. Changes rarely. Survives across years and is referenced by records decades old.
Intersection	Semester × Course → Course Offering	The point where a durable catalogue course becomes a class actually taught in one term.

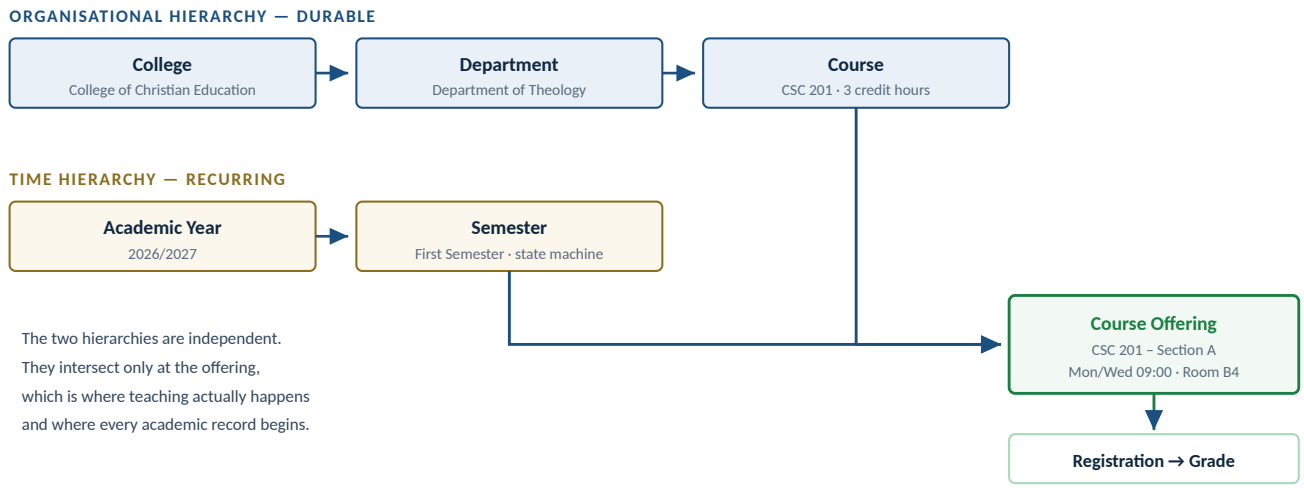


Figure 12.1 — The academic structure is two hierarchies meeting at the course offering, not one containment chain.

12.2 Terminology preservation

REQ-S02 requires the College's own vocabulary. This is enforced everywhere, not only in the interface:

LCC term	Table / field name	Common alternative — never used	Why it matters
College	`college`	Faculty, School	Staff will read error messages and export headings. Using a different word introduces doubt about whether the system means the same thing they do.
Department	`department`	Program, Programme, Major	Same.
Course	`course`	Subject, Module, Unit	Same.

LCC term	Table / field name	Common alternative — never used	Why it matters
Course Offering / Class	`course_offering`	Section, Class instance	"Class" is the College's spoken term; "Course Offering" is used in the data model for precision. Both appear in the interface, always together at first use.
Semester	`semester`	Term	Same.
Academic Year	`academic_year`	Session, School Year	Same.

12.3 Why the Course / Offering distinction is load-bearing

This one distinction determines the correct behaviour of six separate features. If a future developer collapses the two entities to "simplify", every row below breaks.

Feature	Depends on the distinction how	What breaks if they are merged
Course registration	A student registers into a specific **offering** (section), because sections meet at different times and may have different instructors.	Students cannot choose between sections; the schedule-conflict check becomes meaningless.
Scheduling	Meeting times belong to the **offering**, not the course. CSC 201 has no time; CSC 201 – Section A does.	A course could only ever be taught once per semester, at one time.
Grades	A grade attaches to a registration in an **offering**, which is what makes a class grade sheet possible.	No class list exists to print or to enter grades against — REQ-G02 and REQ-G04 become impossible.
Prerequisites	Prerequisites belong to the **course**. "CSC 201 requires CSC 101" is true regardless of section or semester.	The prerequisite rule would have to be restated for every section every semester, and would drift.
Historical records	An imported record references the **course** and the semester. The old section is usually unknown and does not matter.	Historical import would require inventing section data that does not exist on paper.
GPA / CGPA and retakes	"Has this student passed CSC 201?" and "is this a repeat of CSC 201?" are **course**-level questions across all semesters and all sections.	A student who passed CSC 201 – Section A and later took Section B would appear to have taken two different courses; the carry-over rule (REQ-C05) would never fire.

DESIGN RULE

Anything true of the subject regardless of when it is taught belongs on `course`. Anything true only of one term's delivery belongs on `course_offering`. Credit hours are the one deliberate exception: they are defined on the course, but **copied** onto the offering and again onto the academic record, so that changing the catalogue never rewrites history (DER-07).

12.4 Structural rules

#	Rule	Rationale and classification
1	A course belongs to exactly one department, but any student may register for it.	Departments own curriculum; they do not own students' choices. ASSUMPTION ASM-06 — the source does not restrict cross-department registration, and restricting it would be a new rule.
2	A student belongs to exactly one department at a time.	Follows from REQ-T01. Department changes are audited and do not alter past records.
3	Credit hours are a whole number greater than zero, defined on the course.	REQ-S04. Confirmed: no fractional credits (DEC-32). A BSc requires **132 credit hours earned**; a semester plan may not exceed **21** (CR-04).

#	Rule	Rationale and classification
4	An academic year contains **exactly two** ordered semesters. No summer or vacation term.	Confirmed (CR-10, DEC-11). The model still stores semesters as ordered rows, so a third term would be data rather than a schema change — but none exists and none is seeded.
5	An offering exists only within one semester and cannot span semesters.	Follows from §4. A year-long course would be modelled as two offerings, or the College must tell us otherwise.
6	Section identifiers are per offering and need only be unique within (semester, course).	Section "A" recurs every semester; that is expected, not a duplicate.
7	Prerequisites form a directed acyclic graph.	A cycle makes the affected courses permanently unregisterable. Checked on every write.
8	Deactivating structural data hides it from new use but never from history.	DER-22.

12.5 Major courses

The College's repeat rule — "any D in a major course must be repeated" (CR-05) — requires the system to know whether a course is *major* for a given student. Nothing in v4.0 defined that, so it is defined here.

Element	Specification
Definition	A course is **major** for a student when the course's owning department is the student's own department. It is a relationship between a course and a student, not a property of the course alone — CSC 201 is a major course for a Computer Science student and a non-major course for a Theology student.
Evaluated once, frozen	The determination is made **at the moment the result is written** — at grade publication or at historical entry — and stored on the academic record as `was_major_at_record`. It is never recomputed.
Why frozen	A student who changes department must not retroactively acquire or lose repeat obligations for work already completed. Freezing is the only treatment that keeps a student's obligations stable across a department change.
Displayed	The student record marks which completed courses were major, so an obligation is always explicable rather than mysterious.

> **ASSUMPTION ASM-20 — CONFIRM BEFORE STAGE 7**
>
> "Major course" is read here as **a course owned by the student's own department**, because that is the only reading the data model supports without new information. If the College instead means a curated list of core courses within a programme, the rule would fire on a different — probably larger — set. The fix is small if caught early: an `is_core` flag on the course record and one clause in the obligation rule. It is worth one question to the Registrar before Stage 7 begins.

12.6 Admission forfeiture

New requirement, not present in v4.0: **a student who is granted admission and fails to register for two consecutive semesters forfeits the admission** (CR-09). With exactly two semesters per year, that is a full academic year of absence.

Element	Specification
Trigger condition	The student has no approved registration in either of the two most recently closed semesters, and their status is Active.
When it is evaluated	At semester close. There are no background jobs on the chosen platform (Section 7.4), and none is needed — twice a year is exactly the right cadence for a twice-a-year rule.
What the system does	**Produces a candidate list, and nothing else.** Reviewing it is a step in the semester-close runbook. No status changes automatically.
What a human does	An Admin confirms each forfeiture individually, with a reason, setting the status to `ADMISSION_FORFEITED`. The action is audited with actor, reason and the two semesters relied upon.

Element	Specification
Why not automatic	Forfeiting an admission is consequential and, in the student's eyes, irreversible. A student may have deferred for a reason the system does not know — illness, family circumstance, a leave the College handled on paper. An automatic status change would make the software the decision-maker on something that is properly the College's judgement. The system's job is to ensure nobody is *missed*, not to decide.
Effect of the status	Cannot build a course plan; retains read-only access to any academic history already recorded. Reversible by an Admin, with a reason, if the College readmits.

ASSUMPTION ASM-21

"Two inactive semesters" is read as **two consecutive semesters with no approved registration**. If the College means something else — two semesters counted from the admission date regardless of gaps, for instance — the candidate list identifies the wrong students. Because forfeiture is Admin-confirmed rather than automatic, a wrong list is caught by a human before it does harm, which is one more reason for the manual step. Confirm the counting rule before Stage 5.

13. Semester State Machine

The source is explicit that "clear semester states are used instead of a simple open/close switch" and that the state "controls when students may register, when Admins may approve course plans, when grades may be entered, and when records become locked" (§7). The semester state is therefore a precondition consulted by four separate services, and it is modelled as a formal state machine with an enumerated set of legal transitions.

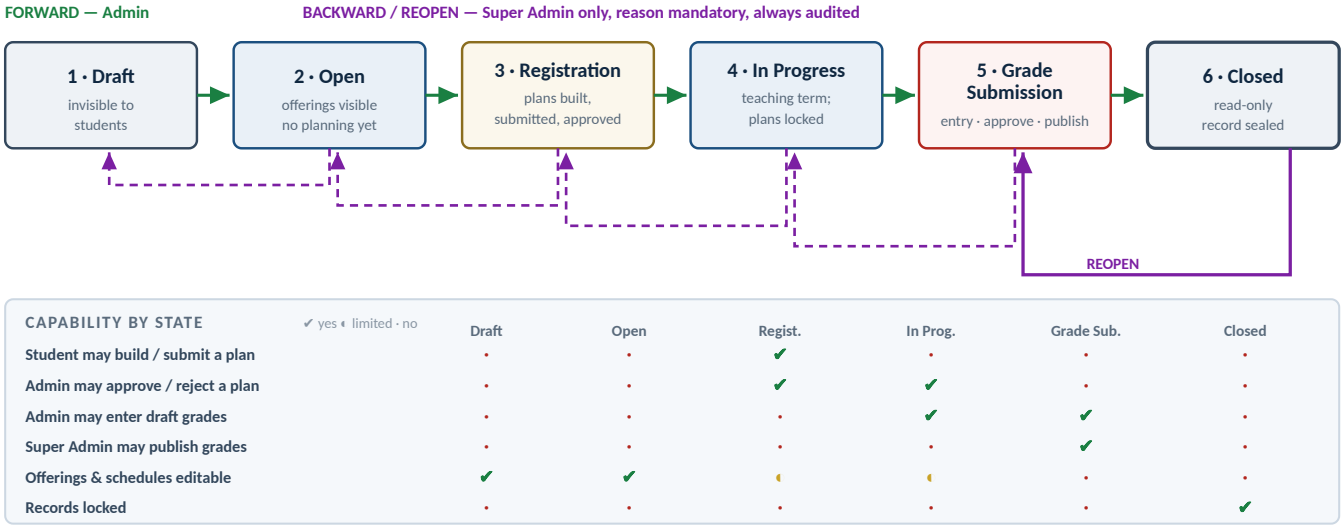


Figure 13.1 — Semester state machine. Forward transitions are routine Admin operations; every backward or reopening transition is a Super Admin act requiring a reason and producing an audit entry (REQ-W03, REQ-W04).

13.1 State specifications

State 1 — Draft

Purpose	The semester is being set up. Offerings, sections, schedules and dates are prepared before anyone sees them.
Allowed	Admin: create and edit the semester; create, edit and delete offerings and meeting times; delete the semester entirely if nothing depends on it.
Restricted	No student visibility of any kind. No plans, no registrations, no grades.
Student	Cannot see the semester exists.
Admin	Full setup authority.
Super Admin	Read only.
Valid transitions	→ Open (Admin). This is also the only state from which a semester may be deleted.
Invalid	Any jump past Open.
Audit	Creation; transition to Open.

State 2 — Open

Purpose	The semester and its offerings are published so students can see what will be available, but course planning has not started. RECOMMENDATION — the source lists Open and Registration as distinct but does not define the difference (GAP-13). This definition gives Open a real purpose: an advertised catalogue and timetable ahead of the registration window.
Allowed	Students browse offerings and schedules. Admin continues to adjust offerings, with changes now visible.
Restricted	No plan may be created or submitted. No registration exists yet.
Student	Read-only browse of the semester's offerings and timetable.

Admin	Edit offerings and schedules; may still cancel an offering with no registrations.
Super Admin	Read only; may move the semester back to Draft with a reason.
Valid transitions	→ Registration (Admin); → Draft (Super Admin, reason).
Invalid	→ In Progress, Grade Submission, Closed.
Audit	Transition in and out; every offering change now that students can see it.

State 3 — Registration

Purpose	The course-planning window. Students build and submit plans; Admin approves or rejects them.
Allowed	Student: create, edit, submit and revise a plan (REQ-P01, REQ-P02). Admin: approve or reject plans with a reason; register a student directly if DEC-14 is approved; still add offerings, with a warning that students may already have planned.
Restricted	No grade entry. Cancelling an offering with registrations is refused; it must be handled by dropping registrations explicitly, which is audited.
Student	Full planning authority over their own plan only, and only while it is DRAFT or REJECTED.
Admin	Approval authority. Approval creates registrations atomically (DER-12).
Super Admin	Read only; may move back to Open with a reason, which suspends planning without discarding plans.
Valid transitions	→ In Progress (Admin); → Open (Super Admin, reason).
Invalid	→ Grade Submission or Closed directly — grades cannot be entered for a term that never ran.
Audit	Every submission, approval, rejection with reason, prerequisite override, direct registration and drop.

State 4 — In Progress

Purpose	The teaching term. Registration is settled; grades are not yet being collected as a batch, but early entry is permitted so the office is not forced to wait.
Allowed	Admin: enter and edit **draft** grades; approve any plans still outstanding from the registration window; perform audited late adds and drops (DEC-14).
Restricted	Students cannot create or edit plans. No grade may be submitted for approval yet. RECOMMENDATION — allowing draft entry here but not submission keeps a clear institutional boundary at the point where results become reviewable.
Student	View approved plan, registrations and timetable. No grades visible (none are published).
Admin	Draft grade entry; late add/drop with reason.
Super Admin	Read only; may move back to Registration with a reason.
Valid transitions	→ Grade Submission (Admin); → Registration (Super Admin, reason).
Invalid	→ Closed directly, which would seal a semester with no grades.
Audit	Every draft grade entry and edit; every late add or drop.

State 5 — Grade Submission

Purpose	The collection and approval window. Admin completes entry and submits class batches; Super Admin reviews, approves and thereby publishes.
Allowed	Admin: finish draft entry; submit class batches; resubmit after rejection; request corrections on already-published grades. Super Admin: approve or reject submissions; approve or reject corrections.
Restricted	No new registrations. No plan changes. Offerings and schedules are frozen.
Student	Sees grades **only** as each class batch is approved (REQ-G07, GAP-21).
Admin	Entry and submission; cannot approve.

Super Admin	Approval authority; may move back to In Progress with a reason if the term genuinely has not finished.
Valid transitions	→ Closed (Admin, but only when no submission is still awaiting approval); → In Progress (Super Admin, reason).
Invalid	→ Closed while any submission is pending. The service refuses and names the outstanding classes.
Audit	Every submission, approval, rejection, publication, lock and correction.

State 6 — Closed

Purpose	The semester is final. Its records are sealed and become permanent history.
Allowed	Reading, printing and exporting. Nothing else.
Restricted	No grade entry, no submission, no plan change, no registration change, no offering change. A correction to a published grade in a closed semester requires the semester to be reopened by a Super Admin first. REQUIRES APPROVAL DEC-33 — the alternative is to allow the ordinary correction workflow to operate on closed semesters. This plan recommends requiring the reopen, because it makes any change to a sealed term a deliberate, visible institutional act.
Student	Full read access to their own published results and final semester GPA.
Admin	Read and print only.
Super Admin	May reopen, with a mandatory reason. Reopening returns the semester to Grade Submission.
Valid transitions	→ Grade Submission (Super Admin reopen, reason).
Invalid	Any other transition, and any data change while closed.
Audit	Closure; every reopen with reason; the semester-end export.

13.2 Transition table

From	To	Who	Reason req.	Guard conditions checked before the transition is allowed
Draft	Open	Admin	No	Semester dates set and within the parent academic year.
Open	Registration	Admin	No	At least one published offering exists. No other semester is currently in Registration (recommendation, 13.6).
Registration	In Progress	Admin	No	Warns — does not block — if plans remain unapproved, listing them. Those plans remain approvable in In Progress.
In Progress	Grade Submission	Admin	No	No other semester is currently in Grade Submission (recommendation).
Grade Submission	Closed	Admin	No	**Blocks** if any submission is SUBMITTED and undecided, or any correction is PENDING. Warns if any registration has no published grade, listing them.
Open	Draft	Super Admin	Yes	No plans exist.
Registration	Open	Super Admin	Yes	Existing plans and registrations are retained, not discarded. Students lose edit ability until Registration resumes.
In Progress	Registration	Super Admin	Yes	No grades have been submitted for approval.
Grade Submission	In Progress	Super Admin	Yes	No submission is currently awaiting a decision. Published grades stay published — publication is never undone by a state change.
Closed	Grade Submission	Super Admin	Yes	None. This is the reopen path (REQ-W03). It unlocks grade operations for that semester only.

Every transition not listed above is invalid and is rejected by the service with a message naming the current state and the legal moves from it. There is no "force" path and no administrative override outside this table.

13.3 Reopening and moving backwards

The source requires that only a Super Admin may move a semester backwards or reopen it, and that every such action is audited. Three further rules are necessary to make that safe, and are introduced here:

#	Rule	Why
1	**Backward transitions never destroy data.** Moving from Registration to Open does not delete plans; moving from Grade Submission to In Progress does not unpublish grades.	A state change is an administrative correction, not a data-recovery tool. If the College needs to undo published grades, that is the correction workflow, which is individually audited and reviewed.
2	**Published grades survive every backward transition.** Once a grade is published and locked, only an approved correction changes it.	Otherwise a single state change could silently alter thousands of student records and every CGPA derived from them.
3	**A reason is mandatory and is shown in the audit log alongside the from-state and to-state.**	REQ-W04. A reopen without a stated reason is indistinguishable from tampering.

13.4 Where the state is consulted

Service operation	Required state	Failure behaviour
Create or edit a course plan	Registration	Refuse with "Course planning is not open for this semester."
Submit a course plan	Registration	Refuse; the student's in-progress plan is preserved as DRAFT.
Approve or reject a course plan	Registration or In Progress	Refuse.
Enter or edit a draft grade	In Progress or Grade Submission	Refuse.
Submit a class grade batch	Grade Submission	Refuse.
Approve a grade submission	Grade Submission	Refuse.
Request or approve a grade correction	Grade Submission (or a reopened semester)	Refuse, with guidance that the semester must be reopened by a Super Admin.
Create or edit an offering or schedule	Draft, Open or Registration	Refuse; schedules are frozen once teaching starts.
Enter a historical record	**Any state**, for past semesters	Permitted always. Historical import is deliberately independent of the state machine — see Section 17.4.

Every one of these checks lives in the service layer, is re-verified inside the transaction, and has a corresponding automated test that attempts the operation in a wrong state and asserts refusal.

13.5 What the state machine does *not* control

Not controlled by semester state	Why
Historical record entry	Historical records belong to past semesters that are already Closed. Gating import on state would make the largest work item in the project impossible.
Reading published results	A student's access to their own history does not expire when a term closes.
Student enrolment and account management	Independent of any particular semester.
Academic structure changes	Colleges, departments and courses are durable and outlive semesters (Section 12.1).

13.6 Concurrent semesters

The model permits several semesters to exist simultaneously in different states — for example 2026/2027 First Semester in Grade Submission while Second Semester is already in Registration. This is normal and necessary.

> **RECOMMENDATION**
>
> Enforce two invariants: **at most one semester in Registration** and **at most one in Grade Submission** at any time. Neither is stated in the source, but both prevent a class of confusing operational errors — a student registering for the wrong term, or an Admin entering a class's grades against last semester's offering. A single Admin office should never legitimately need two of either window open. If the College's calendar requires overlapping registration windows, this recommendation should be withdrawn rather than worked around. REQUIRES APPROVAL DEC-34

14. Course Planning Workflow

14.1 Workflow overview

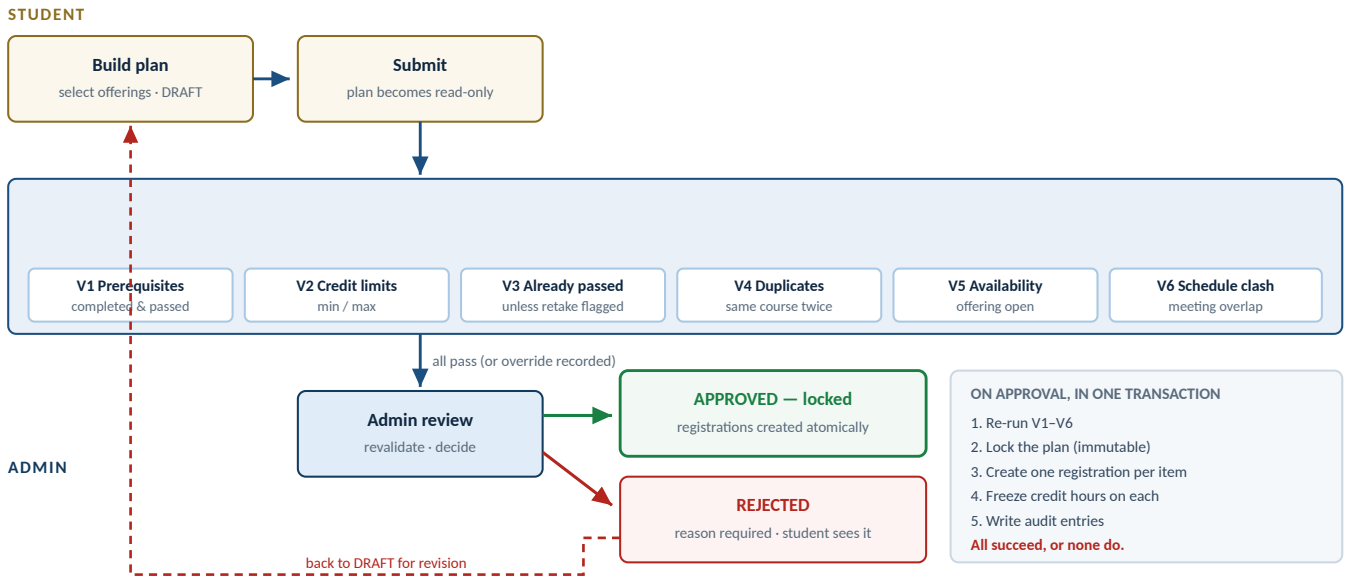


Figure 14.1 — Course planning workflow. The validation gate is the same code invoked at both enforcement points required by REQ-P03; approval is a single atomic operation that locks the plan and creates registrations.

14.2 Plan state machine

State	Student may	Admin may	Transitions out	Notes
DRAFT	Add, remove, edit items; delete the plan; submit	View	→ SUBMITTED (student); deleted (student)	Invisible in the Admin approval queue. Exists only while the semester is in Registration.
SUBMITTED	View only	Approve or reject	→ APPROVED; → REJECTED	REQ-P02's "editable until approved" is honoured by the rejection path, which returns the plan to DRAFT. Direct editing while a decision is pending would let a student change what the Admin is reviewing.
REJECTED	View reason; revise (returns to DRAFT)	View	→ DRAFT (student edits)	The rejection reason and its author are retained on the plan and in the audit log, not overwritten by the next submission.
APPROVED	View only	View; drop/add individual registrations if DEC-14 is approved	Terminal	Locked (REQ-P02). Subsequent changes are made to registrations, not to the plan, so the approved plan remains an accurate record of what was approved.

ON "EDITABLE UNTIL APPROVED"

REQ-P02 says the plan can be edited until it is approved. Two readings are possible: editable right up to the moment of approval, including while under review; or editable until submitted, then again if rejected. This plan implements the second, because the first allows a student to alter a plan an Admin is actively reviewing, producing an approval of something nobody read. The rejection path preserves the student's ability to revise. RECOMMENDATION — flagged for confirmation as DEC-35.

14.3 The six validations

REQ-P03 requires these to run before submission **and again** before approval. They are one function called from two places, never two implementations.

ID	Rule	Precise definition	Data it reads / failure message
V1	Prerequisites satisfied REQ-P04	For each selected course, every prerequisite must appear in the student's `academic_record` with a grade at or above the required threshold, in a semester earlier than the target semester.	Reads academic_record (both origins). Fails as: "CSC 201 requires CSC 101, which is not recorded as passed." Overridable by Admin while the override window is open (REQ-P11).
V2	Credit-hour ceiling REQ-P05, REQ-C12	Sum of `frozen_credit_hours` across all plan items must be **≤ 21** and the plan must contain at least one item. There is no meaningful minimum load (CR-04). A department may set a lower ceiling but never a higher one.	Institution setting `max_credits_per_semester = 21`. Fails as: "This plan totals 24 credit hours; the maximum is 21." An empty plan fails as: "Add at least one course before submitting."
V3	Already completed and passed REQ-P06	If the student already has an academic_record for the course with grade point ≥ 0.70, the item is rejected **unless** `is_retake` is set. A prior F does not trigger this rule — and neither does a D that carries an outstanding mandatory-repeat obligation, which is instead pre-flagged as a retake automatically.	Fails as: "You have already passed CSC 101 (grade B−). Mark it as a retake if you intend to repeat it." Where the prior grade is an F, or a D in a major course, the retake flag is set for the student rather than demanded of them.
V7	Outstanding mandatory repeats REQ-C11 — warning, not a block	If the student has outstanding repeat obligations (an F anywhere, or any D in a major course) that the plan does not address, the plan still validates but carries a warning naming each course.	Shown to the student as "THE 210 must be repeated — it is not in this plan", and to the Admin at approval. **Approval is permitted.** Hard-blocking would strand students whose historical import is incomplete, which in year one is nearly all of them. The warning and the approval decision are both audited.
V4	No duplicate courses REQ-P07	No two items in the plan may reference the same course, even in different sections.	Also enforced by a unique index. Fails as: "CSC 201 appears twice in this plan."
V5	Course availability REQ-P08	The referenced offering must exist, belong to the target semester, have status PUBLISHED, and not be cancelled. If capacity is set (GAP-26), remaining seats must be > 0.	Fails as: "CSC 201 – Section A is no longer available for this semester."
V6	Schedule conflicts REQ-P09	No two selected offerings may have meeting times that overlap on the same day. Overlap is any intersection of [start, end) intervals.	Reads offering_meeting. Fails as: "CSC 201 – Section A (Mon 09:00–10:30) clashes with THE 110 – Section B (Mon 10:00–11:30)."

14.4 When each validation runs

Moment	Runs	Blocking?	Purpose
As the student adds an item (client feedback)	V4, V6 locally; V1, V3, V5 via a server call	Advisory	Immediate feedback. Never authorises anything. The client's copy of V4/V6 is presentation logic over data the server supplied, not a second rule implementation.
On submit (server)	V1–V6	**Blocking**	REQ-P03, first enforcement point. Submission is refused with a list of every failure, not just the first.
On approve (server, in-transaction)	V1–V6	**Blocking**	REQ-P03, second enforcement point. Conditions change between submission and approval: an offering may be cancelled, a schedule may move, another plan may fill the last seat. Re-validation is what makes the approval trustworthy.
On Admin direct registration (DEC-14)	V1, V3, V5, V6	Warn, not block	An administrative act with a recorded reason may legitimately deviate; the Admin must see what they are overriding.

14.5 The prerequisite override

REQ-P11 permits an Admin to override a failed prerequisite check "during the first semester of use", because prerequisite checking depends on historical data that will not yet be complete. Three controls make this safe:

Control	Detail
Time-bounded	Governed by `institution_setting.prerequisite_override_enabled` and an expiry date, set by a Super Admin. When the window closes, the override option disappears entirely rather than remaining available but discouraged. RECOMMENDATION DEC-12.
Per item, not per plan	The Admin overrides one specific failed prerequisite on one specific course, with a reason attached to that plan item. There is no "approve anyway" button for a whole plan.
Fully audited and visible	Reason, actor, timestamp, the course, and the specific prerequisite that was not satisfied. The override remains visible on the student's record afterwards, so that a later question about how they were admitted to a course has an answer.

> **ASSUMPTION ASM-07**
>
> Overrides apply only to V1 (prerequisites). The credit-hour limits, duplicate, availability and schedule-conflict rules are **not** overridable in Phase 1, because none of them depends on incomplete historical data. If the College wants an override for credit limits — for example, a graduating student needing one extra course — that is a separate decision, recorded as DEC-36.

14.6 Edge cases

#	Case	Defined behaviour
1	Student has no historical records at all (import not started)	V1 fails for every course with a prerequisite. This is correct and must not be silently softened. The override path exists precisely for this situation, and the Admin sees "prerequisite cannot be verified — this student's historical import is Not started" rather than a bare failure.
2	Student's import is In progress and the prerequisite is genuinely missing	Indistinguishable from case 1 by data alone. The message names the import status so the Admin knows whether to override or to finish the import first.
3	Offering is cancelled after submission but before approval	V5 fails at approval. The Admin rejects with a reason naming the cancellation; the student revises.
4	Schedule changes after approval, creating a clash	Registrations are not retroactively invalidated. The affected students' timetables show a clash indicator and the Admin is listed the affected registrations so the office can act. Automatically dropping a student would be worse than showing the conflict.
5	Student submits, is rejected, revises, and resubmits	The same plan row is reused; each submission and decision is a separate audit entry. Plan history is reconstructable from the audit log.
6	Two students compete for the last seat in a capped offering	Capacity is checked inside the approval transaction with a row lock on the offering. The second approval fails cleanly with "no seats remaining" rather than over-filling.
7	Retake of a course the student failed	V3 does not fire — a failed course is not "completed and passed". The `is_retake` flag is set automatically and shown to the student, so the carry-over rule is visible before they commit.
8	Retake of a course the student passed	V3 fires; the student must explicitly mark it as a retake. On approval, the resulting academic record gets `attempt_no = 2` and the earlier attempt will be dropped from CGPA once the new grade publishes (REQ-C05).
9	Semester moves out of Registration mid-edit	The save is refused with a clear message; the draft is preserved. Nothing is lost, and the student is not told a false success.
10	Student's status changes to INACTIVE while a plan is pending	Approval is refused. The Admin sees the status as the reason.
11	The same prerequisite is satisfied only by an imported record with an illegible grade	The imported record carries a `source_note`. V1 uses the recorded grade; the note is surfaced to the Admin at review so a doubtful pass is a visible decision rather than an invisible one.

#	Case	Defined behaviour
12	Plan totals exactly the minimum, then an offering is cancelled	V2 fails at approval. Correct: an under-loaded student should not be approved by accident.

15. Grade Management Workflow

15.1 The lifecycle

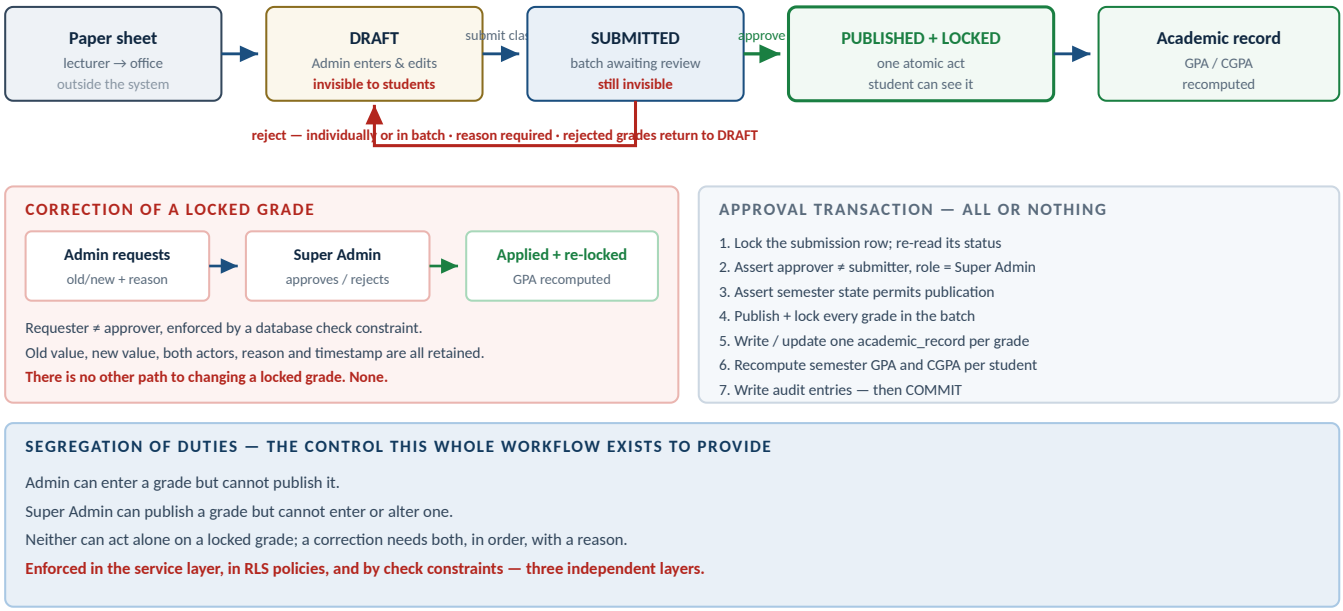


Figure 15.1 — Grade lifecycle from paper sheet to sealed academic record, with the correction path and the approval transaction. Every arrow crossing from one actor to the other is a control point, not a handoff.

15.2 Grade record states

State	Student sees	Admin may	Super Admin may	Exit
DRAFT	**Nothing**	Enter, edit, clear	**Nothing**	→ SUBMITTED, when the class batch is submitted
SUBMITTED	**Nothing**	View only	Approve or reject **this grade**, or the whole batch at once	→ PUBLISHED+LOCKED on approval; → DRAFT on rejection, with a reason
PUBLISHED + LOCKED	**The grade**	Request a correction	Approve or reject a correction	Terminal, except through an approved correction

Publication and locking are a single state, not two. REQ-G07 describes approval as one act that both publishes and locks; representing them separately would create a reachable state in which a grade is visible to a student but still editable by an Admin, which is exactly what the two-key control exists to prevent.

15.2.1 Individual and batch approval (CR-06)

The College has confirmed that a Super Admin may approve grades **individually as well as in batch**. The state machine above is unchanged by this — what changes is that the decision is recorded **per grade**, and a batch action is simply the same decision applied to every undecided grade in the submission at once.

Element	Specification
Decision is per grade	Each `grade_record` carries `decided_by`, `decided_at` and, when rejected, `decision_reason`. A batch approval writes the same actor and timestamp to every grade it covers, and every grade is individually attributable afterwards.

Element	Specification
Batch approve	One action, one transaction: every SUBMITTED grade in the submission publishes and locks; academic records are written; every affected summary and repeat obligation recomputes. This remains the normal path and should be the default the screen offers.
Individual approve	The chosen grades publish and lock in one transaction; the rest stay SUBMITTED, invisible to students, awaiting a decision.
Individual reject	That grade returns to DRAFT with a mandatory reason visible to the Admin. **Its classmates are unaffected** — already-approved grades stay published.
Submission status	`SUBMITTED → PARTIALLY_DECIDED → CLOSED`. A submission closes only when no grade in it is still awaiting a decision. Any grade returned to DRAFT is picked up by the Admin's next submission for that class, as a new attempt.
Semester close guard	A semester cannot move to Closed while any submission is SUBMITTED or PARTIALLY_DECIDED. The blocking message names the classes and the undecided grade count.
Segregation of duties	Unchanged and enforced per grade: `decided_by ≠ entered_by` as a check constraint, in addition to the submission-level constraint. Neither route lets one account complete an end-to-end grade change.

> **WHY PER-GRADE RATHER THAN PER-BATCH DECISIONS**
>
> Version 1.0 recommended batch-only approval and explicitly ruled out partial approval, on the grounds that a mixed submission is hard to reason about. The College has asked for both, and that is the better answer — a single transcription error in a class of forty should not force thirty-nine correct grades back to the Admin. The cost is that "partially decided" becomes a real state the interface and the semester-close guard must handle honestly, which is what 15.2.1 specifies.

15.3 Grade entry

Aspect	Specification
Entry unit	One class at a time — the offering's full registered list on one screen (REQ-G02). Never one student at a time, because that is not how a paper sheet is read.
Row ordering	Matches the printed class sheet the office holds. Default alphabetical by surname, switchable to Student ID order; the printed sheet and the entry screen always use the same ordering (GAP-08).
Input	**Numeric score, 0–100, decimals permitted** (CR-07). The system rounds half-up to a whole number to select the band, derives the letter and grade point, and shows both live beside the field — so 96 immediately reads "A+ · 4.00". The entered mark is stored verbatim to one decimal place; only the band lookup uses the rounded value. **I (Incomplete)** is selectable as a letter instead of a mark. There is no Withdrawal option (CR-02).
Keyboard flow	Enter or Down moves to the next student. No mouse required to complete a class. This is a stated efficiency requirement in §9, not a nicety.
Save behaviour	The whole class saves as one transaction with one idempotency key. Partial rows are not written. A dropped connection leaves the previous saved state intact and the screen shows what was and was not saved.
Validation at entry	Score must be between 0 and 100 with at most one decimal place; the derived letter must exist in the active scale; the student must be REGISTERED in this offering; the offering must belong to a semester in In Progress or Grade Submission; no grade may already be published for this registration. A score above 100 or below 0 is refused outright, and W, AU and TR are not offered anywhere.
Duplicate prevention	The unique constraint on `registration_id` makes a duplicate grade impossible at the data layer, not merely unlikely at the UI layer (REQ-G03).
Missing grades	A blank is permitted while DRAFT. At submission the system reports how many students have no grade and requires the Admin to either complete them or explicitly confirm a partial submission with a note. Silent partial submission is not allowed.
Concurrency	Optimistic version token per grade row. If two Admins open the same class, the second save is refused with a message naming the rows that changed, rather than overwriting (DER-19).

Aspect	Specification
Print	A print view of the class list (for the lecturer to fill in) and of the entered grade sheet (for the file), both generated from the same ordering and data as the screen (REQ-G04, DER-24).

15.4 Submission and approval

Step	Specification
Submission	Admin submits the class. A `grade_submission` row is created; every draft grade in the class is attached to it and moves to SUBMITTED, becoming read-only to the Admin. The Admin cannot approve their own submission — the action does not exist for that role at all.
The review screen	Super Admin sees: the offering, instructor, semester, submitting Admin and time; every student with score, derived letter and grade point; a per-row approve/reject control **and** an "approve all" action; the grade distribution; and flags for anything unusual — a class with no failures, a class where every grade is identical, any mark within one point of a band boundary, any student with no grade. These flags support review; they never block it. RECOMMENDATION
Approval — batch	**The default action.** "Approve all" covers every grade in the submission still awaiting a decision. The transaction in Figure 15.1 executes; on commit those grades are visible to students and immutable.
Approval — individual	The Super Admin selects specific grades and approves them. Same transaction, applied to the selected subset. Unselected grades stay SUBMITTED and invisible to students until decided. CR-06.
Rejection — individual	Requires a reason. That one grade returns to DRAFT and appears in the Admin's rework queue with the reason attached. **Other grades in the same class are untouched**, whether already approved or still pending.
Rejection — batch	Requires a reason. Every still-undecided grade in the submission returns to DRAFT. Grades already approved in an earlier pass remain published — approval is never undone by a later rejection.
Partial submissions	A submission whose grades are partly decided sits at `PARTIALLY_DECIDED` and is shown as such in both queues, with a count of what remains. It closes only when nothing is outstanding, and it blocks the semester from closing until then (15.2.1).
Resubmission	Grades returned to DRAFT are re-entered and submitted again as a new attempt with an incremented `attempt_no`. Every earlier submission and decision is retained as evidence that review happened.
Notification	None. There is no messaging in Phase 1 (OOS-06). Both queues — plans awaiting approval, submissions awaiting approval — are visible on the respective landing pages, which is how each actor learns there is work to do.

15.5 Corrections

Aspect	Specification
Trigger	A grade is PUBLISHED and LOCKED and is found to be wrong — a transcription error, a lecturer's correction, a resolved Incomplete.
Request	Admin opens the published grade, enters the new value and a mandatory reason. The system captures the old score, old letter and old grade point at that moment. Status PENDING.
Constraint	One PENDING request per grade. A second request while one is pending is refused.
Decision	Super Admin approves or rejects, with an optional note. Requester ≠ decider, enforced by check constraint.
Staleness check	At approval the system re-reads the grade and compares it to the captured old value. If it has changed in the interim, the request is rejected as stale rather than applied blindly.
Application	In one transaction: update the grade record; update the linked academic_record (frozen credit hours unchanged, grade point re-derived from the scale version in force on that record); recompute that student's semester summary and cumulative summary; re-lock; write audit entries capturing old value, new value, both actors, reason and timestamp (REQ-G09).

Aspect	Specification
Cascading effects	A correction can change a CGPA, can change which attempt of a repeated course counts, and can change whether a prerequisite is satisfied for a plan already approved. The first two are recomputed automatically. The third is **not** retroactively enforced — an already-approved plan is not invalidated — but the affected registrations are listed to the Admin so the office can decide. REQUIRES APPROVAL DEC-37.
Rejection	Nothing changes. The request remains as evidence that a change was proposed and declined, with the reason.
Closed semesters	A correction in a Closed semester requires a Super Admin to reopen it first (Section 13.1, DEC-33).

15.6 Grade management edge cases

#	Case	Defined behaviour
1	A student is registered but never attended and the sheet has no grade	Blank is permitted in draft. At submission the Admin must confirm the omission with a note, or record **I (Incomplete)** — the only non-participation grade the College uses (CR-02). There is no Withdrawal grade.
2	Lecturer's sheet lists a student who is not registered	Entry is refused — there is no registration to attach the grade to. The Admin must first register the student (an audited administrative act) or correct the sheet. The system does not silently create registrations from grade entry.
3	Two classes of the same course; a student appears on both sheets	Impossible by constraint: V4 prevents duplicate courses in a plan and the unique registration constraint prevents double registration. If the paper says otherwise, the paper is wrong and the discrepancy is surfaced, not absorbed.
4	Connection drops mid-entry of a 60-student class	Nothing is saved unless the whole batch save succeeded. On reconnect the screen shows the last committed state. The idempotency key means a retried save does not double-write.
5	Admin submits, then notices an error before approval	The Admin cannot edit a submitted batch. They ask the Super Admin to reject it; rejection returns every grade to DRAFT. This is deliberate friction: it keeps a record that the batch was withdrawn.
6	The same person holds both an Admin and a Super Admin account	The software cannot prevent this, but the check constraints ensure the same *account* cannot both submit and approve. The College should be advised that holding both accounts defeats the control it asked for. Recorded as a risk in Section 28.
7	A grade is published and the student disputes it	The correction workflow is the answer. The published grade remains visible and unchanged until a correction is approved; nothing is quietly hidden while the dispute runs.
8	A published grade's course credit hours are later edited	No effect on the record. Credit hours were frozen at publication (DER-07). The Admin is warned of this at the moment of editing the course.
9	The grade scale is changed after grades are published	No effect on existing records — they carry a frozen grade point and a policy version. New grades use the new scale. A deliberate recomputation of historical records under a new policy is a separate, explicitly approved operation, never a side effect.
10	An offering has no registered students	Entry screen shows an explicit empty state; submission is refused with "this class has no registered students".
11	A registration is dropped after a draft grade was entered	The draft grade is cleared as part of the drop, and the drop is audited with both facts recorded. A drop is refused outright once the grade is published.
12	A correction changes a grade from F to a pass on a course that is a prerequisite	CGPA and credits earned recompute automatically. Future prerequisite checks pass. Already-approved plans are untouched but listed (DEC-37).

16. GPA / CGPA Architecture

GPA and CGPA are the numbers the College will be judged on. They appear on transcripts, determine progression and graduation, and will be disputed by individual students. This section treats the calculation as a critical domain service with its own specification, configuration surface and test suite.

> **VERSION 2.0 — FULLY SPECIFIED**
>
> Every rule in this section is now a confirmed number supplied by the College (Section 1.6, CR-01 to CR-05 and CR-07). Nothing here is inferred. The scale below **replaces** the five-letter scale in System Overview v4.0 §10.

16.1 Design position

Position	Consequence
A pure function	The engine takes a list of academic records and a policy object and returns figures. No database access, no authorisation, no I/O. Exhaustively testable with fixtures; cannot be broken by a change to a page or a query (DER-09, P3).
Policy is data, not code	The scale, the passing threshold, the repeat rules, the rounding precision and the graduation total are all rows and settings, versioned. A future policy change is configuration plus a deliberate recomputation, not a rewrite.
Exact decimal arithmetic throughout	Grade points now carry two decimal places (3.70, 2.30, 0.70) and GPA three. Binary floating point would produce 3.6999999999999997 and disputes. No float appears anywhere in the path, including JSON serialisation (DER-08, TEC-09).
Only published grades count	REQ-C02. Draft and submitted grades are excluded by the query and independently by RLS. A student's GPA can never move because someone typed a number into a draft.
Every figure carries its provenance	Each value is returned with `is_provisional` and `policy_version`. Given the College's decision to import historical records gradually after go-live (CR-11), **most figures will be provisional for most of year one** — so this flag is a primary feature, not a corner case.

16.2 The grading scale

Confirmed by the College, 25 August 2026. Nine letter grades plus Incomplete. **Withdrawal (W), audit/non-credit and transfer credit are not used and must not appear anywhere in the system** (CR-02).

Letter	Score range	Grade point	Passing	In GPA	In attempted	In earned	Repeat
A+	95 – 100	4.00	Y	Y	Y	Y	—
A−	90 – 94	3.70	Y	Y	Y	Y	—
B+	85 – 89	3.30	Y	Y	Y	Y	—
B−	80 – 84	2.70	Y	Y	Y	Y	—
C+	75 – 79	2.30	Y	Y	Y	Y	—
C−	70 – 74	1.70	Y	Y	Y	Y	—
D+	65 – 69	1.30	Y	Y	Y	Y	**Required if major**
D−	60 – 64	0.70	Y	Y	Y	Y	**Required if major**
F	below 60	0.00	N	Y	Y	N	**Always required**
I	n/a	n/a	N	N	N	N	Resolves to a letter
R	—	—	—	N	see 16.4.4	N	—

R is not a grade. It is a display marker placed on an earlier attempt that has been dropped from CGPA by the carry-over rule (v4.0 §10.1, REQ-C06). **I is a real grade sheet and transcript entry** that carries no academic weight until it is resolved to a letter through the correction workflow.

16.3 Score entry and letter derivation

Lecturers submit numeric marks on paper. The Admin enters the number; the system derives the letter and the grade point. **Decimal marks are accepted** (CR-07).

```
deriveLetter(score):
    require 0 <= score <= 100
    rounded = roundHalfUp(score, 0)          -- 89.5 -> 90;  94.4 -> 94;  94.5 -> 95
    if rounded >= 95 : "A+"   gp 4.00
    if rounded >= 90 : "A-"   gp 3.70
    if rounded >= 85 : "B+"   gp 3.30
    if rounded >= 80 : "B-"   gp 2.70
    if rounded >= 75 : "C+"   gp 2.30
    if rounded >= 70 : "C-"   gp 1.70
    if rounded >= 65 : "D+"   gp 1.30
    if rounded >= 60 : "D-"   gp 0.70
    else             : "F"    gp 0.00

    -- the ENTERED score is stored verbatim to one decimal place
    -- the ROUNDED score is used only to select the band
    -- the letter and grade point are stored, frozen, on the record
```

Rule	Detail
Entered precision	One decimal place. `NUMERIC(4,1)`. A mark of 87.5 is stored as 87.5, not as 88.
Rounding for banding	Half-up to the nearest whole number, once, before the band lookup. 94.5 becomes 95 and therefore A+.
Worked boundary	94.4 → 94 → A− (3.70). 94.5 → 95 → A+ (4.00). 59.5 → 60 → D− (0.70). 59.4 → 59 → F.
Displayed on entry	The screen shows the derived letter and grade point live as the Admin types, so a transcription error is visible immediately (Section 20.6).
Historical records	Old paper often shows a letter with no mark. Letter-only entry is permitted for imported records; the score column is left null and the grade point comes from the letter.
Frozen at publication	Letter and grade point are written onto the academic record and never re-derived. A later scale change cannot rewrite history (DER-07).

16.4 Rules

16.4.1 Passing and credit

Rule	Specification
Minimum passing grade	**D** — that is, D− (0.70) and above. F earns no credit.
Credits earned	Credit hours of every record with grade point ≥ 0.70. A repeated course contributes its credits **once** (v4.0 §10.1, REQ-C07).
Prerequisite satisfaction	Default threshold is the passing grade, D− (0.70). A department may set a higher threshold per prerequisite via `course_prerequisite.min_grade` — the field exists and defaults to the institution passing grade, so no department override is required for Phase 1.
Graduation requirement	**132 credit hours earned** for a BSc. Held as an institution setting, displayed as a progress figure on the student record, and never used to block anything automatically in Phase 1.

16.4.2 Mandatory repeats

This is a new rule with no equivalent in v4.0, and it introduces a concept the system did not previously have.

Concept	Definition
Major course	A course whose owning department is the student's own department. Determined at the time the result is recorded and **frozen onto the academic record** as `was_major_at_record`, so a later department change does not retroactively create or clear a repeat obligation.
Mandatory repeat	An outstanding obligation to retake a specific course. Created when a result is published or imported, and cleared when a later attempt at the same course produces a grade that does not itself trigger the rule.

Grade earned	Major course	Non-major course	Notes
F	**Repeat required**	**Repeat required**	No credit earned; the course must be passed to count toward the 132.
D+ or D−	**Repeat required**	**Stands — repeat optional**	Credit *is* earned in both cases, and the grade counts in GPA. The obligation is about the standard expected in the student's own discipline, not about credit.
C− and above	**Stands**	**Stands**	Repeating is not permitted unless the plan item is explicitly flagged as a retake (v4.0 §8).

> **HOW THE OBLIGATION BEHAVES**
>
> A mandatory repeat is an **advisory obligation, surfaced everywhere** — it does not silently block a student. It appears on the student's own record, on the Admin's student record screen, and as a warning to the Admin at course-plan approval if an outstanding obligation is not included in the plan. The Admin may approve anyway; the warning and the decision are both audited. Hard-blocking a plan on it would strand students whose historical import is incomplete, which in year one is nearly all of them (CR-11).

16.4.3 Repeated courses and CGPA

Unchanged from v4.0 §10.1 and carried forward: **the most recent attempt is counted in CGPA; the earlier attempt is dropped from CGPA and marked R**; both attempts remain on the record; the credit hours count once toward the 132.

> **ASSUMPTION ASM-19 — CARRIED FROM V4.0**
>
> "Most recent" rather than "best" means a student who retakes and does worse lowers their CGPA. Fixture F-13 in Appendix B shows this in numbers. The rule is implemented exactly as v4.0 states it and is a single policy switch in the engine, so reversing it later is a configuration change plus a deliberate recomputation. Worth the Registrar glancing at F-13 once and confirming — but it is not blocking, because v4.0 already states the rule.

16.4.4 Repeats and credits attempted

Question	Position
Does a dropped attempt still count in the GPA of the semester it occurred in?	**Yes.** A semester GPA is a historical fact about that term and does not change years later because of something that happened afterwards. A printed semester record from 2024 must still match the system in 2029.
Does it count in cumulative credits attempted?	**Yes** — the student did attempt those credits.
Does it count in the CGPA denominator?	**No.** Only kept attempts enter CGPA, numerator and denominator alike.
Does it count in credits earned?	**No** — credits count once per course (REQ-C07).

16.4.5 Incomplete (I)

Aspect	Specification
Where it appears	On the class grade sheet and on the transcript (CR-02). It is a visible, meaningful entry, not a hidden placeholder.

Aspect	Specification
Effect on figures	Excluded from GPA, from credits attempted and from credits earned, until resolved.
How it is entered	Selected as a letter on the entry screen instead of a numeric mark. The score column is left null.
How it is resolved	Through the ordinary grade-correction workflow: the Admin requests the change from I to a real letter with a reason; the Super Admin approves; GPA and CGPA recompute in the same transaction.
Resolution deadline	**One semester**, per College policy (CR-13). An I awarded in one semester must be resolved by the end of the **next** semester. The clock runs from the close of the semester in which the I was awarded; with two semesters a year that is roughly four to five months. Held as `institution_setting.incomplete_resolution_semesters = 1`.
Deadline is visible to the student	The student's record shows the deadline against the grade — "I — must be resolved by end of Second Semester 2026/2027". This is the cheapest thing in the whole rule and prevents most overdue cases: it turns an administrative trap into something the student can act on.
What happens at the deadline	**Nothing automatic.** At semester close the system produces a list of every I past its deadline. A human acts on each one. The system never invents a failing grade.
Why not auto-convert	Every other grade in this system originates with a lecturer on a paper sheet. An auto-generated F would be the only grade in the College's record with no human author, and on the transcript it would be indistinguishable from one the student earned. In year one, with historical data arriving piecemeal, letting the software manufacture failing grades is the wrong trade.
How an overdue I is closed	Through the **ordinary grade-correction workflow** — see the callout below. Either the real grade has arrived, or the Admin requests conversion to F with a reason. Both routes are a change to a published, locked grade and therefore need Super Admin approval.
Consequences of conversion to F	In one transaction: the grade becomes F; the academic record updates; semester GPA, CGPA, credits attempted and credits earned all recompute; and — because **F always requires a repeat** — a mandatory repeat obligation is created against that course. Academic standing is re-derived from the new CGPA.

THE EXISTING RULES ALREADY ANSWER THIS — DO NOT BUILD A BYPASS

An I is a **published, locked grade**. Changing it to anything — a real letter or an F — is a correction to a locked grade, and REQ-G08 is unambiguous: the Admin **requests** the change with a reason, and a **Super Admin approves** it. There is no separate "convert overdue Incompletes" power, and no Admin-only path that would let one account close an I unilaterally. The overdue list is a *worklist*, not an action. Segregation of duties applies to an expired Incomplete exactly as it applies to every other grade in the system.

16.5 Formulas

```
GPA-eligible record  = an academic_record whose grade_scale entry has counts_in_gpa = true
                       (every letter except I; R-marked rows are excluded from CGPA only)

quality_points(r)    = r.grade_point x r.credit_hours          -- exact decimal, per record
                                                               -- grade_point NUMERIC(3,2)

SEMESTER GPA (student s, semester t)
    R    = GPA-eligible records of s in t                       -- repeat-dropped rows INCLUDED here
    QP   = SUM over R of quality_points(r)
    CR   = SUM over R of r.credit_hours   where counts_in_attempted
    GPA  = (CR = 0) ? null : QP / CR                            -- ONE division
    credits_attempted = CR
    credits_earned    = SUM of r.credit_hours where grade_point >= 0.70

CUMULATIVE GPA (student s)
    All  = all GPA-eligible records of s across every semester
    Kept = All minus records where is_repeat_dropped = true      -- carry-over rule applied
    QP   = SUM over Kept of quality_points(r)
    CR   = SUM over Kept of r.credit_hours where counts_in_attempted
    CGPA = (CR = 0) ? null : QP / CR
    total_credits_earned    = SUM over Kept of r.credit_hours where grade_point >= 0.70
    total_credits_attempted = SUM over All  of r.credit_hours where counts_in_attempted
    credits_to_graduation   = max(0, 132 - total_credits_earned)

REPEAT RESOLUTION (v4.0 §10.1) — computed, never stored by hand
    for each (student, course) group with more than one record:
        keep  = the record in the LATEST semester (by academic year, then sequence 1|2)
        mark  every other record is_repeat_dropped = true, display marker "R"
    ties impossible: unique (student, semester, course, attempt_no)

MANDATORY REPEAT OBLIGATIONS (CR-05)
    for each record r of student s, taking only the KEPT attempt of each course:
        if r.letter = 'F'                                    -> obligation
        if r.letter in ('D+','D-') and r.was_major_at_record -> obligation
    obligations are recomputed whenever a record for that course changes

ROUNDING (CR-03)
    compute at full precision; store to 6 decimal places
    round HALF-UP to 3 decimal places at presentation only
    never round quality_points or intermediate sums

PROVISIONAL (REQ-H05)
    is_provisional = (student.historical_import_status != 'COMPLETE')
    propagates to semester GPA, CGPA, credits attempted, credits earned
    and to credits_to_graduation alike
```

TWO DENOMINATORS, DELIBERATELY DIFFERENT

`total_credits_attempted` is computed over **all** records; the CGPA denominator is computed over **kept** records only. For a student with no repeats they are identical. For a student who repeated a three-credit course they differ by 3. This is correct — the student did attempt those credits — but it means CGPA cannot be reconstructed as "quality points ÷ credits attempted". The student record shows both, labelled, so the difference is visible rather than mysterious.

16.6 Credit-hour rules

Rule	Value	Where it is enforced
Maximum credit hours per semester	**21**	Course-plan validation V2, at submission and again at approval. Institution setting; a department may set a lower ceiling but not a higher one.

Rule	Value	Where it is enforced
Minimum credit hours per semester	**1**	Effectively no floor. V2 rejects an empty plan and nothing else.
Credit hours to graduate (BSc)	**132**	Institution setting. Displayed as progress on the student record. **Advisory only in Phase 1** — it does not gate registration or trigger any workflow.
Credit hours per course	Whole numbers > 0	Course definition; copied to the offering and frozen onto each academic record.

16.7 Academic standing

Confirmed by the College (CR-14). Standing is displayed in Phase 1, derived from CGPA, using the conventional bands.

Standing	CGPA band	Notes
Honours	3.500 and above	Inclusive at the boundary.
Good standing	2.000 to 3.499	Inclusive at 2.000.
Probation	below 2.000	
No label	CGPA is null, or provisional	See the two rules below.

Rule	Specification
Suppressed while provisional	No standing label is shown for any student whose CGPA carries the provisional flag. Telling a student they are on probation on the basis of a half-entered record would be worse than telling them nothing. **In year one this means most students see no standing at all** — that is correct behaviour, not a bug, and the interface should say "not yet available" rather than leaving a blank.
Compared on the displayed value	The band comparison uses the CGPA **rounded to three decimal places** — the same number the student sees — not the full-precision value. A CGPA of 1.99962 displays as "2.000"; comparing on full precision would show a student the figure 2.000 next to the word Probation. The label must never contradict the number beside it.
Basis	**CGPA only.** No semester-level honours or Dean's List in Phase 1 — that was not asked for and would need its own thresholds.
Effect	**Label only.** Standing gates nothing, triggers no workflow, blocks no registration and sends no notification. There is no advising queue, no credit-hour restriction and no dismissal process in Phase 1.
Where it appears	The student's own academic record (S-05), the Admin and Super Admin student record (A-10, X-07), and the semester export — always alongside the CGPA it was derived from, and always absent when that CGPA is provisional.
Recomputation	Derived from CGPA at read time. It is not stored, so it can never drift from the figure it describes.

A CONSEQUENCE THE COLLEGE SHOULD BE AWARE IT HAS CHOSEN

On this grading scale the passing floor is D− = 0.70, not 1.00. That places the conventional 2.000 probation line well above a passing average:

A student passing every single course at D− has a CGPA of 0.700. A student averaging C− has 1.700. Both are passing everything, and both sit on Probation. A student needs roughly a C+ average (2.300) to reach Good standing.

This was raised before the bands were chosen and the College selected the conventional 2.00 / 3.50 thresholds deliberately, so it is recorded as intent rather than as a defect. If the Registrar later decides probation should mean "at risk of failing" rather than "below a C+ average", the line moves to a number nearer 1.700 — and because standing is derived at read time from configuration, that change is a settings edit with no recomputation and no migration.

16.8 Recomputation triggers

Event	Recomputes	Same transaction?
A grade submission is approved, in whole or in part	Semester and cumulative summaries, and repeat obligations, for every student whose grade was published	**Yes — DER-11**
A grade correction is approved	Both summaries for that student; repeat resolution across that course; repeat obligations	**Yes**
An Incomplete is resolved to a letter, or converted to F at its deadline	As above — it is a correction. Conversion to F additionally creates a mandatory repeat obligation	**Yes**
Any change to CGPA	Academic standing, which is derived at read time and never stored	N/A — nothing to recompute
A historical record is entered or corrected	That semester's summary, the cumulative summary, repeat resolution and obligations	**Yes**
A student's import status changes	The provisional flag on all of that student's summaries	**Yes**
A student's department changes	**Nothing.** `was_major_at_record` is frozen per record, so existing repeat obligations do not move.	—
The grading policy version changes	**Nothing automatically.** Existing records keep their frozen grade points and policy version.	Explicit, separately approved, audited bulk operation only
A course's credit hours are edited	**Nothing.** Records hold frozen credit hours (DER-07).	—

16.9 Worked examples

These are the specification of correct behaviour expressed as numbers, computed against the confirmed nine-point scale with three-decimal rounding. The full fixture set is Appendix B.

Example 1 — Clean semester

Course	Credits	Score	Letter	Grade point	Quality points	Note
CSC 101	3	96	A+	4.00	12.00	
THE 110	3	88	B+	3.30	9.90	
MAT 105	4	73	C−	1.70	6.80	
ENG 101	2	82	B−	2.70	5.40	
Totals					**34.10**	attempted **12**; earned **12**

Semester GPA = 34.10 ÷ 12 = 2.8416666... → "2.842" (half-up, 3 dp).

Example 2 — A failure in the semester

Course	Credits	Score	Letter	Grade point	Quality points	Note
CSC 201	3	84	B−	2.70	8.10	Earns credit
THE 210	3	55	F	0.00	0.00	Attempted, not earned. **Repeat required**
MAT 201	3	91	A−	3.70	11.10	Earns credit

Course	Credits	Score	Letter	Grade point	Quality points	Note
Totals					**19.20**	attempted **9**; earned **6**

Semester GPA = 19.20 ÷ 9 = 2.1333333... → "2.133". The F stays in the denominator; that is what makes a failure affect the GPA and not merely the credit count.

Example 3 — Repeat where the later attempt is better

Semester	Course	Cr	Score	Letter	QP	In CGPA?	Marker
2025/26 S1	CSC 101	3	52	F	0.00	**No — dropped**	**R**
2025/26 S1	THE 110	3	87	B+	9.90	**Yes**	
2025/26 S2	CSC 101	3	83	B−	8.10	**Yes — most recent**	attempt 2
2025/26 S2	MAT 105	3	92	A−	11.10	**Yes**	

Figure	Value	Derivation
Semester 1 GPA	"1.650"	(0.00 + 9.90) ÷ 6 — **unchanged** by the later repeat (16.4.4)
Semester 2 GPA	"3.200"	(8.10 + 11.10) ÷ 6
CGPA	**"3.233"**	(9.90 + 8.10 + 11.10) ÷ 9 = 29.10 ÷ 9 = 3.2333333...
Total credits earned	9	CSC 101 counted once, plus THE 110 and MAT 105
Total credits attempted	12	All four records
Repeat obligations	None outstanding	The F obligation is cleared by the passing retake

Example 4 — Repeat where the later attempt is worse

Semester	Course	Cr	Score	Letter	Effect
2025/26 S1	BIB 101	3	97	A+ (4.00)	Dropped from CGPA, marked **R**
2026/27 S1	BIB 101	3	71	C− (1.70)	Counted — most recent

Policy	CGPA	Comment
Most recent counts — v4.0 §10.1, implemented	**"1.700"**	The retake has *lowered* the CGPA from 4.000 to 1.700.
Best attempt counts — the common alternative	"4.000"	Shown only so the Registrar can see the difference once and confirm (ASM-19).

Example 5 — Mandatory repeat from a D in a major course

Student is in the Department of Theology. THE 210 is a Theology course (major); MAT 105 is not.

Course	Cr	Score	Letter	Major?	Credit earned	Obligation
THE 210	3	62	D−	**Yes**	**Yes — 3**	**Repeat required**
MAT 105	3	66	D+	**No**	**Yes — 3**	**None — stands**
ENG 101	3	78	C+	**No**	**Yes — 3**	**None**

QP = (0.70×3) + (1.30×3) + (2.30×3) = 2.10 + 3.90 + 6.90 = **12.90**; credits 9. **Semester GPA = 12.90 ÷ 9 = 1.4333333... → "1.433".** Credits earned 9 — the D grades *do* earn credit. One outstanding repeat obligation is recorded against THE 210 and shown on the student's record and to the Admin at the next plan approval.

Example 6 — Incomplete in the semester

Course	Cr	Grade	In GPA	In attempted	In earned	QP
CSC 301	3	96 → A+	Y	Y	Y	12.00
THE 301	3	I	N	N	N	—
MAT 301	3	78 → C+	Y	Y	Y	6.90

Semester GPA = 18.90 ÷ 6 = 3.15 → "3.150". Credits attempted 6; credits earned 6. The Incomplete appears on the grade sheet and will appear on the transcript, carrying no weight until resolved.

Example 7 — Rounding direction

Course	Cr	Letter	Grade point	Quality points	Note
CSC 401	3	A+	4.00	12.00	
THE 401	3	B+	3.30	9.90	
ENG 401	2	C−	1.70	3.40	
Totals				**25.30**	credits **8**

25.30 ÷ 8 = **3.1625** exactly. Half-up to three places gives **"3.163"**; truncation would give "3.162". This single fixture is what protects the rounding decision from being reversed by accident during implementation.

Example 8 — Graduation progress

Situation	Behaviour
Student has 118 credits earned, import status **Complete**	Record shows "118 of 132 credit hours earned — 14 remaining". No badge.
Student has 118 credits earned, import status **In progress**	Same figures, every one badged **Provisional**, with the line "Based on records entered so far." **The remaining-credits figure is provisional too** — it is the one most likely to be misread as a graduation clearance, and it must never be presented as one.
Student has 132 or more credits earned	"132 of 132 credit hours earned — requirement met". **No workflow fires.** Graduation clearance is not a Phase 1 feature; this is a displayed figure only.

16.10 What the engine must never do

Prohibition	Reason
Read the database	It is a pure function. Data is fetched by the caller and passed in, so the same inputs always give the same outputs and the tests are trustworthy.
Use floating-point arithmetic	DER-08. Grade points of 3.70 and 2.30 are exactly the values that expose binary floating point. Includes intermediate sums and any value crossing a JSON boundary.
Round intermediate values	Rounding twice gives a different answer than rounding once, and the difference lands exactly on threshold cases.
Round the score more than once	The entered mark is stored verbatim; a single half-up rounding selects the band. Rounding again anywhere would move a boundary mark into the wrong letter.
Include unpublished grades	REQ-C02.
Re-derive a grade point from the current scale for an old record	The record's frozen grade point is authoritative. Re-deriving would silently rewrite history whenever the policy changes.
Recompute `was_major_at_record`	A student changing department must not gain or lose repeat obligations retroactively.

Prohibition	Reason
Return a figure without its provisional flag and policy version	REQ-H05. In year one a bare number is actively misleading.
Be duplicated in client code	P2, Section 8.4. If the browser needs a GPA, it asks the server.

17. Historical Records Strategy

The source document calls this "the largest piece of work in the project and the main risk to the schedule" (§6). This section treats it as an operational workstream with a software component, rather than as a screen.

17.1 Why it dominates the project

Dependency	What breaks while history is incomplete
CGPA	Computed from whatever is present, so it is confidently wrong rather than obviously wrong. This is the most dangerous failure mode in the system.
Prerequisite validation	Every prerequisite fails for a student whose history is missing, which is why REQ-P11's override exists at all.
Retake detection	A repeat cannot be recognised as a repeat, so the carry-over rule never fires and the course is treated as new.
Credits earned / progress to graduation	Understated, which affects who is told they may graduate.
Transcripts (Phase 2)	The source makes transcript generation conditional on this work being finished.

> **THE GOVERNING PRINCIPLE**
>
> The system's job is not to hide incompleteness or to guess around it. It is to **know** that data is incomplete, to say so on every figure derived from it, and to make finishing the work visible and measurable. An incomplete record that announces itself is safe; a complete-looking record that is actually partial is the single worst outcome this project can produce.

17.2 The unified-table decision, restated

Historical records are written into the same `academic_record` table as system-generated results, distinguished by `origin = 'IMPORTED'` (DER-06, Section 9.1). The alternative — a separate historical table — was rejected because it would require every consumer (GPA, CGPA, prerequisites, retakes, transcripts) to union two sources and would guarantee that they eventually diverge.

Field	System-generated	Imported	Enforcement
`origin`	SYSTEM	IMPORTED	Check constraint; set by the writing service, never by user input
`grade_record_id`	Required	Must be NULL	Check constraint tying it to origin
`credit_hours`	Frozen from the offering	Entered by the Admin from the paper record (REQ-H02)	> 0
`score`	Usually present	Often NULL — old sheets may show only a letter	Nullable by design
`source_note`	NULL	Which document, and any legibility or conflict caveat	Free text, surfaced wherever the record is reviewed
`entered_by`	The approving Super Admin's action	The entering Admin	Always populated

17.3 The historical entry screen

REQ-H02 requires a dedicated screen, separate from normal grade entry. Separation matters for more than tidiness: the two screens have different keys, different validation and different risks. Normal grade entry works from a known class list; historical entry works from a paper file with no list to check against.

Element	Specification
Step 1 — select student	Search by Student ID or name. The screen shows the student's current import status, how many semesters have been entered so far, and the earliest semester recorded. The Admin always knows where they are in the job.
Step 2 — select semester	Choose an existing past semester, or create one. Historical semesters may need to be created retrospectively; they are created directly in the Closed state and are excluded from the forward state machine.
Step 3 — enter courses	A repeating row: course code, credit hours, grade, optional score, optional note. Course code autocompletes against the catalogue; a code with no catalogue match is **allowed** but flagged, because a course taught in 2019 may not exist in the current catalogue.
Row behaviour	Credit hours default from the catalogue where the course matches, and remain editable — the paper record is authoritative, not the current catalogue (REQ-H02).
Save	One semester's worth of rows saves as a single transaction with an idempotency key. Duplicate detection runs before the write.
Step 4 — set status	The Admin marks the student In progress (automatic on first save) and later Complete (an explicit, audited act — Section 17.6).
Ergonomics	Desktop-first, keyboard-driven, no mouse required to add a row. The Admin is transcribing, often for hours; every extra keystroke multiplies across thousands of records (REQ-D04).
Missing catalogue course	Offer to create a minimal catalogue entry inline, or record the course code as entered with a flag. Blocking the Admin until someone creates a 2019 course in the catalogue would stall the import.

17.4 Independence from the semester state machine

Historical entry is deliberately exempt from the semester-state gates in Section 13.4. Past semesters are Closed by definition; gating import on state would make the work impossible. In exchange, historical entry has its own controls: it may only write to semesters that have **already ended**, it may never create a `grade_record` or a registration, and every write is audited with `origin = IMPORTED`.

17.5 Validation and conflict handling

#	Check	Behaviour	Detail
1	Duplicate record	**Block**	Unique on (student, semester, course, attempt). The Admin is shown the existing record and asked whether this is a genuine repeat (increment attempt) or a duplicate entry (discard).
2	Grade not in the scale	**Block**	Only letters defined in the active grade scale are accepted. If the paper shows a legacy grade the current scale does not contain, that is a policy question for the Registrar, not a data-entry decision.
3	Credit hours ≤ 0 or implausible	Warn	Blocked below 1. Warned above a configurable plausibility ceiling — a mistyped "30" instead of "3" is the most common transcription slip.
4	Unknown course code	Warn	Permitted with a flag. Retired courses legitimately appear on old records.
5	Semester precedes the student's enrolment year	Warn	Usually an error, occasionally a legitimate transfer or readmission. The Admin confirms with a note.
6	Score inconsistent with letter	Warn	If both are entered and the score falls outside the letter's band, the Admin must confirm which the paper actually says, and the confirmation is recorded.
7	Contradictory records for the same course and semester	**Block**	Rejected as a duplicate (check 1). REQ-H07 is explicit that the system holds the record and does not resolve the conflict — but "hold" cannot mean storing two contradictory truths in the table that drives CGPA.
8	Unreadable or missing paper record	Record, do not invent	The Admin records what is legible and uses `source_note` for the caveat. A course with no readable grade is left unentered and noted — never guessed, never entered as F.

HOW CONFLICTS ARE ACTUALLY HELD

REQ-H07 says the system holds a conflicting record and does not resolve it. This plan implements that as: the `academic_record` table holds **one** row per student-semester-course — the version the College has decided to treat as authoritative — while the conflict itself is captured in `source_note` and in the audit trail, and the student's import status stays **In progress** until an authorised person adjudicates. Storing two contradictory rows would put the contradiction directly into the CGPA. REQUIRES APPROVAL DEC-26 — the College must say who adjudicates and how the outcome is recorded.

17.6 Import status and the provisional marker

Status	Meaning	Set by	Effect on derived figures
Not started	No historical record has been entered for this student.	Default at enrolment	All figures provisional. Prerequisite checks will fail; the Admin sees the status as the reason.
In progress	At least one record entered; the Admin has not declared the work finished.	Automatically on first save	All figures provisional.
Complete	An Admin asserts that this student's full paper history is now in the system.	Explicit Admin action, audited	Provisional marker removed. Figures are presented as final.

Control	Detail
Marking Complete is consequential	It is the moment a student's CGPA becomes an official number. The confirmation screen shows a summary — semesters entered, courses, total credits, computed CGPA — and requires explicit confirmation. It is not a checkbox on a list.
Reopening Complete	Permitted with a mandatory reason; the provisional marker returns immediately. Recommended to require Super Admin approval, matching the treatment of a locked grade. RECOMMENDATION
Newly enrolled students	A student with no prior study has no history to import. They should be marked **Complete** at enrolment, with a note, so first-year students are not permanently provisional. This case is easy to miss and would otherwise leave every new cohort flagged. RECOMMENDATION
Where the marker appears	Every screen showing a GPA, CGPA, credits attempted or credits earned for that student — student portal, admin student record, class list, export. One rule, applied everywhere, driven by the flag returned with the figure.

17.7 Progress reporting

REQ-H06 requires a report of how many students are fully entered. Because this is the instrument the College will use to manage the largest task in the project, it should carry a little more than a single count:

Report element	Purpose
Counts by status, institution-wide	The headline: how many Not started, In progress, Complete.
Breakdown by College and Department	Lets the work be assigned and tracked by unit.
Breakdown by enrolment year / cohort	Supports the source's own recommendation to work by cohort, starting with those closest to graduation (§6.2).
Students with records but flagged issues	Unknown course codes, score/letter mismatches, notes recorded — the queue of things a human must look at.
Records entered per week	The only honest basis for estimating when the import will finish. Without it, "how long will this take?" has no answer.
Students Complete but with suspiciously few records	Catches a status marked Complete by mistake — for example a continuing student with one semester recorded.

17.8 Operational plan

Confirmed by the College (CR-11): the Admin office enters historical records, gradually, after the system is live. That is a defensible choice — it lets go-live happen without waiting months for transcription — but it has one consequence the whole plan must accommodate, and it is worth stating without softening.

> **THE CONSEQUENCE OF ENTERING GRADUALLY, POST-LIVE**
>
> The E-Portal will go live with **essentially every continuing student's CGPA computed from an incomplete record**, and it will stay that way for months. **Provisional is not an edge case in year one — it is the normal state of the system.**
>
> Three things follow. First, the provisional marker must be unmissable and self-explanatory on every screen, not a subtle badge. Second, prerequisite checking will fail for most students for most of the first year, which makes the prerequisite override (REQ-P11) a routine tool rather than an exceptional one — so its expiry date (DEC-12) should be set generously and reviewed, not set to one semester by reflex. Third, and least technical: **the College should tell students what "provisional" means before they see it**, because a CGPA that looks wrong and is labelled in language nobody explained will generate more anxiety than the system saves.

Question	Status	Position
Who enters the data?	**Confirmed**	The Admin office, as part of ordinary duties (CR-11). No separate data-entry team.
When does entry start?	**Confirmed**	After go-live, gradually. The screen must therefore be built to sustain months of intermittent daily use rather than a short concentrated push — which raises, not lowers, the bar on its ergonomics.
How far back, and in what order?	**At the office's discretion**	No boundary was set. RECOMMENDATION — work by cohort, closest to graduation first, as v4.0 §6.2 itself suggests. Those students need trustworthy figures soonest, and it makes the progress report meaningful rather than a flat percentage that barely moves.
How long will it take?	**Measurable, not yet known**	Deliberately not estimated here. The progress report's **records-entered-per-week** figure is the only honest basis for an estimate, and it exists precisely so the College can answer this question with data after the first month rather than with a guess now.
Who adjudicates a conflicting or unreadable record?	**Open — DEC-26**	The system holds and flags; a named person must decide. Needed before the office meets its first conflict, which will be within days of starting — not before the build.
Quality assurance	**Recommended**	A second person re-checks a sample against the paper before a student's status is marked Complete — at least the first twenty students, then periodically. Errors found this way are far cheaper than errors found by a graduating student. This matters more under the gradual model, because the work is spread across many weeks and consistency drifts.
Do new students wait?	**No**	A student with no prior study is marked import-Complete at enrolment (ASM-17). First-year cohorts must be visibly *not* provisional from day one — otherwise the badge appears on every student in the College and stops meaning anything.

17.9 Effect of incompleteness, summarised

Consumer	Behaviour when history is incomplete	Why this is the right behaviour
Semester GPA	Computed for entered semesters; marked provisional	The figure is correct for the data present; the marker is what makes it honest.
CGPA	Computed from what exists; marked provisional	Suppressing it entirely would push the office back to manual calculation.
Credits earned	Understated; marked provisional	Understating is the safe direction — nobody is told they may graduate when they may not.
Prerequisite check	Fails, with the student's import status shown as context	Failing closed is safe; the override path (REQ-P11) exists precisely for this window.

Consumer	Behaviour when history is incomplete	Why this is the right behaviour
Retake detection	May miss a repeat	Unavoidable. The Admin is warned at plan approval when the student's import is not Complete.
Academic standing	Suppressed entirely	Labelling a student "on probation" from partial data is worse than showing nothing (Section 16.5).
Transcript (Phase 2)	Blocked, or issued with an explicit warning on the document	Already the College's stated position in §11.

18. Security Architecture

The threat model is modest and specific. This is not a public-facing system with an adversarial internet population; it is an institutional record with a few hundred to a few thousand legitimate users, where the realistic risks are credential sharing, a student attempting to see or change a grade, an insider acting alone, and an operational mistake that exposes data. The controls below are sized to that, and no larger.

18.1 Controls by priority

Level	Control	Detail
REQUIRED Phase 1 cannot ship without these	Managed password hashing	Delegated to the auth provider; per-user salt; modern algorithm. Application code never sees or stores a password (DER-03).
	Forced first-login password change	Session restricted to the change-password screen until completed; unbypassable by direct URL (REQ-A03, DER-04).
	Server-side authorisation on every mutation	The permission kernel of Section 11, called before any data is touched (REQ-A05).
	Row-Level Security on every table	The independent second gate (Section 10.5).
	Student data isolation	A student's queries are scoped to their own id in the service layer and again by policy. No endpoint accepts a student id from the client for a student-role actor.
	Segregation of duties	Approver ≠ submitter and decider ≠ requester, in service code, in policy and as check constraints (REQ-R05, REQ-R06).
	Input validation on the server	Every input re-validated server-side regardless of client validation. Parameterised queries only; no string-built SQL (TEC-06).
	Transport security	HTTPS only, HSTS enabled, secure/HttpOnly/SameSite session cookies.
	Secrets management	Environment variables in the hosting platform; never in the repository; separate values per environment; rotated if exposed. No service-role key is ever sent to a browser.
	Append-only audit	Privilege-level enforcement (DER-20).
RECOMMENDED Small effort, disproportionate value	Rate limiting on authentication	Per-identifier and per-IP throttling with progressive delay; temporary lockout after repeated failures, with the lockout visible to Admin so a locked-out student can be helped.
	Password policy	Minimum length with a rejection list of obvious values; no forced periodic rotation, which reliably produces weaker passwords written on desks.
	Session lifetime and idle timeout	Shorter for Admin and Super Admin than for students, since staff work on shared office machines.
	Security headers	Content-Security-Policy, X-Content-Type-Options, Referrer-Policy, frame-ancestors deny.
	Safe error handling	Users see a clear message and a correlation id; stack traces, SQL and record contents never reach the browser and never enter logs.
	Log hygiene	No credentials, no session tokens, no full grade payloads in application logs. Identifiers only.
	Dependency management	Automated vulnerability alerts; a dependency update reviewed and merged before each stage gate rather than in a panic later.

Level	Control	Detail
FUTURE Deliberately not Phase 1	Multi-factor authentication for Super Admin	Worth doing once the College has a reliable second channel. Requires SMS or email, which Phase 1 does not have (OOS-06).
	IP allow-listing for administrative access	Attractive, but campus connectivity is not stable enough to rely on it without locking staff out.
	Self-service password reset	Requires a verified contact channel per student. Admin reset is the Phase 1 answer (REQ-A02).
	Field-level encryption of grades	Not proportionate. Access control and audit are the appropriate controls; encryption at rest is already provided by the platform.
	Formal penetration test	Recommended before the system holds several years of records, not before first go-live.

18.2 Authentication design

Aspect	Design
Student login	Student ID + password (REQ-A01). The server resolves the Student ID to the internal identity, then authenticates. A failed resolution and a failed password produce the **same** response and the same timing, so the form cannot be used to enumerate valid Student IDs.
Staff login	Username (or institutional email if the College issues them) + password. Same resolution path.
Account creation	Admin creates a student account and issues a temporary password, delivered in person or on paper — there is no messaging channel in Phase 1. Super Admin creates Admin accounts the same way (REQ-A02, REQ-A06).
First login	`must_change_password` is true; every route except change-password redirects there; the flag clears only on a successful change (REQ-A03).
Password reset	Admin-initiated only. Generates a new temporary password, sets the flag, and is audited with actor and target — never with the password itself.
Session	HttpOnly, Secure, SameSite=Lax cookie. Role is read from the session on the server on every request, never from client-supplied data (DER-05).
Logout	Invalidates the session server-side, not merely by clearing a cookie.

18.3 The Student-ID identity mapping

The chosen auth provider models identifiers in email form; students have no email. The resolution is a deterministic, non-routable internal identifier — for example `<student-id>@students.lcc.invalid` — which exists only inside the auth store.

Requirement on the mapping	Reason
Deterministic and reversible	Login must resolve reliably in one lookup, every time.
Never displayed to any user	Students would reasonably mistake it for an email address they could use.
Never usable as a contact or reset channel	Uses a non-deliverable domain so no password-reset mail can ever be sent to it. Provider-side email flows are disabled for these identities.
Stored once, in one place	`app_user.login_identifier` holds the Student ID as typed; the mapping to the internal identifier is derived. A Student-ID change is a single audited update, not a data migration.
Case- and whitespace-normalised	Students will type their ID inconsistently. Normalisation happens before lookup, on the server.

REQUIRES APPROVAL — DEC-29

The College should be told plainly that a synthetic internal identifier will exist, that it is not an email address, that no mail can be sent to it, and that students will only ever be asked for their Student ID. If the College objects, the fallback is a credentials-based authentication library with a bespoke user table — more control, more security code to own, and a change that costs roughly one stage if made before Stage 2 and considerably more afterwards.

18.4 Academic data protection

Concern	Control
A student seeing another student's record	Service-layer scoping plus RLS. Tested explicitly: authenticate as student A, request student B's record by direct id, assert refusal.
A student seeing an unpublished grade	Excluded by the query, by RLS and by the grade status model. Three independent barriers.
An Admin publishing a grade unilaterally	The action does not exist for the Admin role, and the check constraint would refuse it even if the service were defective.
A Super Admin editing a grade	Same: no permission row, no RLS grant, no service path.
Bulk extraction of academic data	The semester-end export is a permissioned, audited action. Ad-hoc bulk endpoints do not exist. Pagination limits how much any single request can return (DER-23).
Data in the test environment	Production data is never copied to test (Section 8.5). Test data is generated.
Personal data minimisation	Only the attributes needed for the academic record are collected (GAP-24 / DEC-21). Nothing is gathered "in case it is useful".

18.5 Administrative account security

Control	Detail
Least privilege by design	The role model already prevents any single account from performing an end-to-end grade change (Section 11).
Named accounts only	No shared "admin" login. Every action must be attributable to a person, or the audit log is worthless. This is a policy the College must enforce; the software supports it by making account creation cheap.
One person, one role	The College should not give the same individual both an Admin and a Super Admin account. The software cannot prevent it — the constraints only guarantee that the same *account* cannot both submit and approve. Recorded as a risk (Section 28).
Disable, never delete	A departing staff member's account is disabled; their audit history remains attributable.
Minimum one Super Admin	Enforced: the last active Super Admin cannot be disabled.
Bootstrap account	Created once by the documented procedure, audited, and its credentials changed immediately on first use (DER-26).
Periodic review	Recommend the Super Admin reviews the active account list once per semester. Small effort; catches the account nobody remembered to disable.

19. Audit Logging

REQ-E03 requires audit logging "from day one". It is built in Stage 1, before the first grade exists, because an audit facility retrofitted after the workflows are written is always partial — it covers the paths someone remembered.

19.1 What an audit entry contains

Field	Required by	Content
`occurred_at`	REQ-E01 "when"	UTC timestamp, set by the database, not by the client.
`actor_user_id`	REQ-E01 "who"	The authenticated user. Never nullable for a user-initiated action; system-initiated actions use a reserved system actor.
`actor_role_snapshot`	Derived	The role held at the time. A later role change must not rewrite the meaning of a past action.
`action`	REQ-E01 "what"	A value from the typed vocabulary in 19.3. Not free text.
`entity_type`, `entity_id`	REQ-E01 "which record"	The affected record.
`student_id`	Derived	Denormalised so that "everything that happened to this student" is one indexed query rather than a join across six tables.
`old_value`, `new_value`	REQ-E01, REQ-G09	JSONB, containing only the fields that changed. Not a whole-row dump.
`reason`	REQ-G09	Mandatory for corrections, rejections, overrides and backward semester transitions; optional elsewhere.
`request_id`	Derived	Correlates every entry written by one operation, so a batch publication reads as one event rather than sixty unrelated ones.
`ip_address`, `user_agent`	Recommended	Useful in a dispute about who did something from where. Not relied upon.

19.2 How completeness is guaranteed

Mechanism	Detail
One writer	A single audit service. No other code writes to the audit table.
Same transaction	The audit entry commits with the change or not at all (DER-21). An audit written afterwards is lost precisely when a failure makes it most valuable.
Typed vocabulary	Actions are an enumerated set. A new action cannot be logged without being registered, which forces the question "should this be audited?" at the moment the action is written.
Service-layer, not trigger-only	Database triggers can see that a row changed but not *why*, or on whose behalf. Reason and intent exist only in the application (TEC-12). A trigger-based backstop on the most critical tables is recommended as a cross-check, not as the primary mechanism.
Tested as part of the workflow	Workflow tests assert the audit entries as part of asserting the outcome. "Publishing a class produces N published grades **and** N+1 audit entries" is one test, not two.
Append-only by privilege	No UPDATE or DELETE grant exists for any application role (DER-20). Verified at every stage gate.

19.3 Audited actions

Action	Reason required	Old / new captured	Notes
Grades — the most heavily audited area			

Action	Reason required	Old / new captured	Notes
GRADE_ENTERED	No	New value	Draft entry.
GRADE_DRAFT_EDITED	No	Old and new	Every draft change, not only the final one.
GRADE_DRAFT_CLEARED	No	Old value	
GRADE_SUBMISSION_CREATED	No	Grade count, offering	One entry for the batch.
GRADE_SUBMISSION_APPROVED	No	Submission state	Plus one GRADE_PUBLISHED entry per grade, sharing a request_id.
GRADE_SUBMISSION_REJECTED	Yes	Submission state	Reason shown to the Admin.
GRADE_PUBLISHED	No	New value	Per grade.
GRADE_CORRECTION_REQUESTED	Yes	Old and proposed new	REQ-G09.
GRADE_CORRECTION_APPROVED	No	Old and new	Both actors resolvable: requester on the request entry, approver on this one, linked by entity id.
GRADE_CORRECTION_REJECTED	Yes	Proposed value	Nothing changed; the proposal is retained.
Academic records and import			
HISTORICAL_RECORD_ENTERED	No	New value	Includes origin and any source note.
HISTORICAL_RECORD_CORRECTED	Yes	Old and new	
IMPORT_STATUS_CHANGED	Yes on reopen	Old and new status	Marking Complete removes the provisional marker — a consequential act.
ACADEMIC_RECORD_VOIDED	Yes	Old value	The nearest thing to a deletion the system permits.
Semester and planning			
SEMESTER_STATE_CHANGED	Yes if backward	From-state and to-state	REQ-W04.
COURSE_PLAN_SUBMITTED	No	Items and total credits	
COURSE_PLAN_APPROVED	No	Items	Plus REGISTRATION_CREATED per item, sharing a request_id.
COURSE_PLAN_REJECTED	Yes	Reason	REQ-P10.
PREREQUISITE_OVERRIDDEN	Yes	Course and unmet prerequisite	REQ-P11.
REGISTRATION_CREATED / DROPPED	Yes on drop	Offering, source	
Administration and configuration			
USER_CREATED / DISABLED / ENABLED	No	Role and status	Never the password, in any form.
PASSWORD_RESET_BY_ADMIN	No	Target user only	
PASSWORD_CHANGED_BY_SELF	No	Nothing	The fact only.
LOGIN_SUCCEEDED / LOGIN_FAILED	No	Identifier attempted	Never the password. Failures are counted for rate limiting.
STUDENT_CREATED / STUDENT_UPDATED	No	Changed fields	
COURSE_CREDIT_HOURS_CHANGED	No	Old and new	Logged as its own action because of its academic significance.
PREREQUISITE_ADDED / REMOVED	No	Course pair	Changes who may register for what.
GRADE_SCALE_VERSION_CREATED	Yes	Full scale	The most consequential configuration in the system.
INSTITUTION_SETTING_CHANGED	No	Old and new	Credit limits, rounding, override window.

Action	Reason required	Old / new captured	Notes
CLASS_SHEET_PRINTED	No	Offering	A document containing grades left the system.
ACADEMIC_EXPORT_RUN	No	Scope	A full copy of academic data left the system.
AUDIT_LOG_VIEWED	No	Filters applied	Oversight of the oversight.

19.4 The audit log view

Aspect	Specification
Access	Super Admin only (REQ-R08). No export in Phase 1 beyond what the screen shows — an exported audit log is an unaudited copy.
Filters	By student, by actor, by action, by entity, by date range. These five cover every realistic question: "what happened to this student", "what did this Admin do", "who changed grades last week".
Presentation	Newest first, paginated, with old and new values rendered as a readable diff rather than raw JSON.
Correlation	Entries sharing a request_id are grouped, so a batch publication reads as one event that expanded into sixty grade entries.
Retention	Indefinite. The audit log is part of the academic record. Volume is trivially manageable at this scale.
Integrity check	Because the primary key is a monotonic identity, a gap in the sequence is detectable. Recommend a simple periodic check as a tamper indicator. RECOMMENDATION

20. UX / Screen Architecture

This section defines **what screens exist and what each must do**. It is not a visual design. Layout, colour and component styling are produced during implementation against these specifications.

20.1 Two design targets

Surface	Primary device	Design priority	Consequences
Student	Phone	Mobile-first. Few actions, large targets, minimal payload.	Grade sheets and timetables reflow to stacked cards on narrow screens rather than scrolling horizontally (REQ-D02). Target ≤ 150 KB of JavaScript (DER-25).
Admin	Desktop	Density and keyboard speed. Many rows visible at once; no mouse required for repetitive entry.	Dense tables, sticky headers, tab/enter navigation. Degrades to a usable — not optimised — mobile view (REQ-D04). Target ≤ 300 KB.
Super Admin	Desktop	Review clarity. Few screens, each showing enough context to make a decision without navigating away.	Approval screens present the whole batch, its provenance and its outliers on one page.

20.2 Global state conventions

Rather than repeating four state descriptions on every screen, the system uses one consistent vocabulary. Each screen below states only where it deviates.

State	Convention
Loading	Server-rendered pages arrive complete; there is no loading state for initial content. Interactive submissions show an inline pending indicator on the control and disable it to prevent double submission.
Empty	Always explains *why* it is empty and what to do next — never a bare "No records". Example: "No grades have been published for this semester yet. Grades appear here once they are approved."
Error	A plain-language message, the specific field or record at fault, and a correlation id. Never a stack trace, never a raw database message. Validation failures list **every** failure, not the first.
Success	Confirms what changed, in the user's terms: "38 grades submitted for approval for CSC 201 – Section A." Destructive or consequential actions confirm before acting, not after.
Permission denied	Rendered as "not available to your role" rather than "forbidden". Controls the user cannot use are not shown — but hiding them is presentation only, never the enforcement (REQ-A05).
Provisional data	Any GPA, CGPA or credit figure derived from an incomplete import carries a visible badge and one line of explanation, everywhere it appears (REQ-H05).

20.3 Student screens

ID	Screen	Purpose & content	Actions & validation	Empty / error specifics	Depends on
S-01	Login	Student ID and password fields; institution name.	Sign in. Both fields required. Generic failure message; identical timing for unknown ID and wrong password.	Error: "Student ID or password is incorrect." Lockout message names the Admin office as the route to help.	Stage 2

ID	Screen	Purpose & content	Actions & validation	Empty / error specifics	Depends on
S-02	Forced password change	New password, confirmation, policy hint.	Change password. Meets policy; matches confirmation; differs from the temporary one.	No other route is reachable until this succeeds.	S-01
S-03	Portal home	Name, Student ID, department, enrolment year, current semester and its state; links to the four sections below; a single status line if a course plan needs attention. **Not a dashboard** — no charts, no analytics (OOS-09).	Navigate only.	Empty: if no semester is open, says so plainly.	Stage 11
S-04	My results — semester	Published grades for a chosen semester: course code, title, credit hours, grade, grade point; semester GPA, credits attempted, credits earned; R markers on repeated attempts; **any Incomplete shown with its resolution deadline**.	Change semester. Print.	Empty: "No grades published for this semester yet." Provisional badge where applicable.	Stage 7, 10
S-05	My academic record	All semesters with published results; CGPA to three decimal places; total credits attempted and earned; **progress toward the 132 credit hours required**; **academic standing** where the CGPA is not provisional; any outstanding mandatory repeats named; any unresolved Incomplete with its deadline; both credit figures explained where they differ because of repeats (Section 16.5).	Print.	Empty: explains that records appear as they are published or entered by the office. Provisional badge governs the whole page.	Stage 7
S-06	My schedule	Weekly timetable for the current semester from approved registrations: day, time, course, section, room, instructor name.	Change semester. Print.	Empty: "You have no approved registrations for this semester." Conflict indicator if a schedule changed after approval (Section 14.6 case 4).	Stage 8, 9
S-07	Course planning — build	Available offerings for the open semester with times and remaining capacity; the working plan with running credit total; live validation feedback.	Add, remove, mark as retake, save draft, submit. Server runs V1–V6 on submit.	Empty: "Course planning is not currently open." Errors list every failed rule with the specific course named.	Stage 9
S-08	Course planning — status	Current plan and its state; if rejected, the reason and who rejected it; if approved, the resulting registrations.	Revise (rejected plans only).	Empty: no plan yet, with a link to build one.	S-07
S-09	My profile	Read-only personal and academic details; import status shown honestly.	Change password.	Explains that corrections are made by the Admin office.	Stage 5

20.4 Admin screens

ID	Screen	Purpose & content	Actions & validation	Empty / error specifics	Depends on
A-01	Admin home	Work queues: plans awaiting approval, classes with grades not yet submitted, submissions rejected and needing rework, import progress summary. Queues, not analytics.	Navigate.	Empty: "Nothing is waiting for you."	Stage 11
A-02	Colleges	List with code, name, department count, status.	Create, edit, deactivate. Unique code; deactivation blocked while dependents exist.	Error names the blocking dependents.	Stage 3
A-03	Departments	List by college; code, name, student count, credit-limit overrides.	Create, edit, deactivate.	As above.	A-02
A-04	Courses	Searchable catalogue: code, title, department, credit hours, prerequisites, status.	Create, edit, deactivate. Unique normalised code; credit hours > 0. Editing credit hours warns that existing records are unaffected.	Empty state offers to create the first course.	A-03
A-05	Course prerequisites	Prerequisite list for a course, with the minimum grade required for each.	Add, remove. Cycle check on add.	Cycle error names the courses forming the loop.	A-04
A-06	Academic years	List with label, dates, semesters, current flag.	Create, edit. Label pattern; non-overlapping dates.		Stage 4
A-07	Semesters	List with year, sequence, name, dates, **state**, and the legal next transitions.	Create, edit, advance state. Transition guards of Section 13.2.	Blocked transitions explain exactly what is outstanding, listing the records.	A-06
A-08	Course offerings	Offerings for a semester: course, section, instructor, capacity, meeting times, registered count, status.	Create, edit, publish, cancel, manage meeting times. Unique section; end after start; cancel blocked with registrations.	Empty: offers to create the first offering for the semester.	A-04, A-07
A-09	Students	Searchable list: Student ID, name, department, status, import status, CGPA (with provisional badge).	Enrol, edit, change status, reset password. Unique Student ID; format per DEC-02.	Search-first for large lists; paginated (DER-23).	Stage 5
A-10	Student record	Full academic history across semesters; per-semester GPA; CGPA and academic standing; credits earned against 132; import status; outstanding repeat obligations; unresolved Incompletes with deadlines; registrations; plans; recent audit entries for this student.	Edit profile, open historical entry, open plan review.	Empty history explains the import status.	A-09
A-11	Course plan review	Queue and detail: student, semester, items with credit hours and times, validation results, prerequisite failures with the student's import status shown.	Approve, reject with reason, override a prerequisite with reason. Full revalidation at approval.	Empty: "No plans awaiting approval." Approval failure names the rule that failed.	Stage 9

ID	Screen	Purpose & content	Actions & validation	Empty / error specifics	Depends on
A-12	Class grade entry	**The most-used screen in the system.** Registered students in the chosen order, one row each; numeric mark field (decimals allowed) with **live derived letter and grade point** beside it; an Incomplete option; running completion count. No Withdrawal option exists. See 20.6.	Save draft (whole class, one transaction), submit for approval, print class list, print grade sheet.	Empty: "No students are registered in this class." Conflict error lists the rows another user changed.	Stage 10
A-13	Grade submissions	The Admin's own submissions with status; rejected ones show the Super Admin's reason.	Open, resubmit after rework.	Empty: nothing submitted yet.	A-12
A-14	Grade correction request	The published grade, its history, a new value and a mandatory reason.	Submit request. One pending request per grade.	Error if a request is already pending, showing it.	A-12
A-15	Historical entry	Student and past semester selection; repeating rows of course code, credit hours, grade, optional score and note; per-student progress context. See 20.6.	Save semester (one transaction), set import status. Validations of Section 17.5.	Empty: guidance on starting a student. Duplicate error shows the existing record and asks repeat-or-duplicate.	Stage 6
A-16	Import progress	Counts by status; breakdown by College, Department and cohort; flagged-issue queue; **records entered per week** — the figure that reveals a stall, and the only honest basis for estimating completion.	Filter, drill into a student.	Empty before any entry begins.	A-15
A-17	Registrations	Registrations for an offering or a student; source and status.	Register directly, drop with reason (DEC-14). Blocked once a grade is published.	Error explains the published-grade block.	Stage 9
A-18	Print — class list	Blank-grade class list for the lecturer, print-optimised.	Print.	—	A-12
A-19	Print — class grade sheet	Entered grades for filing, with class, semester, instructor, entry date and a signature block.	Print. Logged.	—	A-12
A-21	Semester-close review	Two worklists produced when a semester closes, reviewed as one step in the close runbook. **(a) Forfeiture candidates** — students with no approved registration in the two most recently closed semesters. **(b) Overdue Incompletes** — every I past its one-semester deadline, with the student, course, semester awarded and deadline.	Forfeiture: confirm or dismiss a candidate, with a reason. Overdue I: open the grade and start a correction request. **Nothing on this screen changes anything automatically**, and the Incomplete path still requires Super Admin approval like any locked grade (CR-09, CR-13).	Empty: each list says so separately — "No students meet the forfeiture condition" / "No Incompletes are overdue."	Stage 5, 10
A-20	Semester export	Scope selection and a plain CSV download of academic data (REQ-B03).	Generate. Logged.	Warns if any grade in scope is unpublished.	Stage 11

20.5 Super Admin screens

ID	Screen	Purpose & content	Actions & validation	Empty / error specifics	Depends on
X-01	Super Admin home	Two queues: grade submissions awaiting approval, corrections awaiting decision. Plus semester states at a glance.	Navigate.	Empty: "Nothing is awaiting your approval."	Stage 11
X-02	Grade submission review	Offering, semester, instructor, submitting Admin, time; every grade with entered mark, derived letter and grade point; a **per-row approve/reject control** alongside an **"approve all"** action; distribution summary; outlier flags including marks within one point of a band boundary; missing-grade count; a running count of what remains undecided.	Approve all; approve selected; reject selected with a reason; reject all remaining with a reason. Approver ≠ entering Admin, verified per grade in-transaction (CR-06).	Error if the submission changed since the page loaded. Partially decided submissions show plainly how many grades still await a decision, because that count blocks the semester from closing.	Stage 10
X-03	Correction review	Old value, proposed value, reason, requesting Admin, the grade's full history, and the effect on the student's CGPA if approved.	Approve, reject with note. Staleness check at approval.	Stale request explained clearly.	Stage 10
X-04	Admin accounts	Admin and Super Admin accounts with status and last login.	Create, disable, enable, reset password. Cannot disable the last active Super Admin.	Error explains the last-Super-Admin rule.	Stage 2
X-05	Semester control	All semesters and states; backward and reopen transitions.	Move backward, reopen. Reason mandatory. Guards of Section 13.2.	Confirmation states plainly what the transition will and will not change.	Stage 4
X-06	Audit log	Filterable log with grouped correlated entries and readable diffs.	Filter, paginate. Viewing is itself logged.	Empty for a filter that matches nothing, with the filter restated.	Stage 1
X-07	Institution-wide records	Read-only access to students and academic records across the institution (REQ-R03).	Search, view. No edit controls exist.	Search-first, paginated.	Stage 5
X-08	Grading policy	Active grade scale and institution settings; version history.	Approve a proposed new version. Never edits a version in use.	Shows which records were computed under which version.	Stage 7

Total: 38 screens (A-21, the forfeiture candidate report, is new in v2.0). No screen exists for announcements, transcripts, dashboards, reports, lecturers, fees or attendance. If one appears during implementation, it is out of scope (Section 5.3).

20.6 The two screens that deserve extra care

A-12 — Class grade entry

Why it matters	It is where the College's academic record is created, thousands of times, by one person reading a paper sheet. Its defect rate is the system's defect rate.
Information	Class identity (course, section, semester, instructor) fixed at the top of the screen; one row per registered student showing Student ID, name, and an input; live letter and grade point beside the input; a running count of "38 of 42 entered"; the semester state and what it permits.
Ordering	Matches the printed sheet exactly — same sort as A-18/A-19, switchable between surname and Student ID (GAP-08). The order is displayed, so the Admin can confirm it matches the paper before starting.

Interaction	Enter or Down commits the field and moves to the next student. Row number and student name stay visible while typing, because the classic error is drifting one row out of alignment with the paper.
Validation	Mark between 0 and 100 with at most one decimal; derived letter must exist in the active scale; Incomplete selectable as a letter; refuses if a grade is already published for that registration; refuses if the semester state does not permit entry. Withdrawal, audit and transfer-credit options do not exist.
Save	Whole class, one transaction, one idempotency key. Never row by row — partial saves are what produce a half-entered class after a dropped connection.
Concurrency	Version token per row. A conflicting save names the rows that changed and who changed them; it does not overwrite and does not discard the Admin's work.
Submit	Separate, deliberate action. Shows a summary and the count of missing grades, and requires explicit confirmation for a partial submission. Grades returned individually by a Super Admin reappear here for rework, each carrying its rejection reason.
States	Empty: no registered students. Error: per-row messages plus a page-level summary. Success: "38 grades saved" or "38 grades submitted for approval", naming the class.

A-15 — Historical entry

Why it matters	It is the largest volume of manual work in the project, and the data it produces feeds every derived academic figure.
Information	Student identity and import status pinned at the top; a list of semesters already entered for this student with record counts; the working semester's rows; a running total of credits for the semester being entered.
Interaction	Add-row on Enter; course code autocompletes but accepts unmatched codes with a flag; credit hours prefill from the catalogue and remain editable, because the paper is authoritative.
Validation	The eight checks of Section 17.5, distinguishing blocking errors from warnings that the Admin can accept with a note.
Save	One semester at a time, one transaction. Duplicate detection runs before the write and offers the repeat-or-duplicate choice.
Status change	Marking Complete opens a confirmation showing semesters, courses, total credits and the resulting CGPA, and states that the provisional marker will be removed.
States	Empty: guidance on selecting a student and semester. Error: duplicate shown side by side with the attempted entry. Success: "6 records saved for First Semester 2023/2024. This student now has 4 semesters entered."

20.7 Responsive requirements

Requirement	Source	Test
Grade sheets and schedules readable on a narrow screen without horizontal scrolling	REQ-D02	Automated: render S-04, S-05 and S-06 at 360 px and assert no horizontal overflow.
Student pages light on a slow connection	REQ-D03, DER-25	Automated: assert transferred JavaScript ≤ 150 KB and interactive within 3 s on a throttled 3G profile.
Admin pages usable but desktop-optimised	REQ-D04	Manual: confirm A-12 and A-15 are fully operable by keyboard at 1366×768 and remain readable, if cramped, at 768 px.
No half-saved records or duplicate submissions on connection loss	REQ-D05	Automated: interrupt a class save mid-flight and assert no partial write; replay the same idempotency key and assert exactly one effect.
Print output correct on A4	REQ-G04	Manual: print A-18 and A-19 for a 40-student class and confirm pagination, headers on every page, and the signature block.

21. Backend and API Architecture

21.1 Layers and responsibilities

Layer	Responsibility	Must never
Route / action	Receive the request, parse input, invoke one service operation, translate the result into a response or a redirect.	Contain business logic, query the database directly, or make an authorisation decision.
Validation	Parse and type-check input against a shared schema; reject malformed input before anything else runs.	Enforce business rules that depend on stored state — that is the service's job.
Service (use case)	One named operation: authorise, load, apply domain rules, transact, audit, return. This is where the system's behaviour lives.	Render anything, or know that HTTP exists.
Domain	Pure functions: GPA engine, plan validator, semester transition table, repeat resolution.	Touch the database, the clock, the session or the network.
Repository	Typed queries and writes. One place per aggregate.	Make authorisation decisions or apply business rules.
Audit	Write audit entries inside the caller's transaction.	Be optional, be skippable, or run outside the transaction.

21.2 The standard mutation shape

Every write in the system follows the same eight steps. The shape is fixed so that a reviewer — or an AI agent — can tell at a glance whether an operation is complete, and so that "we forgot the audit entry" is visible rather than invisible.

```
async function <operationName>(actor, input, idempotencyKey) {

  1. validate(input)                      // shape and types; throws ValidationError listing every
problem
  2. assertCan(actor, ACTION, resourceRef) // permission kernel; throws ForbiddenError
  3. idempotency.check(idempotencyKey)     // returns the stored result if already processed

  4. return db.transaction(async tx => {

       a. load(tx, ...) with row locks on anything being decided upon
       b. assertState(...)                 // semester state, record status, lock status
       c. assertConcurrency(input.version) // optimistic token; throws ConflictError
       d. assertSeparationOfDuties(actor)  // approver != submitter, where applicable
       e. result = domainRule(loadedData)  // pure function, no I/O
       f. write(tx, result)                // all rows, including derived summaries
       g. audit.write(tx, actor, ACTION, entity, oldValue, newValue, reason)

       return result
     })

  5. idempotency.store(idempotencyKey, result)
  6. revalidate affected views
}
```

Step	Why it is not optional
2 before 4	Authorisation before any data is read, so an unauthorised actor cannot infer existence from timing or error shape.

Step	Why it is not optional
4a row locks	Prevents two approvals of the same submission, or two students taking the last seat.
4b inside the transaction	State can change between page load and submit. Checking only at entry is checking the past.
4e pure	Keeps the rule testable in isolation and free of transaction concerns.
4g inside the transaction	DER-21. An audit entry outside the transaction can be lost exactly when it matters.
3 and 5	DER-13. Directly satisfies "no duplicate submissions" on an unreliable connection.

21.3 Service inventory

Service	Operations	Transactional operations
identity	login, logout, changePassword, createStudentAccount, createStaffAccount, resetPassword, disableAccount, enableAccount	Account creation (user + profile + audit)
structure	college and department CRUD, course CRUD, prerequisite add/remove	Prerequisite add (cycle check + write + audit)
calendar	academicYear CRUD, semester CRUD, transitionSemester	**transitionSemester** — guard evaluation + state change + audit
students	enrol, update, changeStatus, setImportStatus	**setImportStatus** — status + provisional recomputation + audit
offerings	create, update, publish, cancel, meeting-time management	Cancel (checks registrations)
planning	createPlan, updatePlan, submitPlan, approvePlan, rejectPlan, overridePrerequisite	**approvePlan** — revalidate + lock plan + create registrations + audit (DER-12)
registration	registerDirect, drop	Both
grades	saveClassDraft, submitClass, approveSubmission, rejectSubmission, requestCorrection, decideCorrection	**All six.** approveSubmission is the largest transaction in the system (DER-11)
academicRecord	enterHistorical, correctHistorical, voidRecord	All three — each recomputes summaries
gpa	computeSemester, computeCumulative, recomputeForStudent	Recomputation participates in the caller's transaction
audit	write, query	write is always inside a caller's transaction
export	semesterExport	Read-only, but audited

21.4 Operations requiring a transaction

Operation	Rows written	What a partial write would mean
approveSubmission	N grade records, N academic records, N semester summaries, N cumulative summaries, 1 submission, N+1 audit entries	Some students see a grade and others do not; some CGPAs move and others do not. The single most damaging partial write available.
approvePlan	1 plan, N registrations, N+1 audit entries	An approved plan whose registrations do not exist — students vanish from class lists and grade sheets.
decideCorrection	1 request, 1 grade, 1 academic record, 2 summaries, 2 audit entries	A grade changed without its CGPA following, or without an audit trail.
saveClassDraft	N grade records, N audit entries	A half-entered class after a dropped connection — explicitly forbidden by REQ-D05.
enterHistorical	N academic records, 2 summaries, N audit entries	A semester partially entered, with a CGPA computed from half of it.

Operation	Rows written	What a partial write would mean
transitionSemester	1 semester, 1 audit entry	A state change with no record of who made it.
createStudentAccount	1 app_user, 1 student, 1 audit entry	An identity with no profile, or a profile nobody can log in to.

21.5 Preventing duplicates and partial failures

Mechanism	Prevents	How
Idempotency keys	Duplicate submissions from retries, double-clicks and reconnects	The client generates a key per intent. A repeated key returns the stored result. A repeated key with a different payload is an error, not a replay.
Database transactions	Partial writes	Every multi-row operation commits atomically.
Row locks	Concurrent decisions on the same record	`SELECT … FOR UPDATE` on the submission, plan or offering being decided.
Optimistic version tokens	Silent overwrite of another user's edit	Stale token → conflict error naming what changed (DER-19).
Unique constraints	Duplicate records surviving a race the application missed	The database refuses; the service translates it into a clear message.
Disabled controls during submission	Accidental double submission	Presentation-layer courtesy only. The idempotency key is the actual control.

21.6 Error handling

Error type	Cause	User sees	Logged
ValidationError	Malformed or out-of-range input	Every failing field named	No — expected
ForbiddenError	Permission kernel refusal	"Not available to your role"	Yes — with actor and action; repeated refusals are a signal
StateError	Wrong semester state or record status	Current state and why the action is unavailable	No
ConflictError	Stale version, or a competing decision	What changed and by whom; a reload option that preserves entered work	Yes — frequent conflicts indicate a workflow problem
NotFoundError	Missing record, or a record the actor may not see	"Not found" — deliberately identical in both cases	Yes
IntegrityError	A constraint the service should have caught	Generic message with a correlation id	**Yes, as a defect.** Reaching a constraint means a missing service-level check.
UnexpectedError	Anything else	Generic message with a correlation id	Yes, with full context, to the error tracker

22. Data Integrity Strategy

Integrity is layered deliberately. The frontend guides, the backend decides, the database refuses. Each layer assumes the one above it may fail, and the strategy is stated per threat rather than as a general aspiration.

22.1 Layer responsibilities

Layer	Role	Assumes	Never
Frontend	Guide the user away from mistakes and give immediate feedback.	That it can be bypassed entirely.	Is relied upon for correctness. A user with a browser console is an unconstrained client.
Backend service	Decide. All business rules, all authorisation, all state checks, all transactions.	That the client sent anything at all.	Trusts a client-supplied role, id or computed value.
Database	Refuse anything that must never be true, regardless of which code path asked.	That the service layer contains a defect.	Is the only place a rule lives, if that rule needs to produce a helpful message.

22.2 Threat-by-threat

Threat	Frontend	Backend	Database
Duplicate records	Disables an already-selected course; disables the submit control while a request is in flight.	Idempotency key; explicit duplicate check with a helpful message before the write.	Unique constraints on (registration), (student, semester, course, attempt), (student, offering), (plan, course), (semester, course, section). The last line, and the one that actually holds.
Invalid grades	Constrains the input to 0–100 and shows the derived letter live.	Re-validates the range; checks the letter exists in the active scale; verifies the score falls in the letter's band.	Check constraints on grade point range and letter membership.
Invalid Student IDs	Format hint and inline pattern check.	Format validation per DEC-02; existence check; status check.	Unique index on the normalised Student ID; foreign keys make an invalid reference impossible.
Invalid course codes	Autocomplete from the catalogue.	Normalises and resolves; historical entry permits an unmatched code with an explicit flag.	Unique index on the normalised code; foreign key on every reference except the flagged historical case.
Broken references	Only offers valid options.	Verifies existence and state before writing.	Foreign keys on every relationship, **all ON DELETE RESTRICT**. No cascade deletes anywhere in the academic path.
Unauthorised updates	Hides controls. Cosmetic only.	Permission kernel before any data access; separation-of-duties re-checked in-transaction.	RLS policies; check constraints on approver ≠ submitter and decider ≠ requester.
Concurrent edits	Warns if the page is stale.	Optimistic version token; row locks on records being decided upon.	Transaction isolation; the version column itself.
Partial saves	Shows a single pending state for the whole batch, not per row.	One transaction per logical operation (Section 21.4).	Atomic commit.
Duplicate submissions	Disables the control; one idempotency key per intent.	Idempotency store; a repeated key returns the original result.	Unique constraint on the key.
Incorrect academic calculations	Displays only what the server returns; computes nothing.	Single pure GPA engine; exact decimals; frozen inputs on every record.	`NUMERIC` columns; range checks; frozen credit hours and grade points.

Threat	Frontend	Backend	Database
Orphaned or rewritten history	Offers deactivate rather than delete.	Refuses deletion where dependents exist, naming them.	RESTRICT foreign keys; no application role holds DELETE on academic tables.
Silent audit gaps	—	Audit written in the same transaction by a single service.	INSERT-only privilege; monotonic id makes a gap detectable.

22.3 Invariants

Statements that must be true of the database at all times. Each is enforced, and each has a test that attempts to violate it through the real service path.

#	Invariant	Enforced by
I-01	Every academic_record has credit_hours > 0 and grade_point between 0.0 and 4.0.	Check constraints
I-02	No two academic_record rows share (student, semester, course, attempt).	Unique constraint
I-03	Every registration has at most one grade_record.	Unique constraint
I-04	A grade with status PUBLISHED has a non-null published_at and locked_at.	Check constraint
I-05	A published grade has exactly one corresponding academic_record.	Service + integration test; verified by a periodic reconciliation query
I-06	The approver of a submission is never its submitter.	Check constraint + service
I-07	The decider of a correction is never its requester.	Check constraint + service
I-08	An APPROVED course plan has one registration per plan item.	Transaction + integration test
I-09	A rejected plan or submission always has a non-empty reason.	Check constraint
I-10	origin = SYSTEM ⇔ grade_record_id is not null.	Check constraint
I-11	At least one ACTIVE Super Admin exists.	Service guard on the disable path + test
I-12	A semester's state is one of the six defined values.	Check constraint
I-13	Every state transition that occurred appears in the audit log.	Transaction + test
I-14	The prerequisite graph is acyclic.	Service cycle check on write + test
I-15	student_semester_summary equals the GPA engine's output for that student and semester.	Recomputed transactionally; verified by a reconciliation query that rebuilds all summaries and compares
I-16	For each (student, course) with multiple records, exactly one has is_repeat_dropped = false.	Repeat resolution + reconciliation query
I-17	No audit_log row is ever updated or deleted.	Privilege; verified at every stage gate

22.4 Reconciliation checks

Three read-only queries that can be run at any time — and are run before every go-live and at every semester end — to confirm the database is internally consistent. They are cheap, they need no special tooling, and they turn "we believe it is correct" into "we checked".

Check	Query	Expected
Summaries match the engine	Rebuild every student_semester_summary and student_cumulative_summary in memory and compare to stored values.	Zero differences (I-15).

Check	Query	Expected
Published grades have records	Published grade_records with no linked academic_record, and academic_records with origin SYSTEM whose grade is not published.	Zero rows (I-05).
Repeat resolution is coherent	(student, course) groups with zero or more than one non-dropped record.	Zero rows (I-16).

23. Testing Strategy

The testing strategy is written before implementation because on this project the tests are the specification of correct behaviour. That is doubly true where an AI coding agent produces the code: the tests are how a plausible implementation is distinguished from a correct one.

23.1 Layers

Layer	Scope	Speed / volume	What it proves
Unit	Pure domain functions: GPA engine, plan validator, transition table, repeat resolution, letter derivation, rounding.	Milliseconds; hundreds of cases.	The arithmetic and the rules are right.
Integration	Services against a real PostgreSQL instance: transactions, constraints, RLS policies, audit writes.	Seconds; one per service operation plus failure paths.	The rules survive contact with the database, and the database refuses what it should.
Authorisation	Every "N" cell of the permission matrix, exercised through the real service path.	Seconds; roughly 60 cases.	Forbidden actions are actually forbidden, not merely hidden.
Workflow / end-to-end	Complete journeys through the browser: plan to registration, entry to publication, correction, import.	Minutes; roughly 20 journeys.	The parts compose into the workflow the College described.
Responsive / performance	Viewport rendering and payload budgets.	Minutes.	REQ-D01 to D03 are met, measurably.
Failure	Interruption, replay, concurrency, constraint violation.	Seconds.	REQ-D05: no half-saved records and no duplicate submissions.
User acceptance	Real Admin office staff, real paper sheets, staging environment.	Days.	The system fits the way the office actually works.

> **INTEGRATION TESTS USE A REAL DATABASE**
>
> Row-level security, check constraints, transaction isolation and unique-constraint races cannot be tested against a mock. A disposable PostgreSQL instance per test run is a hard requirement of this strategy, not a preference — most of this system's integrity guarantees live in the database.

23.2 GPA and CGPA — the highest-value suite

#	Scenario	Assertion
1	Clean semester, four courses (Example 1)	GPA exactly "2.92"; attempted 12; earned 12. String comparison, so a floating-point artefact fails the test.
2	Semester containing an F (Example 2)	GPA "2.33"; attempted 9; earned 6.
3	Repeat, later attempt better (Example 3)	CGPA "3.33"; earlier attempt marked R and is_repeat_dropped; semester 1 GPA unchanged at "1.50"; earned 9; attempted 12.
4	Repeat, later attempt worse (Example 4)	CGPA "2.00" under the stated policy. A second test asserts "4.00" under the best-attempt switch, proving the policy is genuinely configurable.
5	Three attempts at one course	Only the latest counts; two earlier attempts marked R; credit counted once.
6	Repeat across academic years	Chronology resolved by year then sequence, not by insertion order or id.

#	Scenario	Assertion
7	Incomplete (F-26 to F-29)	I is excluded from GPA, attempted and earned; resolving it recomputes correctly; a whole semester of Incompletes returns null, not 0.000. **Withdrawal, audit and transfer-credit grades cannot be entered at all** (F-29).
8	Zero GPA-eligible credits	Returns null, not zero and not a division error. A student with only audited courses does not have a 0.00 GPA.
9	Rounding boundary: 3.4949 / 3.4950 / 3.4951	"3.49", "3.50", "3.50" under half-up. This is the honours-threshold test.
10	Precision: 1/3-type ratios	Exact decimal throughout; no 2.9166666666666665 anywhere, including after JSON serialisation.
11	Provisional propagation	Import status not COMPLETE ⇒ every returned figure flagged provisional. Status COMPLETE ⇒ none flagged.
12	Unpublished grades excluded	A draft and a submitted grade in the same semester change no figure.
13	Mixed origins	Imported and system records combine into one correct CGPA.
14	Frozen values honoured	Changing the course's credit hours after publication does not move any historical figure.
15	Policy version isolation	Records computed under version 1 keep their figures when version 2 is created.

Appendix B holds the full fixture set with input records and expected outputs, ready to be transcribed into the test suite on the day Stage 7 begins.

23.3 Authorisation — every forbidden action

Test family	Cases	Assertion pattern
Student isolation	Own vs. another student's record, plan, schedule, profile	Authenticate as A, request B's resource by direct id → refused, and the response is indistinguishable from "not found".
Unpublished grade invisibility	Draft and submitted grades	Student query returns nothing; direct id access refused; RLS refuses independently of the service.
Super Admin denials	Enrol student; create college, department, course, semester; enter grade; edit grade; correct grade; approve a course plan	Each refused with ForbiddenError, and the database is unchanged afterwards.
Admin denials	Approve a grade submission; approve a correction; move a semester backwards; reopen a semester; manage an Admin account; view the audit log	Each refused; database unchanged.
Separation of duties	Same account submits then approves; same account requests then approves a correction	Refused by the service; and, with the service bypassed, refused by the check constraint.
Locked-grade immutability	Direct update attempt on a published grade outside the correction path	No service path exists; direct database attempt refused by policy.
Audit immutability	Update and delete attempts on audit_log	Refused at the privilege level for every application role.
Role tampering	Request carrying a forged role field	Ignored entirely; the session's role governs.

23.4 Workflow tests

#	Journey	Assertions
W-1	Student builds, submits; Admin approves	Plan locked; one registration per item with frozen credit hours; student appears on the class list; audit entries for submission, approval and each registration share one request id.
W-2	Plan rejected, revised, resubmitted, approved	Reason visible to the student; plan returns to DRAFT; both decisions retained in the audit log; final approval creates registrations exactly once.
W-3	Each validation rule fails in turn	Six tests. Submission refused with the correct rule named and the offending course identified.
W-4	Prerequisite override inside the window	Approval succeeds; override reason, actor and the unmet prerequisite recorded and visible afterwards.
W-5	Prerequisite override after the window closes	Refused; the override control does not exist.
W-6	Grade entry through to publication	Drafts invisible to the student at every step; on approval all grades published and locked in one transaction; academic records written; GPA and CGPA correct; audit complete.
W-7	Submission rejected and reworked	All grades return to DRAFT; reason visible to the Admin; resubmission creates attempt 2; nothing ever became visible to students.
W-8	Full correction cycle	Old and new values, both actors and the reason captured; academic record and both summaries updated; grade re-locked; CGPA moves by the expected amount.
W-9	Correction rejected	Nothing changes; the request survives as evidence.
W-10	Historical entry for a student across three semesters	Records written with origin IMPORTED; CGPA provisional throughout; import status transitions logged.
W-11	Import marked Complete	Provisional badge disappears from every figure and every screen; the status change is audited.
W-12	Historical duplicate attempted	Refused; the existing record is shown; the repeat-or-duplicate choice is offered.
W-13	Semester advances through all six states	Each state permits and forbids exactly the capabilities in Section 13; every transition audited.
W-14	Illegal transitions attempted	Every pair not in the transition table is refused with the legal moves listed.
W-15	Super Admin reopens a closed semester	Reason required; grade operations unlock for that semester only; published grades remain published; audited.
W-16	Close blocked by a pending submission	Refused, naming the outstanding classes.
W-17	Student account lifecycle	Created with a temporary password; forced change unbypassable by direct URL; reset re-arms the flag; disabled account cannot log in.
W-18	Retake through the full cycle	Plan flags the retake; grade published; earlier attempt marked R; CGPA reflects only the later attempt; credit counted once.
W-19	Print outputs	Class list and grade sheet contain the right students in the same order as the entry screen; printing is audited.
W-20	Semester export	CSV opens in a spreadsheet, contains published records only, and the run is audited.

23.5 Failure tests

Scenario	Assertion
Connection drops mid class-grade save	Either all rows saved or none. No partial class in any circumstance.
Same idempotency key replayed	Exactly one effect; the second call returns the first result.

Scenario	Assertion
Same key, different payload	Rejected as an error, not silently replayed.
Two Admins enter the same class concurrently	The second save is refused with a conflict naming the changed rows; no silent overwrite; neither user loses work.
Two Super Admins approve the same submission concurrently	Exactly one succeeds; the other receives a clear conflict; grades published exactly once.
Two students take the last seat	One approval succeeds, one fails with "no seats remaining". Capacity is never exceeded.
Transaction fails at the final step of approveSubmission	Complete rollback: no grade published, no academic record written, no summary changed, no audit entry.
Database unavailable	Clear error with a correlation id; no partial state; the user is not told a write succeeded.
Malformed input to every service	ValidationError listing every problem; nothing written.

23.6 Historical-data tests

Scenario	Assertion
Student with no history at all	CGPA null or provisional as specified; prerequisite checks fail with the import status shown as context.
Student with partial history	All figures computed and flagged provisional; nothing suppressed.
Imported and system records combined	One correct CGPA across both origins.
Imported record with no score, letter only	Accepted; grade point derived from the letter; GPA correct.
Unknown course code	Accepted with a flag; appears in the flagged-issue queue on the progress report.
Credit hours differing from the current catalogue	The entered value is used, not the catalogue value.
Semester earlier than the enrolment year	Warns; accepted with a note; the note is retained and visible.
Repeat spanning an imported and a system record	Carry-over rule applies correctly across origins.
Import status reopened after Complete	Provisional badge returns everywhere immediately; audited with reason.
Progress report accuracy	Counts by status, College, Department and cohort match the underlying data exactly.

23.7 Responsive and performance tests

Test	Threshold
Student results, record and schedule at 360 px	No horizontal overflow; all data reachable (REQ-D02).
Student page JavaScript transferred	≤ 150 KB (DER-25).
Admin page JavaScript transferred	≤ 300 KB.
Time to interactive on a throttled 3G profile	≤ 3 s for student pages.
Class grade entry, 60 students, keyboard only	Complete without a mouse; no dropped keystrokes.
Print class grade sheet, 40 students	Correct A4 pagination; headers repeat on every page; signature block present.

23.8 Regression policy

Rule	Detail
Every defect becomes a test	The failing test is written first, then the fix. The suite therefore grows in exactly the places the system has proved weak.

Rule	Detail
Full suite at every gate	No stage may exit with a failing or skipped test. A skipped test is a failing test with better manners.
GPA suite is protected	Changing an expected GPA value requires a written justification in the pull request referencing the decision that changed. Silent adjustment of an expected figure is the most dangerous edit possible in this repository.
Authorisation suite grows with the matrix	Adding a permission row requires adding its positive and negative tests in the same change.

23.9 User acceptance testing

Aspect	Detail
Who	The actual Admin office staff who will use the system daily, plus the person who will hold the Super Admin role, plus two or three students.
Where	The staging environment, with fabricated data — never production data.
With what	**Real paper grade sheets** from a completed semester, and **real historical records** from the file. Synthetic data will not surface the problems that matter: illegible entries, students not on the list, courses no longer in the catalogue.
Scenarios	Enter a full class from paper and submit it; approve it as Super Admin; correct one grade end to end; enter one student's complete history and mark it Complete; build and approve a course plan; print and file a grade sheet; run the semester export.
Success criteria	Each scenario completes without developer assistance; entry speed for a full class is acceptable to the person doing it; no data-integrity error is produced; every participant can state what the provisional badge means.
Output	A signed record of scenarios passed and defects found. UAT sign-off is a gate condition for go-live (Section 26).

24. Development Stages

Phase 1 is divided into eleven stages ordered by technical dependency, not by convenience. Each produces working, tested, reviewable software. No stage is "backend only" or "frontend only" — a stage that cannot be demonstrated has not been completed.

24.1 Why this order

#	Ordering decision	Reason
1	Audit infrastructure is in Stage 1, before any business feature	REQ-E03 requires it "from day one". Every subsequent service is written against an audit facility that already exists, so no service can be written without one.
2	Identity and the permission kernel precede all academic features	Every later operation is expressed as "who may do this". Retrofitting authorisation is how permission gaps are created.
3	**Historical import (6) comes before the GPA engine (7), offerings (8), planning (9) and grades (10)**	Two reasons. It is the largest work item and the main schedule risk, so the Admin office's data entry must start as early as possible and run in parallel with the remaining development. And its table — `academic_record` — is the input to the GPA engine, the prerequisite check and the retake rule.
4	The GPA engine (7) precedes course planning (9) and grades (10)	Prerequisite checking needs the "has passed" determination, and grade publication must recompute summaries at the moment it publishes. Building GPA later means retrofitting it into two transactions.
5	Offerings and scheduling (8) precede planning (9)	A plan selects offerings, and the schedule-conflict rule needs structured meeting times.
6	Grade management (10) is last of the feature stages	It depends on registrations from planning, on the academic record from import, on the GPA engine, and on the semester state machine. It is also the highest-risk area, and should be built when every dependency is stable and tested.

24.2 Stage 1 — Foundation and Platform

Objective	A deployable, tested, observable skeleton with migrations, environments, the audit facility and the transaction wrapper in place — so that every later stage is built on rails rather than inventing them.
Scope	Repository, application skeleton, three environments, migration pipeline, connection pooling, error monitoring, CI, base layout and print stylesheet, audit table and service, transaction/idempotency/concurrency wrapper, permission-kernel skeleton, institution settings and grade-scale tables with seeds.
Dependencies	Stack approval; hosting accounts provisioned.
Database	`audit` schema and `audit_log` with INSERT-only grants; `institution_setting` ; `grade_scale` seeded with A–F from REQ-C01; `idempotency_key` ; migration tooling and the first migration.
Backend	Audit service; transaction wrapper with idempotency and optimistic concurrency; typed error hierarchy; permission-kernel interface reading the permission table; structured logging; health endpoint.
Frontend	Application shell, responsive layout primitives, form and table components, error and empty-state components, print stylesheet foundation.
Security	HTTPS and security headers; secrets in the platform, never in the repository; separate credentials per environment; verification that no service-role key is exposed to the browser.
Testing	CI running unit and integration suites; disposable PostgreSQL for integration tests; a test proving audit rows cannot be updated or deleted; a test proving a replayed idempotency key produces one effect.
Documentation	Repository README; environment setup; migration procedure; how to add an audited service operation.
Risks	Pooled connections behaving unexpectedly with multi-statement transactions — **must** be proven in this stage, because everything later depends on it. Migration tooling chosen badly and replaced in Stage 4 at high cost.

Acceptance criteria	A migration applied from scratch produces the schema in all three environments. A sample audited operation writes its row and its audit entry in one transaction. Attempting to update or delete an audit row fails at the database level. A deliberately failed transaction leaves nothing behind. A replayed idempotency key returns the original result. CI runs on every push and blocks merge on failure.
Exit gate	G1 — Platform Gate (Section 26).
Complexity	**MEDIUM — low conceptual difficulty, but the decisions made here are expensive to reverse.**

24.3 Stage 2 — Identity, Authentication and Role-Based Access

Objective	Every user can authenticate as the right role, and the permission matrix is enforced in both the service layer and the database.
Scope	app_user; Student-ID login resolution; staff login; forced first-login password change; Admin-issued and Admin-reset passwords; Super Admin management of Admin accounts; the full permission table; RLS policy generation; bootstrap procedure.
Dependencies	Stage 1. **Student ID format confirmed (CR-08):** digits only, first four the admission year, 6–8 digits, existing IDs reused. Pattern `^(19|20)\d{2}\d{2,4}$`. DEC-29 (identity mapping) awaits acknowledgement only.
Database	`app_user`; permission table; RLS enabled on all existing tables with policies; unique index on the normalised login identifier.
Backend	Login with server-side identifier resolution; session handling; password change and reset; account creation and disable; `assertCan` fully implemented; the last-Super-Admin guard; bootstrap script.
Frontend	S-01 login, S-02 forced password change, X-04 admin accounts, role-aware navigation.
Security	Rate limiting on login; identical response and timing for unknown identifier and wrong password; forced-change unbypassable by direct URL; session cookie flags; no password value in any log or audit entry.
Testing	Full authorisation suite skeleton — every "N" cell asserted for the actions that exist so far. Login, lockout, forced change, reset, disable. RLS tested independently of the service layer.
Documentation	The permission matrix as the authoritative reference; bootstrap runbook; account management procedure for the College.
Risks	The Student-ID mapping proving awkward in practice — mitigated by proving a real student login end to end before the gate. A hierarchical permission model creeping in — mitigated by review against Section 11.1.
Acceptance criteria	**A student can log in with their Student ID, is forced to change the temporary password before reaching any other page, and cannot retrieve another student's data through any route including a direct id.** An Admin can create a student account and reset its password. A Super Admin can create and disable an Admin account and cannot disable the last active Super Admin. Every forbidden action tested so far is refused by the service and independently by RLS. No client bundle contains a database credential.
Exit gate	G2 — Identity and Authorisation Gate.
Complexity	**HIGH — the Student-ID mapping, two enforcement layers kept in step, and a failure here compromises everything.**

24.4 Stage 3 — Academic Structure

Objective	The College's organisational hierarchy and course catalogue exist, with prerequisites, and only Admin can change them.
Scope	College, Department, Course, credit hours, prerequisites with cycle detection, activation and deactivation.
Dependencies	Stage 2.
Database	`college`, `department`, `course`, `course_prerequisite`; normalised unique indexes; RESTRICT foreign keys; RLS policies.
Backend	CRUD services with dependency-aware deactivation; code normalisation; prerequisite cycle check; credit-hour change logged as its own action.
Frontend	A-02 to A-05.
Security	Confirm Super Admin is refused all creation and editing here (REQ-R04) — the first real test of the non-hierarchical model.

Testing	Duplicate codes refused; deactivation blocked with dependents and the dependents named; cycle detection including indirect cycles; Super Admin denied every write.
Documentation	Terminology note (Section 12.2); guidance on retiring a course rather than deleting it.
Risks	Low. The main risk is generic naming ("faculty", "program") creeping into the schema or interface.
Acceptance criteria	An Admin can create the College's real structure and a representative set of courses with prerequisites. A duplicate course code is refused. A prerequisite cycle is refused with the loop named. A department with active students cannot be deactivated. A Super Admin cannot create or edit anything in this stage. The words "College" and "Department" appear in the schema, the interface and the error messages.
Exit gate	G3 — Academic Structure Gate.
Complexity	**LOW — conventional CRUD; cycle detection is the only real logic.**

24.5 Stage 4 — Academic Calendar and Semester State Machine

Objective	Academic years and semesters exist, and the six-state machine of Section 13 governs transitions with the correct role restrictions and audit.
Scope	Academic year; semester; the state machine; the transition guards; Super-Admin-only backward and reopen transitions; the capability lookup every later service will consult.
Dependencies	Stage 3. **Confirmed:** exactly two semesters per academic year, no summer term, year label `2026/2027` (CR-10). State is institution-wide; both concurrency invariants enforced.
Database	`academic_year`, `semester` with a state check constraint; unique (year, sequence); date range checks.
Backend	Calendar CRUD; `transitionSemester` with the full guard set; the pure transition table; the capability matrix as a reusable function; mandatory reason on backward transitions.
Frontend	A-06, A-07, X-05.
Security	Backward and reopen transitions restricted to Super Admin at the service layer and by policy; Admin refused.
Testing	Every legal transition succeeds; **every illegal pair is refused** — an exhaustive test over all 6×6 combinations; backward transition without a reason refused; Admin refused a backward transition; every transition produces an audit entry with from-state and to-state.
Documentation	The state machine and the capability matrix as the operational reference for the College.
Risks	The state machine being treated as advisory by later stages — mitigated by making the capability function the only way any service asks "may this happen now?".
Acceptance criteria	An Admin can create a year and its semesters and advance a semester forward through every legal transition. **All 30 illegal transitions are refused with the legal moves listed.** Only a Super Admin can move a semester backwards or reopen it, and only with a reason. Every transition appears in the audit log with from-state, to-state, actor and reason.
Exit gate	G4 — Calendar and State Machine Gate.
Complexity	**MEDIUM — the machine itself is simple; the guards and their interaction with later stages are not.**

24.6 Stage 5 — Students and Academic Profiles

Objective	Students can be enrolled, given accounts, edited and searched, with the import status field in place and correct isolation between students.
Scope	Student record; enrolment with account creation in one transaction; status lifecycle; search; the student's own profile view; import status field (values only, no import workflow yet).
Dependencies	Stages 2, 3. **Confirmed:** statuses Active, Inactive, Suspended, Graduated, Admission Forfeited; graduated students keep read-only access; minimal attribute set. The forfeiture candidate report is part of this stage (CR-09).
Database	`student`; unique normalised student number; import-status column and check constraint; index for search; RLS restricting a student to their own row.
Backend	Enrol (creates app_user + student + audit atomically); update with field-level audit; status change; paginated search.

Frontend	A-09, A-10 (structure, empty of history), S-09, X-07.
Security	Student isolation tested exhaustively — the single most important negative test in the system.
Testing	Duplicate Student ID refused; enrolment atomicity (a failure leaves neither user nor profile); status transitions audited; student A cannot reach student B by any route; Super Admin can read but not write.
Documentation	Enrolment procedure for the Admin office, including how temporary passwords are issued and handed over.
Risks	The Student-ID format changing after data exists. Mitigated by DEC-02 being a blocking prerequisite for Stage 2.
Acceptance criteria	An Admin can enrol a student, who receives a temporary password and can then log in, change it and see their own — empty — record. A duplicate Student ID is refused. **A student cannot access another student's record through any route.** A Super Admin can view students but cannot create or edit one. Every profile change is in the audit log with old and new values.
Exit gate	G5 — Student Records Gate.
Complexity	**LOW to MEDIUM — straightforward, but the isolation testing must be thorough.**

24.7 Stage 6 — Historical Records Import

Objective	The Admin office can begin entering paper records immediately, with duplicate prevention, conflict flagging, import status and a progress report. **This stage exists to unblock the College's largest task.**
Scope	`academic_record` table; historical entry screen; the eight validations of Section 17.5; import status workflow; progress report; retrospective creation of past semesters.
Dependencies	Stages 3, 4, 5. **Confirmed:** the Admin office enters records gradually, after go-live (CR-11) — so this screen must sustain months of intermittent daily use, not a short push. DEC-26 (conflict adjudication) is needed before the office meets its first conflict, which will be within days of starting.
Database	`academic_record` with origin discriminator, frozen credit hours and grade point, the three `counts_in_*` flags, attempt number, source note, and the transfer-source column recommended in Section 16.4.4; unique (student, semester, course, attempt); origin coherence check.
Backend	`enterHistorical` (one semester, one transaction); duplicate detection with the repeat-or-duplicate choice; validation with blocking errors separated from acceptable warnings; import status service; progress-report queries.
Frontend	A-15, A-16; A-10 extended to show entered history.
Security	Historical entry restricted to Admin; Super Admin refused; every write audited with origin.
Testing	The full historical-data suite of Section 23.6; transaction atomicity for a multi-row semester; duplicate refusal; warnings accepted with notes and the notes retained.
Documentation	**A data-entry procedure for the Admin office** — how to handle a missing grade, an unreadable grade, a course no longer in the catalogue, a repeat. This is a deliverable, not an afterthought: it is what makes the import consistent across weeks and across people.
Risks	Entry ergonomics being poor, which silently doubles the largest task in the project. Mitigated by testing the screen with a real Admin and a real paper file **before** the gate, not at UAT.
Acceptance criteria	**An Admin can enter a complete past semester for a real student from a real paper record, in one save, and see it on the student's record marked as imported.** A duplicate is refused and the existing record shown. An unknown course code is accepted with a flag and appears in the progress report's issue queue. Import status moves through its three values with audit. The progress report shows correct counts by status, College, Department and cohort. Entry is fully keyboard-operable.
Exit gate	G6 — Historical Import Gate. Note that by the College's decision the Admin office begins entry **after go-live**, not at this gate (CR-11) — so the ergonomics validated here must hold for months of real use, and this gate is the last cheap opportunity to get them right.
Complexity	**HIGH — the data model is central to the whole system and the ergonomics carry real operational weight.**

24.8 Stage 7 — GPA / CGPA Engine

Objective	Correct semester GPA, CGPA, credits attempted and credits earned, computed from the academic record with the College's confirmed rules, and marked provisional where the import is incomplete.

Scope	The pure engine; the policy object; repeat carry-over resolution; provisional propagation; summary tables and transactional recomputation; student-facing result screens.
Dependencies	Stage 6. **All grading rules confirmed** (CR-01 to CR-05, CR-07): nine-point scale, three-decimal GPA, D pass mark, mandatory-repeat rules, 132-credit graduation total, decimal score entry, Incomplete only. ASM-20 (definition of *major course*) should be confirmed with the Registrar before this stage begins.
Database	`student_semester_summary`, `student_cumulative_summary` with NUMERIC columns, provisional flag and policy version; `grade_scale` extended with the confirmed non-standard grades.
Backend	The pure engine (no I/O); repeat resolution; recomputation service participating in a caller's transaction; the three reconciliation queries of Section 22.4.
Frontend	S-04, S-05; A-10 extended with GPA and CGPA; provisional badge component used everywhere a figure appears.
Security	Confirm only published and imported records enter any calculation, at the query and at the policy layer.
Testing	The complete Appendix B fixture suite. Exact string comparison on every figure. Parameterised tests over the policy flags so that changing a decision changes one table, not the tests.
Documentation	The grading rules as implemented, in plain language, for the Registrar to verify against their own policy. **This document is a deliverable of this stage** — it is how the College confirms the software matches the policy.
Risks	**Still the highest-risk stage, for a different reason than in v1.0.** The rules are now known, so the risk is no longer ignorance — it is that **every one of them differs from the industry default**: nine letters not five, 5-mark bands, two-decimal grade points, three-decimal GPA, most-recent-not-best repeats. An AI coding agent's priors are wrong on all five. Mitigated by fixtures written before the code, exact decimal arithmetic, mandatory human review of the scale seed and derivation function, and the Registrar's written confirmation as a gate condition. Risk R-01b.
Acceptance criteria	**All forty-seven fixtures in Appendix B pass with exact string equality** — including F-01e (59.5 becomes a pass), F-07 (3.1625 rounds to "3.163"), F-13 (a worse retake lowers the CGPA), F-19 to F-24 (mandatory repeats) and F-41 to F-47b (standing boundaries and Incomplete expiry). A student with a repeat sees the correct CGPA, the R marker and the credit counted once. A student with an incomplete import sees every figure badged provisional, including credits-to-graduation. Marking the import Complete removes every badge in one transaction. No floating-point value appears anywhere in the grade path, including JSON payloads. The three reconciliation queries return zero rows. **The Registrar has confirmed in writing that the implemented rules match the College's policy.**
Exit gate	G7 — Academic Calculation Gate. **The most important gate in the project.**
Complexity	**VERY HIGH — modest code, extreme consequence, and dependent on decisions outside the team's control.**

24.9 Stage 8 — Course Offerings and Scheduling

Objective	Admin can create the semester's classes with sections, instructors and structured meeting times; students can see the timetable.
Scope	Course offering; meeting times; publish and cancel; instructor name; optional capacity; student schedule view.
Dependencies	Stages 3, 4. DEC-26 not relevant; GAP-26 capacity decision affects one field.
Database	`course_offering` with frozen credit hours and unique (semester, course, section); `offering_meeting` with a time-order check.
Backend	Offering CRUD gated on semester state; meeting-time management with overlap prevention within an offering; cancel blocked with registrations; post-publication change auditing.
Frontend	A-08, S-06.
Security	Admin only; Super Admin refused (REQ-R04).
Testing	Duplicate section refused; offering against a Closed semester refused; overlapping meetings within one offering refused; timetable renders correctly at 360 px without horizontal scrolling.
Documentation	Semester setup procedure for the Admin office.
Risks	Low. The main risk is modelling schedules as free text, which would silently disable the conflict rule in Stage 9.

Acceptance criteria	An Admin can create a full semester of offerings with sections, instructors and meeting times, and publish them. A student can see a readable timetable on a phone. An offering cannot be created against a Closed semester or cancelled once it has registrations. Post-publication schedule changes are audited.
Exit gate	G8 — Offerings and Scheduling Gate.
Complexity	**MEDIUM — time modelling and the responsive timetable carry most of the effort.**

24.10 Stage 9 — Course Planning and Approval

Objective	Students build and submit plans; the six validations run at both enforcement points; Admin approves or rejects with a reason; approval creates registrations atomically.
Scope	Course plan and items; the validator; submission; Admin review and decision; prerequisite override; registration creation; direct registration and drop (DEC-14).
Dependencies	Stages 6, 7, 8. **Confirmed:** 21-credit ceiling, no meaningful floor (CR-04); passing grade D (CR-05). The override window and expiry (DEC-12) are set at go-live, not at build time — and Section 17.8 explains why one semester may be too short in year one.
Database	`course_plan` with a rejection-reason check; `course_plan_item` with unique (plan, course); `registration` with unique (student, offering) and frozen credit hours.
Backend	The pure validator implementing V1–V6; plan services; `approvePlan` as one transaction that revalidates, locks and creates registrations; override with reason; capacity check under a row lock.
Frontend	S-07, S-08, A-11, A-17.
Security	A student may only touch their own plan; Admin approves; Super Admin refused entirely.
Testing	Workflow tests W-1 to W-5; every validation failing individually and in combination; concurrent approval for the last seat; approval atomicity; every edge case in Section 14.6.
Documentation	Course-planning guide for students; approval procedure and override policy for the Admin office.
Risks	Prerequisite validation behaving badly while historical data is incomplete — the reason the override exists, and the reason the failure message names the import status. Validation drifting between the client and the server — prevented by there being only one implementation.
Acceptance criteria	**A student can build a plan, see every validation failure with the specific course named, submit a valid plan, be rejected with a reason they can read, revise, and be approved — after which registrations exist and appear on the class list.** All six validations run at both submission and approval. A prerequisite override is possible only within the window and is fully audited. A partial approval failure leaves no registrations behind. Two students cannot both take the last seat.
Exit gate	G9 — Course Planning Gate.
Complexity	**HIGH — six interacting rules, two enforcement points, a transactional side effect and real concurrency.**

24.11 Stage 10 — Grade Management Lifecycle

Objective	The complete grade lifecycle: class entry from paper, batch submission, Super Admin approval that publishes and locks, the correction workflow, and printable outputs.
Scope	Grade record; class entry screen; grade submission; approval and rejection; publication into the academic record with GPA recomputation; correction request and decision; print class list and grade sheet.
Dependencies	Stages 7, 9. **Confirmed:** numeric entry with decimals (CR-07); Super Admin approves **individually and in batch** (CR-06) — which makes this stage's approval transaction more complex than v1.0 planned; students see published grades only.
Database	`grade_record` with unique (registration) and a status check; `grade_submission` with the approver ≠ submitter constraint; `grade_correction_request` with the decider ≠ requester constraint and a partial unique on PENDING.
Backend	Class draft save as one transaction; submission; **approveSubmission** — the largest transaction in the system; rejection; correction request and decision with the staleness check; print data services.
Frontend	A-12, A-13, A-14, A-18, A-19, X-01, X-02, X-03; S-04 extended to show published grades.

Security	The full segregation-of-duties suite; drafts invisible to students at every layer; locked grades immutable outside the correction path; printing audited.
Testing	Workflow tests W-6 to W-9 and W-18; every failure test in Section 23.5; the complete forbidden-action suite for grades; print output verified on A4 for a 40-student class.
Documentation	Grade entry, submission and approval procedures; the correction procedure; guidance for the Super Admin on what to look for when reviewing a batch.
Risks	**Highest-consequence stage after Stage 7.** A defect here can publish a wrong grade, lose a grade, or let one account do both halves of the two-key control. Mitigated by three independent enforcement layers, exhaustive negative testing and a full transaction-failure suite.
Acceptance criteria	**An Admin can enter a full class from a paper sheet using only the keyboard — typing 96 and seeing "A+ · 4.00" appear — save it as one transaction, and submit it; a Super Admin can approve the whole batch in one action *or* approve and reject individual grades within it; every approved grade becomes visible to the right student, is locked, is written to the academic record, and every affected GPA, CGPA and repeat obligation is correct — all in one transaction.** An individually rejected grade returns to the Admin with a reason while its classmates stay published. A partially decided submission blocks the semester from closing and says which grades remain. An Admin cannot approve; a Super Admin cannot enter or edit. The same account cannot both enter and approve a grade. A locked grade cannot be changed except through an approved correction, which records old value, new value, both actors and the reason. A dropped connection mid-save leaves no partial class. The printed grade sheet matches the entry screen exactly and shows Incompletes as I.
Exit gate	G10 — Grade Management Gate.
Complexity	**VERY HIGH — the largest transaction, the strictest permissions, the highest data-integrity stakes and the most-used screen.**

24.12 Stage 11 — Hardening, Export, Backup and Go-Live

Objective	The system is operable by the College: exports work, backups are proven by a real restore, landing pages tie the workflows together, performance budgets are met, and acceptance testing has been passed.
Scope	Semester-end CSV export; backup configuration and a rehearsed restore; landing pages S-03, A-01, X-01; audit log view X-06; grading policy view X-08; performance and responsive verification; security review; runbooks; UAT; production go-live.
Dependencies	All prior stages. **Confirmed:** announcements and reports are Phase 2; academic records retained indefinitely; **automatic weekly database backup adopted** (CR-12). Remaining go-live items are the database tier and hosting region.
Database	Final index review against real query patterns; the three reconciliation queries packaged as a runnable check; backup and point-in-time recovery configured on production.
Backend	Export service (streaming, audited); reconciliation runner; health and monitoring endpoints.
Frontend	S-03, A-01, A-16 finalised, A-20, X-01, X-06, X-08; accessibility and responsive polish across all 38 screens.
Security	Full review against Section 18; dependency vulnerability sweep; confirmation that no service key reaches the browser; verification that production data has never been copied to a lower environment.
Testing	Complete regression suite; responsive and performance budgets; **a restore from backup into a scratch environment, verified by comparing record counts and a sample of CGPAs**; full UAT with real staff, real paper sheets and real historical records.
Documentation	Administrator manual; student guide; backup and restore runbook; incident runbook; deployment runbook; the final as-built architecture note recording where this plan was departed from and why.
Risks	Backup discovered not to work at the moment it is needed — mitigated by rehearsing the restore before go-live, which is a gate condition, not a recommendation. UAT surfacing a workflow mismatch late — partly mitigated by testing the two critical screens with real users back in Stages 6 and 10.
Acceptance criteria	The semester export opens in a spreadsheet and contains published records only. **A restore from backup has been performed and verified.** All performance and responsive budgets are met. The full regression suite passes with nothing skipped. The security review is complete with no open critical or high finding. UAT is signed off by the Admin office and the Super Admin. Every runbook exists and has been followed once by someone other than its author. **No out-of-scope feature has been implemented.**
Exit gate	G11 — Go-Live Gate.
Complexity	**MEDIUM — individually straightforward, but broad, and the place where deferred work accumulates.**

25. Stage Deliverables Summary

Every deliverable below is stated as an observable outcome, not as an activity. "Backend completed" is not a deliverable; "a Super Admin can approve a class of 40 grades and every affected CGPA is correct" is.

Stage	Name	Primary demonstrable deliverable	Supporting artefacts
1	Foundation and Platform	An audited, transactional operation runs end to end in three environments, and an audit row cannot be altered or deleted.	Migration pipeline; CI; error monitoring; transaction and idempotency wrapper; seeded grade scale; setup documentation.
2	Identity and RBAC	A student logs in with their Student ID, is forced to change the temporary password, and cannot reach another student's data by any route.	Permission table driving both enforcement layers; RLS policies; bootstrap runbook; the first authorisation test suite.
3	Academic Structure	The College's real hierarchy and a representative catalogue with prerequisites exist, and a Super Admin cannot change any of it.	Cycle detection; deactivation with dependency reporting; terminology note.
4	Calendar and State Machine	A semester advances through all six states; all 30 illegal transitions are refused; only a Super Admin can move backwards, with a reason.	Pure transition table; capability matrix function; state-machine reference for the College.
5	Students and Profiles	An Admin enrols a student who can then log in and see their own record; no student can reach another's.	Atomic enrolment; search; import-status field; enrolment procedure.
6	Historical Import	**An Admin enters a real student's full past semester from a real paper record, in one save, marked as imported — on a screen built to sustain months of daily use.**	Unified academic_record; duplicate and conflict handling; import status; progress report; data-entry procedure.
7	GPA / CGPA Engine	**All forty Appendix B fixtures pass exactly — the nine-point scale, three-decimal rounding, mandatory repeats, provisional marking — and the Registrar has confirmed the rules in writing.**	Pure engine; policy object; summary tables; reconciliation queries; the plain-language rules document.
8	Offerings and Scheduling	A full semester of classes with sections, instructors and meeting times exists, and a student sees a readable timetable on a phone.	Structured meeting times; state-gated editing; semester setup procedure.
9	Course Planning	A student is rejected with a readable reason, revises, and is approved — after which registrations exist and appear on the class list.	Pure validator; two-point enforcement; audited override; atomic approval; student and admin guides.
10	Grade Management	**A class is entered from paper, submitted, approved by a different actor — individually or in batch — published, locked, written to the academic record, and every affected CGPA and repeat obligation is correct, in one transaction.**	Grade lifecycle; correction workflow; print outputs; the full segregation-of-duties suite.
11	Hardening and Go-Live	**A restore from backup is performed and verified, UAT is signed off with real paper records, and the system goes live.**	Semester export; landing pages; audit log view; runbooks; security review; as-built note.

26. Development Gates

Phase 1 is not one continuous coding task. Each stage ends at a formal gate that must be passed before the next stage starts. On a solo build with an AI coding agent, the gate is the primary quality control: it is the point at which a human deliberately stops, reads, verifies and decides.

26.1 What every gate requires

Dimension	Requirement — applies to every gate without exception
Complete	Every stage deliverable exists and can be demonstrated live, not described. Nothing is stubbed, mocked or "to be finished next stage".
Tested	The full suite passes with zero failures and zero skips. New tests exist for every new rule and every new forbidden action. Coverage of the stage's business logic is meaningful, not incidental.
Reviewed	A human has read the code — not merely run it. On this project that specifically means reading, line by line: every authorisation check, every transaction boundary, every audit call, and every piece of academic arithmetic.
Migrations verified	A database built from scratch from the migrations alone matches the deployed schema. No manual change has been made in any console.
Audit verified	Every mutating operation added in the stage writes an audit entry, in-transaction, with the required fields. Audit rows still cannot be updated or deleted.
Authorisation verified	Every "N" cell in the permission matrix touched by the stage has an automated negative test that passes.
Scope verified	Explicit confirmation that nothing out of scope (Section 5.3) was implemented, and that no unrequested feature was added.
Documented	The stage's documentation deliverable exists and is accurate.
Decisions honoured	No blocking decision for this stage was worked around. Any assumption relied upon is recorded in Section 35.
Deployed	The stage is running in the testing environment and has been exercised there, not only locally.

26.2 Gate-specific conditions

Gate	Stage	Additional conditions	Blocks progression if
G1	Foundation	Pooled multi-statement transactions proven to work. Audit immutability proven at the privilege level. Idempotency replay proven. Three environments live and separate.	Transactions behave unexpectedly under the connection pooler — everything downstream depends on this.
G2	Identity and RBAC	A real Student-ID login works end to end. Forced password change is unbypassable by direct URL. RLS refuses cross-student access independently of the service layer. No database credential appears in any client bundle.	Any authorisation path is enforced in one layer only.
G3	Academic Structure	The College's actual College and Department names are loaded. Super Admin is refused every write in this area.	Terminology deviates from the College's own words.
G4	Calendar and State Machine	All 30 illegal transitions refused. Backward transitions Super-Admin-only and reason-mandatory. Every transition audited with from-state and to-state.	Any illegal transition succeeds, or any transition is unaudited.
G5	Students	Student isolation proven exhaustively, including direct-id access. Enrolment atomicity proven by a deliberately failed transaction.	Any route exposes one student's data to another.

Gate	Stage	Additional conditions	Blocks progression if
G6	Historical Import	**A real Admin has entered a real student's history from a real paper file and confirmed the screen is workable.** The data-entry procedure exists. The progress report is accurate.	Entry ergonomics are poor — this gate protects the largest task in the project.
G7	GPA / CGPA	**All forty-seven Appendix B fixtures pass with exact string equality** — the nine-point scale, 59.5 becoming a pass, 3.1625 rounding to "3.163", a worse retake lowering the CGPA, every mandatory-repeat case, the academic-standing boundaries, and the rule that standing is compared on the displayed value so it can never contradict the CGPA beside it. **The Registrar has confirmed in writing that the implemented rules match the College's policy. No floating point anywhere in the grade path. All three reconciliation queries return zero rows.**	**Any fixture fails, or the Registrar has not confirmed. This gate has no discretionary pass.**
G8	Offerings	Meeting times are structured, not free text. Student timetable readable at 360 px without horizontal scrolling.	Schedules cannot support conflict detection.
G9	Course Planning	All six validations enforced at both points. Approval atomicity proven by a deliberately failed transaction. Concurrent last-seat approval resolves correctly. Override is window-bounded and audited.	Any validation runs at only one enforcement point.
G10	Grade Management	**Segregation of duties proven in all three layers, for both the batch and the individual approval path.** Publication atomicity proven by a deliberately failed transaction. A partially decided submission behaves correctly and blocks semester close. Drafts invisible to students at every layer. Locked grades immutable outside the correction path. Print output verified on A4, showing Incompletes as I.	**Any single account can complete an end-to-end grade change.**
G11	Go-Live	**A restore from backup performed and verified. UAT signed off by the Admin office and the Super Admin, using real paper records. Security review closed with no critical or high finding. All runbooks executed once by someone other than their author.**	The backup has not been restored successfully, or UAT is not signed off.

26.3 Gate procedure

Step	Action	Output
1	Run the full test suite in CI	A green run with no skips, recorded.
2	Rebuild the database from migrations and compare to the deployed schema	A confirmed match.
3	Demonstrate every acceptance criterion live in the testing environment	A checklist with each criterion ticked and dated.
4	Read the stage's authorisation, transaction, audit and arithmetic code	Review notes; defects raised as work, not waved through.
5	Confirm scope: nothing out of scope built, nothing extra added	A signed statement in the gate record.
6	Update the assumption and decision registers with anything learned	Updated Sections 34 and 35.

Step	Action	Output
7	Record the gate outcome	Pass, conditional pass with a named and dated remediation, or fail.

ON CONDITIONAL PASSES

A conditional pass is permitted only for items that cannot block downstream work and that have a named owner and a date. It is **never** permitted at G7 or G10. Two conditional passes carrying at once means the project stops and clears them: that is the point at which accumulated "we'll fix it later" becomes the dominant risk.

27. Definition of Done

27.1 Feature-level Definition of Done

A feature is not done because it works in the developer's browser. It is done when every line below is true.

Dimension	Criterion
Requirement	Traces to a REQ or DER identifier in Section 4, and does exactly what that identifier says — no more.
Business rules	Every applicable rule is implemented server-side, in one place, and matches this document. Any deviation is recorded with a reason.
Database integrity	Constraints, unique indexes and foreign keys exist for everything that must never be true. No cascade delete on any academic path.
Authorisation	Enforced in the service layer and by RLS. Every forbidden role has a passing negative test.
Validation	Server-side, listing every failure rather than the first, with messages that name the specific record.
Transactions	Multi-row operations are atomic. A deliberately failed transaction leaves nothing behind.
Idempotency	Mutations carry a key; replay produces one effect.
Concurrency	Editable records carry a version token; conflicts are reported, never silently resolved.
Audit	Every mutation writes an entry, in-transaction, with actor, action, entity, old and new values, reason where required, and timestamp.
Error handling	Every failure path produces a clear message and a correlation id. No stack trace or database text reaches a user.
States	Empty, loading, error and success states are implemented per Section 20.2, and the empty state explains itself.
Responsiveness	Student surfaces work at 360 px without horizontal scrolling; admin surfaces are keyboard-operable at desktop width.
Performance	Within the payload budgets of DER-25. Lists over ~50 rows are paginated.
Testing	Unit tests for rules, integration tests for services and constraints, negative tests for permissions, and a workflow test if the feature completes a journey.
Regression	The whole suite passes. Nothing skipped.
Documentation	Any procedure a College user must follow is written down. Any non-obvious decision is commented where it lives.
Security	No secret in the repository; no credential in a client bundle; no sensitive value in a log.
Review	A human has read the authorisation, transaction, audit and arithmetic code.
Deployed	Running and exercised in the testing environment.

27.2 Stage-level Definition of Done

Every feature-level criterion, for every feature in the stage, plus:

Dimension	Criterion
Acceptance criteria	Every criterion in the stage's specification demonstrated live and recorded.
Gate conditions	All Section 26.1 conditions and the stage's specific conditions met.
Schema fidelity	A database rebuilt from migrations matches the deployed schema exactly.
Scope	Confirmed that nothing out of scope was implemented.
Registers	Decision and assumption registers updated with anything the stage revealed.
No accumulated debt	No more than one conditional-pass item outstanding, with an owner and a date.

27.3 Project-level Definition of Done

#	Criterion	Evidence
1	All eleven gates passed	Signed gate records.
2	Every explicit requirement implemented and tested	The traceability matrix (Section 36) complete, with no row lacking a test.
3	Every blocking decision resolved	Section 34 shows no open blocking item.
4	Grading rules confirmed by the Registrar in writing	Signed confirmation against the plain-language rules document from Stage 7.
5	GPA and CGPA correct against every fixture	Appendix B suite green; reconciliation queries return zero rows.
6	Segregation of duties enforced in three layers	Passing negative tests in service, policy and constraint layers.
7	Audit complete and immutable	Every mutating operation covered; update and delete refused at the privilege level.
8	Backup taken and **restore verified**	A restore performed into a scratch environment, with record counts and sample CGPAs compared (REQ-B02).
9	Semester export works	A CSV opened in a spreadsheet, containing published records only (REQ-B03).
10	Responsive and performance budgets met	Automated checks green.
11	UAT signed off	Real staff, real paper sheets, real historical records, on staging.
12	Security review closed	No open critical or high finding.
13	Runbooks exist and have been executed	Backup, restore, deployment, bootstrap, incident — each run once by someone other than its author.
14	No Phase 2 feature implemented	Scope review against Section 5.3.
15	Handover complete	Repository, this document, the as-built note, and the runbooks sufficient for a different developer to continue.

28. Risk Register

Severity combines probability and impact. Owner names the party who must act — several are the College, not the delivery team, and saying so is part of the point.

ID	Risk	Prob	Impact	Sev	Prevention	Detection	Mitigation if it occurs	Owner / approval
R-01	**GPA / CGPA calculated incorrectly**	Medium was High	Critical	HIGH was CRIT	Rules now fully specified (CR-01…CR-05, CR-07); pure engine; exact decimals; policy as data; 47 fixtures written before code; G7 has no discretionary pass	Appendix B suite; reconciliation queries; UAT against hand-computed records	Fix the engine, recompute affected students in an audited bulk operation, notify anyone who received a wrong figure	Dev Rules received 25 Aug 2026
R-01b	**An AI agent implements the wrong scale from its priors**	High	Critical	CRIT	The nine-point scale, 5-mark bands, two-decimal points, three-decimal GPA and "most recent" repeats are **all** non-default. Fixtures F-01a…F-01i and F-07 exist specifically to catch each wrong default; mandatory human review of the scale seed and the derivation function	Fixture failures — a wrong scale fails a dozen fixtures at once, loudly	Rewrite against the fixtures. The fixtures are the specification, not the code	Dev
R-02	**Historical records incomplete for most of year one**	Certain by design	High	HIGH	Not preventable and not a defect — it is the College's chosen approach (CR-11). Managed by making incompleteness visible: provisional marking on every derived figure, three-state import status, duplicate constraints, validation warnings, QA sampling	Progress report with entry rate; flagged-issue queue; sample re-checks against paper	Correct through the audited path; reopen import status; recompute. **The real mitigation is communication** — students must be told what provisional means before they see it	College / Admin office
R-02b	**A provisional figure is acted on as if final**	Medium	High	HIGH	Badge on every figure everywhere, including credits-to-graduation; standing suppressed entirely; no graduation workflow fires on the 132-credit figure; new students marked Complete so the badge stays meaningful	UAT question: can every participant say what the badge means?	Correct the specific decision; review who else saw the same figure	College / Dev
R-03	**Unauthorised grade modification**	Low	Critical	HIGH	Three enforcement layers; locked grades immutable; correction requires two actors	Audit log review; a change with no correction request is an anomaly	Audit log identifies actor and old value; restore the correct value through the correction path	Dev / Super Admin

ID	Risk	Prob	Impact	Sev	Prevention	Detection	Mitigation if it occurs	Owner / approval
R-04	**Permission failure exposes or allows the wrong thing**	Medium	Critical	CRIT	Allow-list model; one source for both layers; every "N" cell tested; gate condition at every stage	Automated negative suite; audit log anomalies	Patch, review every adjacent path, examine the audit log for exploitation	Dev
R-05	~~Blocking decisions not answered in time~~ — **closed**	—	—	CLOSED	All nine blocking decisions answered 25 August 2026 and implemented in this revision	—	—	Closed
R-23	**A late change to a confirmed grading rule**	Low	High	MED	Every rule is versioned configuration, not code. Records carry a frozen grade point and a policy version, so a new version never rewrites history	Registrar countersignature at G7 is the last checkpoint before figures reach students	New policy version plus a deliberate, approved, audited recomputation. Safe technically; the difficulty is telling students their number moved	Registrar
R-06	**Scope creep into Phase 2**	High	Medium	HIGH	Explicit out-of-scope register; scope check in every gate; no traceability row could justify a Phase 2 feature	Gate review; unexpected screens or tables appearing	Remove it. Partially built Phase 2 features are worse than none	Project owner
R-07	**Prerequisite errors admit or refuse students wrongly**	Medium	High	HIGH	Single validator; two enforcement points; import status shown in failure messages; audited override	Override frequency — a spike means the rule or the data is wrong	Correct the data or the prerequisite; already-approved plans are reviewed, not auto-invalidated	Dev / Registrar
R-08	**Semester workflow errors lock out users or unlock records**	Medium	High	HIGH	Exhaustive transition table; all 30 illegal pairs tested; guards on close	Failed transitions reported; users unable to act	Super Admin corrects the state; the reason is recorded	Dev / Super Admin
R-09	**Duplicate records**	Medium	High	HIGH	Unique constraints as the last line; service-level checks with helpful messages; idempotency keys	Constraint violations in logs; reconciliation queries	Void the duplicate through the audited path; both the error and the fix are retained	Dev
R-10	**Database corruption or data loss**	Low	Critical	HIGH	Managed Postgres with point-in-time recovery; constraints preventing invalid states; no cascade deletes	Reconciliation queries; monitoring; user reports	Point-in-time restore; re-enter from the semester export and the filed paper sheets	Dev / College (DEC-28, DEC-31)
R-11	**Backup fails or proves unrestorable**	Low	Critical	HIGH	Managed automated backup; **restore rehearsed before go-live as a gate condition**; semester CSV export as an independent copy	Restore rehearsal; backup monitoring	Fall back to the CSV export and the filed paper sheets, which is why both are required	Dev / College

ID	Risk	Prob	Impact	Sev	Prevention	Detection	Mitigation if it occurs	Owner / approval
R-12	**Requirement ambiguity resolved by silent assumption**	High	High	HIGH	27 gaps and 37 decisions enumerated; four-label classification; assumption register with stated rework cost	Gate review of the registers; UAT surprises	Raise it, decide it, record it, and re-plan the affected stage	Project owner / VPAA
R-13	**Deployment or migration failure**	Low	High	MED	Three environments; forward-only reviewed migrations; schema verified at every gate; go-live checklist	Failed deployment; schema mismatch check	Roll back the deployment; for schema, restore from the pre-migration backup	Dev
R-14	**Connection loss produces half-saved or duplicate grades**	Medium	Medium	MED	Whole-class transactions; idempotency keys; version tokens	Failure test suite; user reports of odd behaviour	The transaction guarantees nothing partial exists; the user re-saves	Dev
R-15	**Key-person risk — one developer**	Medium	High	HIGH	This document; everything in version control; no undocumented environment step; runbooks for every operation	Handover attempt reveals gaps	A second developer continues from the repository and this plan	Project owner
R-16	**One person holds both Admin and Super Admin accounts**	Medium	High	HIGH	Constraints guarantee one *account* cannot do both halves; the College must not issue both to one person	Account review; audit log showing the same person on both sides	Revoke one account; review the affected grades	**College — policy, not software**
R-17	**Admin office finds the entry screens too slow to use**	Medium	High	HIGH	Keyboard-first design; real-user testing at G6 and G10 rather than at UAT	Entry rate on the progress report; direct feedback	Redesign the screen. Discovering this at UAT costs a stage; discovering it in production costs the import	Dev / Admin office
R-18	**Vendor outage or policy change (hosting, database)**	Low	High	MED	Portable schema and logic; standard Postgres; semester export; paper sheets remain the physical fallback	Provider status; monitoring	Short outage: wait. Long-term: migrate the standard Postgres dump elsewhere	Dev / College
R-19	**Data residency or privacy concern about hosting outside Liberia**	Medium	Medium	MED	Region chosen consciously and recorded; personal data minimised	Raised at approval	Change region — cheapest before any real data exists, which is why DEC-30 is asked now	**College (DEC-30)**
R-20	**AI-generated code contains plausible-but-wrong domain logic**	High	High	HIGH	Fixtures before code; explicit rules in this document; concentrated high-risk logic in small reviewable modules; mandatory human review of arithmetic and authorisation	Fixture failures; review; UAT	Rewrite against the fixtures. The fixtures are the specification, not the code	Dev

ID	Risk	Prob	Impact	Sev	Prevention	Detection	Mitigation if it occurs	Owner / approval
R-21	**Historical import stalls or never finishes**	Medium raised by CR-11	High	HIGH	Entry is now ordinary Admin work competing with daily operations rather than a funded push, so it can quietly deprioritise. Mitigated by: keyboard-first entry designed for sustained use; cohort prioritisation; a progress report whose entry-rate figure makes a stall visible within weeks; provisional marking so the system stays usable meanwhile	**Records entered per week** on the progress report — the single number that reveals a stall	Add temporary help, or narrow the import to currently enrolled students — a decision the College can only take if the rate is visible, which is why the report exists	**College / Admin office**
R-22	**Free-tier database used in production**	Low	Critical	**MED**	Explicitly raised as DEC-31; go-live checklist verifies the tier and its backup retention	Go-live checklist	Upgrade before any real data is entered	**College (DEC-31)**

29. Backup and Recovery

§14 of the source is explicit: once grades are entered, the E-Portal becomes the College's academic record. It also acknowledges that a single semester-end backup leaves up to one semester of entry work at risk, and offers an automatic weekly backup at no additional development cost.

29.1 Minimum Phase 1 requirements

#	Requirement	Source	Implementation
1	Full backup at the end of every semester	REQ-B01	A manual verified snapshot taken as part of the semester-close runbook, in addition to any automated backup.
2	A restore tested at least once	REQ-B02	Restore into a scratch environment before go-live; verify by comparing record counts and a sample of CGPAs. **A gate condition at G11.**
3	End-of-semester export in a plain readable format	REQ-B03	CSV covering students, courses, offerings, registrations, published grades and computed summaries — openable in any spreadsheet, without the system.
4	Signed paper grade sheets continue to be filed	REQ-B04	Process, not software. The system supports it by producing the printable sheet (REQ-G04).

29.2 Recommended improvements

Improvement	Cost	Value
Automated daily backup	Included in a paid database tier	Reduces the exposure window from one semester to one day. The single highest-value line in this section.
Point-in-time recovery	Included in a paid tier	Recovery to any moment within the retention window — the correct answer to "an Admin deleted the wrong thing an hour ago". Effectively removes the risk the source accepts in §14.
Automatic weekly backup	No development cost (§14)	**Adopted by the College** (CR-12, DEC-28). Narrows the exposure window from one semester to one week. Configured in Stage 11 and verified on the go-live checklist.
Off-provider copy	Small	A periodic dump stored outside the hosting provider protects against account loss, not only data loss. Recommend monthly, held by the College.
Restore rehearsal each semester	An hour	A backup nobody has restored is a hypothesis. Rehearsing once per semester turns it into a fact.
Export retained by the College	None	Each semester's CSV kept by the College on its own storage means the academic record survives even total loss of the hosted system.

> **RECOMMENDATION**
>
> Take the paid database tier with daily backup and point-in-time recovery, keep the semester-end CSV export, and keep filing the signed paper sheets. Those three together mean no single failure — provider, application or human — can destroy the College's academic record. The cost difference is small measured against re-entering a semester of paper.

29.3 Recovery procedures

Scenario	Target	Procedure
Accidental data change by a user	Minutes	Usually no restore is needed: the audit log holds the old value and the correction workflow restores it, with a record of both.
Bad migration	< 1 hour	Restore from the backup taken immediately before the migration, then reapply corrected migrations. The pre-migration backup is mandatory for any destructive change.

Scenario	Target	Procedure
Data corruption discovered	< 4 hours	Point-in-time recovery to just before the corruption; re-apply subsequent legitimate work from the audit log and the paper sheets.
Complete database loss	< 24 hours	Restore the latest backup into a new project; repoint the application; verify with record counts and the reconciliation queries; re-enter anything lost from the paper file.
Provider account loss	Days	Rebuild from the off-provider dump and the semester exports. Slow, but survivable — which is the entire reason both exist.

29.4 The semester-end export

Aspect	Specification
Format	CSV, one file per entity, UTF-8, openable in any spreadsheet without the system (REQ-B03).
Contents	Students; colleges and departments; courses; offerings and meeting times; registrations; **published** grades; computed semester and cumulative summaries with their provisional flags.
Excluded	Draft and submitted grades — the export is the academic record, not the working state. Passwords and session data, obviously. The audit log, because an exported audit log is an unaudited copy.
Marked	Every row derived from an incomplete import carries its provisional flag, so the exported figures cannot be mistaken for final ones.
Audited	Yes — a full copy of academic data leaving the system is a significant event.
Who	Admin or Super Admin.

30. Deployment Strategy

No deployment is performed as part of this plan. This section defines the lifecycle the implementation stages will follow.

30.1 Environments

Environment	Purpose	Data	Deployed by	Access
Development	Building	Synthetic seed; reset freely	Local	Developer
Testing / Staging	Gate demonstrations, UAT	Realistic but fabricated, including deliberately incomplete histories	Automatically on merge to main	Developer, College testers
Production	Live academic record	Real	Deliberate promotion from a verified staging build	All users; database access limited to the developer and one named administrator, and audited

Production data is never copied downward. If realistic volume is needed for testing, it is generated. This is a hard rule, verified at G11.

30.2 Pipeline

Step	Action	Gate
1	Branch per stage; commits reviewed as work progresses	—
2	CI runs unit, integration, authorisation and end-to-end suites on every push	Failure blocks merge
3	Preview deployment per branch, with its own throwaway data	—
4	Human review of authorisation, transaction, audit and arithmetic code	Required to merge
5	Merge to main; automatic deployment to testing; migrations applied automatically	—
6	Stage gate demonstrated in testing	Section 26
7	Promotion to production, with migrations applied after a backup	Go-live checklist

30.3 Migrations in deployment

Rule	Detail
Applied in order, forward only	Never edited after merge. A mistake is corrected by a new migration.
Backup before production migration	Mandatory, without exception, for any migration that drops or narrows anything.
Additive first	Where a change can be expressed additively, it is — a new nullable column now, a constraint later, rather than a destructive rewrite.
Verified at every gate	A database rebuilt from migrations must match the deployed schema.

Rule	Detail
Never applied by hand	No schema change is made in a hosted console, in any environment. This is the drift risk of Section 6.3.

30.4 Rollback

Situation	Response
Code defect, schema unchanged	Redeploy the previous build. Fast and safe.
Code defect after an additive migration	Redeploy the previous build; the extra column or table is harmless. This is why additive migrations are preferred.
Defective destructive migration	Restore from the pre-migration backup. **Code rollback alone does not undo a schema change** — the distinction must be understood before, not during, an incident.
Data written incorrectly by a defect	Correct through the audited application path where possible; restore point-in-time where not. Never edit production data directly in a console.

30.5 Monitoring

Signal	Action
Unhandled errors	Alert the developer with release tag, correlation id and stack trace.
IntegrityError reaching the database	Treated as a defect — it means a service-level check is missing.
Repeated ForbiddenError from one actor	Investigate: either a permission bug or someone probing.
Elevated ConflictError rate	Investigate: usually two people working the same class, which is a workflow problem worth solving.
Failed login rate	Rate limiting engages; a spike is investigated.
Slow queries	Reviewed against the index plan in Section 10.6.
Backup status	Checked at every semester close as part of the runbook.

30.6 Go-live checklist

#	Item
1	All eleven gates passed and recorded.
2	Production database on a tier with point-in-time recovery, plus the **automatic weekly backup the College has adopted** (CR-12, DEC-28, DEC-31).
3	A restore rehearsed and verified.
4	Bootstrap Super Admin created; its initial credentials changed.
5	Grade scale seeded with the nine letters plus Incomplete, and **confirmed in writing by the Registrar** against the plain-language rules document from Stage 7. **No Withdrawal, audit or transfer-credit entry exists.**
6	Institution settings configured: `max_credits_per_semester = 21`; `credits_to_graduate = 132`; `gpa_decimal_places = 3`; passing threshold `0.70`; `incomplete_resolution_semesters = 1`; standing bands `probation < 2.000`, `honours ≥ 3.500`; prerequisite override window and expiry date set deliberately — see Section 17.8 on why one semester may be too short in year one.
7	Real College and Department structure loaded.
8	Course catalogue loaded with credit hours and prerequisites.
9	Current academic year and semester created in the correct state.

#	Item
10	Admin accounts created for real staff; no shared accounts; no person holding both Admin and Super Admin.
11	Error monitoring live and alerting a real person.
12	UAT signed off.
13	Runbooks delivered and each executed once.
14	Reconciliation queries return zero rows against production.
15	Historical import progress reviewed and the College explicitly informed which students' figures are still provisional.

31. Future-Phase Compatibility

Phase 1 builds none of the features below. This section records only the architectural decisions already made in Phase 1 that keep them possible without restructuring — and, equally importantly, the decisions deliberately *not* made, because speculative generality is its own defect.

31.1 Phase 2 features

Future feature	What Phase 1 already provides	What Phase 2 would still need to build
Transcript generation and PDF	The unified `academic_record` is already the complete, ordered, frozen academic history with repeat markers and provisional status. A transcript is a rendering of data that already exists in the right shape.	A document layout, an issuance record (student, date, issued by), the official/unofficial watermark distinction from §11, and the incomplete-import warning. No schema change to the academic record.
Announcements and events	Identity, roles and the audit facility. Nothing else — deliberately.	Its own tables, screens and permission rows. It is genuinely additive and touches nothing academic, which is precisely why it was safe to defer.
Role-specific dashboards	Every figure a dashboard would show is already computed by the GPA engine or derivable by query. The summary tables make aggregate queries cheap.	Presentation and aggregation queries only. No new source of truth — and a dashboard must never become one.
Academic reports	Normalised relational data with the indexes of Section 10.6.	Report definitions and export formats. At LCC's scale, direct queries suffice; no warehouse is needed.
In-portal notifications	Every event worth notifying already produces an audit entry with actor, entity and timestamp.	A notification table and a read-state model. The audit log is the natural event source but must not be repurposed as a queue — it is append-only for a reason.

31.2 Later expansion

Feature	Phase 1 architectural accommodation	What it would require
Lecturer accounts	The role model is an allow-list, so a fourth role is a set of new rows rather than a restructuring. `course_offering.instructor_name` already marks where a lecturer link would attach.	A lecturer entity, an identity link, new permission rows, lecturer screens, and a change to the grade workflow so a lecturer submits and an Admin verifies. The grade state machine already supports an extra step; it would gain a state, not a redesign.
Attendance	`registration` and `offering_meeting` are exactly the two entities attendance hangs off.	An attendance table keyed on (registration, meeting, date). No change to anything existing.
Examination management	`grade_record` holds a final result, and `academic_record` stores frozen values, so a component breakdown can be added beneath without disturbing the published grade.	An assessment-component model feeding the final grade. The published grade would become derived rather than entered — a real change, but contained to one service.
Admissions	Student creation is already a single service operation with one entry point.	An applicant model and a conversion path into `student`. The single entry point is what makes this tractable.
Fees and payments	Nothing. Deliberately: no money concept exists anywhere in Phase 1.	Its own domain. The only integration point would be a gate on course registration, which the plan keeps as a single service operation so a precondition can be added in one place.
Online clearance	Nothing beyond identity.	A clearance model and, again, a precondition on registration.
Document management	Nothing. No file storage in Phase 1.	Object storage, access control per document, retention rules — a substantial security surface, correctly deferred.

Feature	Phase 1 architectural accommodation	What it would require
SMS / email notifications	Nothing. No contact channel is assumed, and the synthetic auth identifier is deliberately non-deliverable (Section 18.3).	A provider, verified contact data per student, deliverability handling, and a preferences model.
QR-based verification	Nothing. No signing infrastructure.	A document-issuance record with a verifiable identifier and a public verification endpoint — the first public surface the system would have, needing its own threat model.

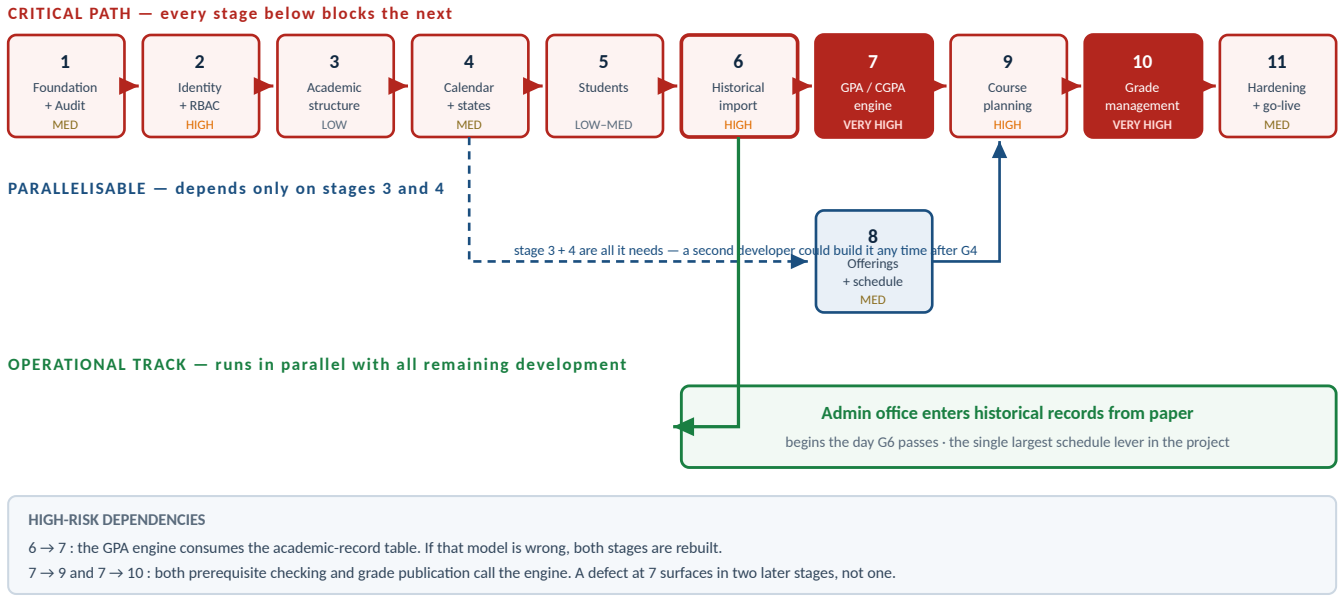
31.3 The four decisions that keep the future open

#	Phase 1 decision	Why it matters later
1	**One unified academic record with frozen values**	Transcripts, reports, dashboards and any future analytics all read one table whose historical figures cannot shift. Without this, every future feature would need its own reconciliation logic.
2	**Business logic in a service layer, not in pages**	A future REST API, a mobile client or a lecturer portal is an adapter over existing services rather than a reimplementation of the rules.
3	**Allow-list permissions driven by a table**	Adding a lecturer role is data plus tests, not a rewrite of every authorisation check.
4	**Audit from day one with a typed vocabulary**	Every future feature inherits a working audit facility, and the College's oversight capability does not degrade as the system grows.

31.4 What Phase 1 deliberately does *not* do for the future

Not done	Reason
No abstract plugin or module system	Speculative extensibility for features nobody has specified adds cost now and constrains the design later.
No public API	No consumer exists. It would be surface to secure, document and version for no user (Section 7.3.6).
No notification infrastructure "ready for later"	An unused queue is untested code that will not fit the eventual requirement.
No empty tables for future features	An unused table is a schema commitment made before the requirement is known.
No generic "documents" or "attachments" model	Document management is a whole domain with its own security obligations; a placeholder would be wrong in detail and would invite misuse.
No multi-tenancy	One College. Adding tenancy speculatively would complicate every query and every policy for a requirement that does not exist.

32. Development Dependency Map



HIGH-RISK DEPENDENCIES

6 → 7 : the GPA engine consumes the academic-record table. If that model is wrong, both stages are rebuilt.

7 → 9 and 7 → 10 : both prerequisite checking and grade publication call the engine. A defect at 7 surfaces in two later stages, not one.

Figure 32.1 — Stage dependency map. Nine of eleven stages sit on the critical path; only offerings (8) is genuinely parallelisable, and only the historical data-entry work — not its software — overlaps the rest of the build.

32.1 Dependency table

Stage	Name	Requires	Blocks	Critical path	Note
1	Foundation	—	Everything	Yes	Nothing can begin before it.
2	Identity and RBAC	1	3, 5, and every later stage	Yes	Every later operation is expressed as a permission.
3	Academic structure	2	4, 5, 6, 8	Yes	Courses are referenced by records, offerings and prerequisites.
4	Calendar and states	3	6, 8, 9, 10	Yes	Every academic record hangs off a semester.
5	Students	2, 3	6, 9	Yes	The subject of every record.
6	Historical import	3, 4, 5	7, 9, 10	Yes	Creates `academic_record`, the input to everything academic. **Also releases the operational track.**
7	GPA / CGPA engine	6	9, 10	Yes	Prerequisite checking and grade publication both call it.
8	Offerings and schedule	3, 4	9, 10	No	**The one parallelisable stage.** A second developer could build it any time after G4.
9	Course planning	6, 7, 8	10	Yes	Produces the registrations grades attach to.
10	Grade management	7, 8, 9	11	Yes	The most dependent stage in the project.
11	Hardening and go-live	All	—	Yes	Cannot start meaningfully until the workflows exist.

32.2 Where a second developer would help

Opportunity	Value	Constraint
Stage 8 built in parallel with Stages 6 and 7	Removes one stage from the critical path	Requires clean separation and a shared understanding of Section 12.3. On a solo build, sequential is safer.

Opportunity	Value	Constraint
Test authoring in parallel with implementation	Genuine, especially for the Appendix B fixtures and the authorisation suite	Fixtures can be written by a second person from this document alone, before Stage 7 begins.
Documentation and runbooks in parallel with Stage 11	Shortens the final stage	Needs the system to be stable enough to document accurately.
Historical data entry	**Largest lever of all**	Not development work. Depends entirely on the College's staffing answer (DEC-23).

33. Complexity Assessment

Relative complexity, not hours. The available information does not support an hour estimate, and offering one would be false precision — the largest single variable in this project's schedule is the College's decision turnaround, not the code.

Stage	Name	Complexity	What drives it	What would reduce it
1	Foundation	MEDIUM	Little conceptual difficulty, but the decisions are expensive to reverse: migration tooling, transaction wrapper shape, audit design, pooled-connection behaviour.	Nothing — this is irreducible setup, and rushing it is paid for in every later stage.
2	Identity and RBAC	HIGH	The Student-ID mapping; two enforcement layers kept in step from one source; a failure here compromises the whole system; the non-hierarchical role model runs against every default assumption.	Answering DEC-02 and DEC-29 before the stage starts.
3	Academic structure	LOW	Conventional CRUD. Only prerequisite cycle detection has real logic.	—
4	Calendar and states	MEDIUM	The machine is simple; the guards, the 30 illegal transitions and the capability matrix that every later service consults are not.	Answering DEC-11, DEC-13 and DEC-34 first.
5	Students	LOW–MED	Straightforward CRUD, but student isolation must be tested exhaustively rather than assumed.	Answering DEC-16 and DEC-21 first.
6	Historical import	HIGH	The `academic_record` model is the foundation of everything academic; duplicate and conflict handling is subtle; entry ergonomics carry real operational weight; the import-status model propagates everywhere.	Answering DEC-26 first; testing the screen with a real Admin before the gate.
7	GPA / CGPA engine	VERY HIGH	Modest code, extreme consequence. Every rule now specified but **every one of them non-default**: nine letters, five-mark bands, two-decimal points, three-decimal GPA, most-recent repeats, mandatory major-course repeats, standing compared on the displayed value. Exact decimal arithmetic; repeat carry-over; provisional propagation; policy versioning.	**Answering DEC-01, 05, 07, 08 before the stage starts.** Without them the stage cannot exit its gate at all.
8	Offerings and schedule	MEDIUM	Time modelling, overlap detection and a responsive timetable that stays readable at 360 px.	—
9	Course planning	HIGH	Six interacting validations, two enforcement points, an atomic side effect creating registrations, a time-bounded audited override, and genuine concurrency on capacity.	Rules now confirmed. Remaining lever: set the prerequisite override window (DEC-12) generously, since in year one most students' history is incomplete.
10	Grade management	VERY HIGH	The largest transaction in the system; the strictest permission requirements; three enforcement layers for segregation of duties; the correction sub-workflow; print output; and the most-used screen in the product, whose ergonomics determine daily throughput.	Having Stages 7 and 9 fully stable. Note the College's answer to DEC-20 — individual *and* batch approval — makes this stage harder than v1.0 planned, not easier.

Stage	Name	Complexity	What drives it	What would reduce it
11	Hardening and go-live	MEDIUM	Individually straightforward, but broad — and it is where deferred work accumulates. UAT with real staff and real paper is where the surprises land.	Not deferring work into it. Every conditional gate pass makes this stage larger.

33.1 Complexity concentration

Observation	Implication for the build
Two stages are VERY HIGH: 7 and 10	They deserve disproportionate review time, disproportionate test coverage, and no conditional gate pass under any circumstances.
Three more are HIGH: 2, 6 and 9	Five of eleven stages carry most of the risk. The remaining six are conventional work.
Complexity is driven by rules and consequences, not by volume of code	The GPA engine is a few hundred lines. Its complexity is that being wrong is expensive and being right depends on answers the team does not control.
Every HIGH and VERY HIGH stage is gated on a College decision	The most effective thing the College can do to reduce this project's risk is answer the blocking decisions in Section 38 early.

34. Decision and Approval Register

Every question raised in version 1.0, with the College's answer of 25 August 2026 and how this revision implements it. **All nine blocking decisions are closed**, and v2.1 closes the last two minor items. **No academic-policy decision remains open.**

ID	Decision	Was blocking	Answer received	How this revision implements it	Status
DEC-01	**Registrar's grading policy**	YES	Supplied as the nine-point scale, passing grade, repeat rules, credit limits and precision listed in Section 1.6. These answers **are** the grading policy of record until a formal document supersedes them.	Section 16 rewritten in full; Appendix B recomputed; the plain-language rules restatement is a Stage 7 deliverable for the Registrar to countersign.	**RESOLVED**
DEC-02	**Student ID format and reuse**	YES	Digits only. First four digits are the admission year (e.g. 202634). Existing IDs are reused unchanged. Total length 6–8 digits so an intake above 99 is representable.	Pattern `^(19|20)\d{2}\d{2,4}$`, normalised and uniquely indexed. Stage 2 validation, seeding and import keying all specified. CR-08.	**RESOLVED**
DEC-03	**Credit-hour limits**	YES	Maximum 21 per semester. No meaningful minimum — one credit hour is acceptable. 132 credit hours to graduate with a BSc.	Validation V2 enforces an upper bound only and rejects an empty plan. Graduation total added as an institution setting and a progress figure. CR-04.	**RESOLVED**
DEC-04	**Score or letter at grade entry**	YES	Numeric marks. Lecturers submit numbers; the system converts to letters — 96 must produce A+.	Score stored to one decimal place; letter and grade point derived and frozen. Entry screen shows the derived letter live. CR-07.	**RESOLVED**
DEC-05	**Passing grade and repeat rules**	YES	Minimum passing grade is D. An F must be repeated. Any D in a major course must be repeated.	Passing threshold 0.70. New *major course* concept, frozen per record; new mandatory-repeat obligation surfaced on the student record and as a warning at plan approval. CR-05.	**RESOLVED**
DEC-06	Decimal scores at entry	No	Decimals are permitted. The system rounds, then assigns the letter.	Half-up to a whole number, once, before band lookup. The entered mark is stored verbatim. Fixtures F-01b to F-01i.	**RESOLVED**
DEC-07	**GPA rounding**	YES	Three decimal places.	Half-up at presentation, stored to six places so the displayed figure is re-derivable. Fixture F-07 protects the direction. CR-03.	**RESOLVED**
DEC-08	**Non-standard grades**	YES	Incomplete (I) is used and appears on both grade sheet and transcript. Withdrawal (W) appears on neither and is not used. Audit and transfer credit not used.	Scale carries nine letters plus I. W, AU and TR removed from every table, screen, fixture and export. Fixture F-29 asserts they cannot be entered. CR-02.	**RESOLVED**
DEC-09	Repeat direction — most recent vs best	No	Not separately re-answered; v4.0 §10.1 stands.	Most recent counts, implemented as a single policy switch. Fixture F-13 shows the consequence when the retake is worse; recorded as ASM-19 for the Registrar to glance at once.	**CARRIED**
DEC-10	Repeats and credits attempted	No	Not separately raised; team position adopted.	A dropped attempt stays in its own semester's GPA and in cumulative credits attempted; it is excluded only from CGPA and credits earned. Section 16.4.4; fixture F-12.	**RESOLVED**

ID	Decision	Was blocking	Answer received	How this revision implements it	Status
DEC-11	Semesters per academic year	No	Exactly two. No vacation or summer school. Year label `2026/2027`.	Calendar model already supported any number; seeds and tests fixed at two. CR-10.	**RESOLVED**
DEC-12	Prerequisite override window	No	Not yet set.	Institution setting with an expiry date, defaulting to the end of the first live semester, Super-Admin controlled. Set at go-live, not at build time.	**Open — config**
DEC-13	Semester state scope	No	Not separately raised.	Institution-wide for Phase 1, consistent with a single Admin office and two terms a year.	**RESOLVED**
DEC-14	Add, drop, withdrawal after approval	No	Not separately raised.	Admin-performed, audited add and drop with a reason, while the semester is in Registration or In Progress. Blocked once a grade is published. Confirm at UAT.	**Open — minor**
DEC-15	Who manages Super Admin accounts	No	Not separately raised.	A Super Admin may manage other Super Admins; at least one active must always remain; the first is created by the bootstrap procedure.	**RESOLVED**
DEC-16	Student status values	No	Graduated students retain read-only access. An admitted student who misses two consecutive semesters forfeits the admission.	Statuses: Active, Inactive, Suspended, Graduated, **Admission Forfeited**. Only Active may build a plan; all non-deleted students retain read access to their own history. CR-09.	**RESOLVED**
DEC-17	Announcements are Phase 2	No	Not contested.	Confirmed Phase 2 per v4.0 §5.2. The §3 reference is recorded as a source inconsistency and does not create Phase 1 scope.	**RESOLVED**
DEC-18	Reports and dashboards are Phase 2	No	Not contested.	Phase 1 gives Super Admin institution-wide read access, the audit log, the import progress report and the semester export. No dashboards, no analytics.	**RESOLVED**
DEC-19	Grade visibility timing	No	Students see grades only when published, never in draft.	Enforced by the grade status model, by query filters and independently by RLS — three barriers. Fixture F-33.	**RESOLVED**
DEC-20	**Grade approval granularity**	YES	**Both.** Super Admin approves grades individually and as a batch.	**Approval model redesigned.** Decisions recorded per grade; a submission may be partially decided; an individually rejected grade returns to the Admin with a reason while its classmates publish. Sections 15.2–15.4. CR-06.	**RESOLVED**
DEC-21	Student attributes collected	No	Not separately raised.	Minimal set: name parts, Student ID, department, enrolment year, status, one contact field. Nothing collected without a use.	**RESOLVED**
DEC-22	Retention and graduated access	No	Graduated students retain read-only access to their academic history.	Academic records retained indefinitely; graduated accounts stay active in read-only mode.	**RESOLVED**

ID	Decision	Was blocking	Answer received	How this revision implements it	Status
DEC-23	**Historical import owner and timing**	YES	The Admin office enters the records, gradually, after the system is live.	Section 17.8 rewritten. **Major operational consequence:** go-live happens with nearly every continuing student's figures provisional, for months. Risk R-02 and R-21 rewritten; provisional marking elevated to a primary feature. CR-11.	**RESOLVED**
DEC-24	Transcript issuance process	No	No fee and no request form. A transcript is valid only when signed and sealed. Signed by the VPAA together with the Dean of Admission and Records.	Recorded for Phase 2. The only Phase 1 implication is that the future issuance record must capture two signatories, not one. CR-12.	**RESOLVED (Phase 2)**
DEC-25	Electronic signature on transcripts	No	No. Signed and sealed by hand.	Confirms OOS-05. No signing infrastructure in any phase currently planned. CR-12.	**RESOLVED**
DEC-26	Who adjudicates a conflicting paper record	No	Not yet named.	The system holds one authoritative row, captures the conflict in the source note and the audit log, and keeps the student's import status at In progress. A named Registrar-office role is still needed — before the first conflict, not before the build.	**Open — operational**
DEC-27	**Academic standing thresholds**	No	**Standing is displayed.** Conventional CGPA bands: probation below 2.000, good standing 2.000–3.499, honours 3.500 and above.	Section 16.7 rewritten. Derived from CGPA at read time, never stored. **Label only** — gates nothing. **Suppressed while provisional**, which in year one is most students. Compared on the displayed three-decimal value so the label can never contradict the figure beside it. CR-14. Fixtures F-41 to F-44.	**RESOLVED**
DEC-28	Backup frequency	No	Automatic weekly database backup.	Adopted, in addition to the semester-end backup and the CSV export. Narrows the exposure window from one semester to one week. Section 29. CR-12.	**RESOLVED**
DEC-29	Synthetic internal identity for login	No	Not separately raised.	Deterministic non-deliverable internal identifier; students only ever see and type their Student ID. Awaiting acknowledgement, with a stated fallback if declined.	**Open — acknowledge**
DEC-30	Data residency region	No	Not separately raised.	Region to be chosen consciously and recorded before production data exists.	**Open — go-live**
DEC-31	Database tier and environments	No	Not separately raised.	Paid tier for production with daily backup and point-in-time recovery, on top of the weekly backup now adopted. Free tier is unacceptable for a system of record.	**Open — go-live**
DEC-32	Fractional credit hours	No	Not raised; all stated values are whole numbers.	Whole-number credit hours only.	**RESOLVED**
DEC-33	Corrections in a Closed semester	No	Not separately raised.	Require a Super Admin reopen first, so changing a sealed term is a deliberate, visible act.	**RESOLVED**

ID	Decision	Was blocking	Answer received	How this revision implements it	Status
DEC-34	One semester in Registration / Grade Submission at a time	No	Not separately raised.	Both invariants enforced. With exactly two terms a year and one Admin office, neither should ever be a constraint in practice.	**RESOLVED**
DEC-35	Meaning of "editable until approved"	No	Not separately raised.	Editable until submitted, and again if rejected. A student cannot alter a plan an Admin is actively reviewing.	**RESOLVED**
DEC-36	Credit-limit override	No	Not raised.	No credit-limit override in Phase 1. The 21-credit ceiling is absolute. Raise it if the office finds a genuine exception at UAT.	**RESOLVED**
DEC-37	Correction affecting approved plans	No	Not separately raised.	Approved plans are not retroactively invalidated; affected registrations are listed for the office to review.	**RESOLVED**
DEC-38	**Incomplete resolution deadline** (new in v2.0)	No	**One semester**, per College policy. At the deadline the system reports; **nothing converts automatically**.	Section 16.4.5 rewritten. Deadline shown to the student. Overdue Incompletes appear on the semester-close review (A-21); each is closed through the **existing correction workflow** — Admin requests, Super Admin approves — so no new permission is created. Conversion to F recomputes GPA, credits and creates a repeat obligation. CR-13. Fixtures F-45 to F-47.	**RESOLVED**
DEC-39	**Forfeiture confirmation is manual** (new in v2.0)	No	Implied by CR-09.	The system **identifies** candidates for forfeiture at semester close and reports them; an Admin confirms each one with a reason. No status changes automatically. Section 12.5.	**Confirm at UAT**

34.1 Summary

Category	Count	Note
Blocking decisions closed	**9 of 9**	DEC-01, 02, 03, 04, 05, 07, 08, 20, 23.
Non-blocking decisions closed	**23**	Including all four Phase 2 transcript items, plus DEC-27 and DEC-38 closed in v2.1.
Open — academic policy	**0**	**None.** Every rule that affects a student's record is now specified.
Open — operational or go-live	**6**	DEC-12, 14, 26, 29, 30, 31 — configuration and confirmations, not design.
Carried from v4.0 for a glance	**1**	DEC-09 / ASM-19 — repeat direction.

35. Assumption Register

Assumptions still relied upon after the College's answers, with the cost of being wrong stated plainly. Six assumptions from version 1.0 have been retired because the decisions of 25 August 2026 replaced them with facts.

ID	Assumption	Why it was made	Affects	Risk if incorrect	How to validate
ASM-01	One institution, one campus, one instance.	Nothing suggests otherwise.	Whole architecture	Multi-tenancy would touch every query and every policy — a fundamental redesign.	One sentence from the VPAA.
ASM-02	English throughout; Africa/Monrovia (UTC+0); dates `DD Mon YYYY`; 24-hour times.	Institutional context.	All screens	Localisation retrofitted across 38 screens.	Confirm at the first gate demonstration.
ASM-03	Low thousands of students; low hundreds of thousands of academic records over a decade.	Single-campus college. The 6–8 digit ID allows up to 9,999 students per intake year, so the schema does not constrain this.	Indexing, pagination	An order of magnitude larger would need partitioning. Not fatal; a Stage 11 surprise.	Ask for current enrolment and typical annual intake.
ASM-06	Any student may register for any department's course.	Nothing restricts it, and restricting would be a new rule.	Course planning	An extra validation rule and message. Low cost.	Ask the Registrar.
ASM-07	The Admin override applies only to prerequisites, not to the 21-credit ceiling.	v4.0 §8 mentions only prerequisites; the ceiling was given as a firm number.	Course planning	The office has no route for a genuine exception — a graduating student needing one extra course.	DEC-36; raise at UAT if it bites.
ASM-09	Temporary passwords are handed over in person or on paper.	No messaging channel exists in Phase 1.	Account creation	If unacceptable, a delivery channel is needed — out of scope.	Confirm with the Admin office.
ASM-10	Lecturers are identified by name only, with no account.	OOS-01 excludes lecturer accounts; DER-15 supplies the name.	Offerings, print output	If lecturers need identity in Phase 1, that is a scope change.	Confirm at G8.
ASM-11	A course offering does not span semesters; a year-long course is two offerings.	v4.0 §4 places the offering under a semester.	Offerings, records	Year-long courses would need a different credit and grading model.	Ask the Registrar.
ASM-12	The Admin office has internet reliable enough for daily work, even if slow.	The source asks for light pages, not offline capability.	Whole architecture	**Raised in importance by CR-11:** historical entry now happens daily, live, for months. If connectivity is frequently absent, that work becomes painful in a way no amount of UI polish fixes.	Ask the Admin office directly, before Stage 6.
ASM-13	Admin and Super Admin are held by different people.	The entire rationale for two roles.	Segregation of duties	The control the College asked for does not exist in practice, however well the software enforces it.	Confirm with the VPAA; risk R-16.
ASM-14	Grade sheets arrive per class, matching the class grade-entry screen.	v4.0 §9 describes exactly this.	Grade entry design	If sheets arrive per student, the entry screen is wrong for the actual workflow.	**Inspect a real grade sheet before Stage 10** — not at UAT.
ASM-15	Past semesters can be created retrospectively, directly in the Closed state.	Historical records must attach to a semester (DER-28).	Historical import	Low. The alternative is free-text term labels, which would break chronological ordering and therefore the repeat rule.	Confirm at G6.

ID	Assumption	Why it was made	Affects	Risk if incorrect	How to validate
ASM-16	The College accepts hosted infrastructure outside Liberia.	The owner selected Vercel and Supabase.	Deployment	On-campus hosting needs an administrator the College does not have.	DEC-30.
ASM-17	Newly enrolled students with no prior study are marked import-Complete at enrolment.	Otherwise every new cohort is permanently provisional.	Import status, GPA display	**Raised in importance by CR-11.** With provisional as the norm for continuing students, first-years must be visibly *not* provisional — otherwise the badge loses all meaning.	Confirm at G6.
ASM-18	No SMS or email in Phase 1, including for password reset.	OOS-06; reset is an Admin function.	Identity	Adding a channel is a scope change with a provider dependency.	Confirm with the VPAA.
ASM-19	**"Most recent attempt" is intended even when the retake is worse.**	v4.0 §10.1 states it plainly; the College did not revisit it.	CGPA	Reversing later means recomputing and republishing every affected CGPA — safe technically, awkward institutionally.	**Show the Registrar fixture F-13 once.** Thirty seconds of reading closes it.
ASM-20	**"Major course" means a course owned by the student's own department.**	CR-05 says "major courses" without defining the term. This is the only definition the data model supports.	Mandatory repeats	If the College means a curated list of core courses instead, the rule fires on the wrong set — too many courses, most likely.	**Ask the Registrar before Stage 7.** If it is a curated list, a flag on the course record covers it — small if caught early.
ASM-21	**"Two inactive semesters" means two consecutive semesters with no approved registration.**	CR-09 says "failed to register in 2 inactive semesters".	Student status	If it means two semesters counted from admission regardless of gaps, the wrong students are flagged.	Confirm the counting rule before Stage 5. Forfeiture is Admin-confirmed (DEC-39), so a wrong list is caught by a human.
ASM-22	**The 132-credit total applies to every BSc programme equally.**	CR-04 gives one number with no per-department variation.	Graduation progress display	Low. It is a displayed figure that gates nothing, and a per-department override is one nullable column.	Ask the Registrar whether any department differs.

35.1 Assumptions retired by the College's answers

Retired	Was	Replaced by
ASM-04	Two semesters per year, pending confirmation.	Confirmed fact — CR-10.
ASM-05	Credit hours are whole numbers.	Confirmed by the stated values — DEC-32.
ASM-08	Import prioritised by cohort, closest to graduation first.	Superseded: the Admin office works gradually post-live, sequence at its own discretion — CR-11.

36. Requirements Traceability

Every explicit requirement mapped through the system to the test that proves it. A requirement with no test row is not implemented, whatever the code says. Acceptance criteria are held at the stage level (Section 24) and referenced by stage number here.

Req	Component / workflow	Database	Backend	Frontend	Test	Stage
REQ-A01	Student-ID authentication	app_user.login_identifier	identity.login with resolution	S-01	Login suite; enumeration-timing test	2
REQ-A02	Admin-issued / reset passwords	app_user, auth store	identity.resetPassword	A-09, X-04	Reset re-arms the change flag	2
REQ-A03	Forced first-login change	must_change_password	identity.changePassword; session restriction	S-02	Direct-URL bypass attempt refused	2
REQ-A04	Three roles	app_user.role	Permission kernel	Role-aware navigation	Role assignment and immutability	2
REQ-A05	Backend + database enforcement	RLS on every table	assertCan on every mutation	—	Full forbidden-action suite; RLS tested without the service	2–11
REQ-A06	Super Admin manages Admins	app_user	identity account services	X-04	Admin refused; last-Super-Admin guard	2
REQ-R01	Student isolation	RLS on student, academic_record, course_plan	Actor-scoped queries	S-04…S-09	Student A cannot reach B by any route	5
REQ-R02	Admin operational authority	Permission table	All admin services	A-01…A-20	Positive tests per action	3–10
REQ-R03	Super Admin governance	Permission table, RLS	Approval and audit services	X-01…X-08	Positive tests per action	10–11
REQ-R04	Super Admin denials	Permission table, RLS	assertCan refusal	Controls absent	Seven explicit negative tests	3–10
REQ-R05	Entry / approval separation	Check: reviewed_by ≠ submitted_by	In-transaction assertion	A-12 vs X-02	Same account submit-then-approve refused in all three layers	10
REQ-R06	Correction two-key rule	Check: decided_by ≠ requested_by	grades.decideCorrection	A-14, X-03	Self-approval refused	10
REQ-R07	Super-Admin-only backward transitions	Permission table	calendar.transitionSemester	X-05	Admin refused; reason mandatory	4
REQ-R08	Audit log visible to Super Admin only	RLS on audit_log	audit.query	X-06	Student and Admin refused	1, 11

Req	Component / workflow	Database	Backend	Frontend	Test	Stage
REQ-S01	Academic hierarchy	college, department, course, academic_year, semester, course_offering	structure, calendar, offerings	A-02…A-08	Referential integrity suite	3, 4, 8
REQ-S02	College / Department terminology	Table and column names	Service names	All labels and messages	Terminology review at G3	3
REQ-S03	Course vs Course Offering	Separate tables	Separate services	A-04 vs A-08	Offering created from a course; frozen credit hours	8
REQ-S04	Admin manages structure	Permission table	structure, calendar	A-02…A-07	Super Admin refused	3, 4
REQ-S05	Schedules created and viewed	offering_meeting	offerings.meetings	A-08, S-06	Timetable renders at 360 px	8
REQ-T01	Student record attributes	student	students.enrol / update	A-09, A-10	Field-level audit	5
REQ-T02	Admin enrols students	student, app_user	students.enrol (atomic)	A-09	Atomicity; Super Admin refused	5
REQ-T03	Student views own information	RLS	Scoped reads	S-04, S-05, S-06	Published records only	7, 8
REQ-H01	Manual per-student import	academic_record	academicRecord.enterHistorical	A-15	Historical suite	6
REQ-H02	Dedicated entry screen	credit_hours entered per record	enterHistorical	A-15	Entered credit hours override the catalogue	6
REQ-H03	Imported-record flag	origin + coherence check	Set by the service, never by input	Marker on A-10, S-05	Origin cannot be forged	6
REQ-H04	Three-state import status	student.historical_import_status	students.setImportStatus	A-15, A-16	Transitions audited	6
REQ-H05	Provisional marking	is_provisional on both summaries	gpa engine returns the flag	Badge on every figure	Badge present when not Complete; absent when Complete	7
REQ-H06	Progress report	Index on import status	Progress queries	A-16	Counts match the underlying data	6
REQ-H07	Conflicts held, not resolved	source_note; unique constraint	Duplicate detection	A-15 conflict dialogue	Contradictory entry refused; note retained	6
REQ-W01	Six semester states	semester.state check	Transition table	A-07, X-05	All 30 illegal pairs refused	4
REQ-W02	State gates four capabilities	—	Capability function used by every service	Disabled controls with explanation	Each capability tested in each state	4–10

Req	Component / workflow	Database	Backend	Frontend	Test	Stage
REQ-W03	Super-Admin-only backward moves	Permission table	transitionSemester	X-05	Admin refused	4
REQ-W04	Transitions audited	audit_log	In-transaction audit	X-06	From-state, to-state and reason present	4
REQ-P01	Student submits a plan	course_plan, course_plan_item	planning.submitPlan	S-07	W-1	9
REQ-P02	Editable until approved	Plan status	Plan state machine	S-07, S-08	Approved plan immutable; rejected plan editable	9
REQ-P03	Validation at two points	—	One validator, two call sites	—	State changed between submit and approve → refused	9
REQ-P04	Prerequisite check	course_prerequisite, academic_record	Validator V1	S-07 messages	W-3; import status shown in the message	9
REQ-P05	Credit-hour limits	institution_setting, department override	Validator V2	Running total on S-07	W-3 above and below the limits	9
REQ-P06	Completed courses not repeatable	academic_record	Validator V3	Retake flag on S-07	W-3; failed course does not trigger it	9
REQ-P07	No duplicates in a plan	Unique (plan, course)	Validator V4	S-07	W-3 and a constraint test	9
REQ-P08	Course availability	course_offering.status	Validator V5	S-07	W-3; cancelled between submit and approve	9
REQ-P09	Schedule conflicts	offering_meeting	Validator V6	S-07 clash indicator	W-3 overlap and adjacency cases	9
REQ-P10	Approve / reject with reason	Check on rejection_reason	planning.approvePlan / rejectPlan	A-11, S-08	W-2; rejection without a reason refused	9
REQ-P11	Prerequisite override	Override columns; setting with expiry	planning.overridePrerequisite	A-11	W-4, W-5	9
REQ-G01	Admin enters grades	grade_record	grades.saveClassDraft	A-12	W-6	10
REQ-G02	Class list in sheet order	registration; index	Class list query	A-12	Screen and print order identical	10
REQ-G03	Validation; duplicates rejected	Unique (registration_id)	Entry validation	A-12	Duplicate refused at both layers	10
REQ-G04	Printable sheet and list	—	Print data services	A-18, A-19	A4 pagination for 40 students	10
REQ-G05	Drafts invisible to students	Status; RLS	Query filters	S-04 excludes them	Student cannot retrieve a draft by any route	10

Req	Component / workflow	Database	Backend	Frontend	Test	Stage
REQ-G06	Class submission	grade_submission	grades.submitClass	A-12, A-13	W-6; partial submission requires confirmation	10
REQ-G07	Approval publishes and locks	Status, published_at, locked_at	grades.approveSubmission (one transaction)	X-02	W-6; atomicity under deliberate failure	10
REQ-G08	Correction workflow	grade_correction_request	requestCorrection / decideCorrection	A-14, X-03	W-8, W-9	10
REQ-G09	Correction audit content	audit_log old/new/reason	audit.write	X-06	Old, new, both actors, reason, timestamp all present	10
REQ-C01 (amended CR-01)	Nine-point grading scale	grade_scale seed, 9 letters + I, NUMERIC(3,2) points	Letter derivation from a numeric mark	Derived letter shown live on A-12	F-01a…F-01i — every band boundary	7
REQ-C09 (new)	Decimal score entry	score NUMERIC(4,1)	Half-up banding, once	A-12, A-15	F-01b, F-01c, F-01e, F-01i	7, 10
REQ-C10 (new)	Three-decimal GPA / CGPA	Summaries stored to 6 dp	Half-up at presentation	S-04, S-05, A-10	F-07 rounding direction; F-04 trailing zeros	7
REQ-C11 (new)	Passing grade D; F and any major-course D must be repeated	grade_scale.is_passing; academic_record.was_major_at_record	Repeat-obligation service	Student record; warning on A-11	F-18…F-25	7, 9
REQ-C12 (new)	132 credit hours to graduate; 21-credit semester ceiling	institution_setting	Validator V2; progress figure	S-05, A-10	F-10 exactly 21; F-38, F-39	7, 9
REQ-C13 (new)	Incomplete on grade sheet and transcript; no Withdrawal	grade_scale entry for I; W absent	Excluded from all three totals	A-12, A-19, S-04	F-26, F-27, F-28, F-29	7, 10
REQ-C14 (new)	Incomplete must be resolved within one semester	institution_setting; deadline derived per record	Semester-close overdue worklist; closure via the correction workflow	S-04, S-05, A-21	F-45, F-46, F-47	7, 10
REQ-C15 (new)	Academic standing displayed from CGPA	institution_setting thresholds; not stored	Derived at read time from the rounded CGPA	S-05, A-10, X-07, export	F-41, F-42, F-43, F-44	7
REQ-T04 (new)	Admission forfeiture after two consecutive unregistered semesters	student.status = ADMISSION_FORFEITED	Semester-close candidate report; Admin confirms each	A-09, A-21	Candidate list correct; no automatic status change	5
REQ-G10 (new)	Super Admin approves individually and in batch	grade_record decision columns; submission partial state	approveGrades (subset or all)	X-02	W-6a batch, W-6b individual, W-7a partial	10

Req	Component / workflow	Database	Backend	Frontend	Test	Stage
REQ-C02	Figures from approved grades	Summary tables	GPA engine	S-04, S-05	Fixtures 1, 2, 12	7
REQ-C03	Registrar's policy is authoritative	policy_version	Policy object	X-08	Written confirmation at G7	7
REQ-C04	Both attempts retained	attempt_no	Repeat resolution	S-05	Fixture 3	7
REQ-C05	Most recent counts in CGPA	is_repeat_dropped	Repeat resolution	S-05	Fixtures 3, 4, 5, 6	7
REQ-C06	R marker on the dropped attempt	is_repeat_dropped	Display derivation	S-05, A-10	Fixture 3	7
REQ-C07	Credits count once	counts_in_earned	GPA engine	S-05	Fixtures 3, 5	7
REQ-C08	Both attempts on the record	academic_record	—	S-05	Fixture 3; transcript is Phase 2	7
REQ-D01	Responsive across devices	—	—	All screens	Viewport suite	11
REQ-D02	No horizontal scrolling	—	—	S-04, S-05, S-06	360 px overflow assertion	11
REQ-D03	Light pages	—	Server rendering; pagination	All	Payload budget assertions	11
REQ-D04	Admin desktop-optimised	—	—	A-12, A-15	Keyboard-only completion	10, 11
REQ-D05	No half-saves or duplicates	Transactions; idempotency_key	Transaction wrapper	Disabled controls	Failure suite (Section 23.5)	1, 10
REQ-E01	Audit content	audit_log columns	audit.write	X-06	Every field present per action	1
REQ-E02	Append-only, Super Admin visible	INSERT-only grants; RLS	—	X-06	Update and delete refused at privilege level	1
REQ-E03	Audit from day one	Built in Stage 1	Audit service exists before any feature	—	Every gate verifies coverage	1
REQ-B01	Semester-end backup	—	—	—	Semester-close runbook	11
REQ-B02	Restore tested	—	—	—	Restore rehearsal — a G11 condition	11
REQ-B03	Plain-format export	—	export.semesterExport	A-20	W-20	11
REQ-B04	Paper sheets filed	—	—	A-19 supports it	Process, confirmed at UAT	10

All 72 explicit requirements — the original 62 plus the ten added or amended on 25 August 2026 — are traced. No requirement lacks a database home, a service, a screen where applicable, and a test.

37. Pre-Development Readiness Assessment

Version 1.0 of this document raised nine blocking decisions. **All nine have been answered** (Section 1.6). This section states what that leaves.

37.1 READY — cleared to build

Area	Stage	Why it is ready
Platform, environments, migrations, audit infrastructure	1	Architecture settled; no College decision required. **Start immediately.**
Student identity	2	**Resolved.** Digits only, first four the admission year, 6–8 digits total, existing IDs reused unchanged. Pattern, validation, seeding and import keying all specified (CR-08).
Role and permission model	2, all	v4.0 §3 and §3.1 are explicit, including the denials. Super Admin account management has a confirmed default.
Academic structure	3	Hierarchy, terminology and Admin-only ownership stated. Whole-number credit hours confirmed.
Academic calendar & semester state machine	4	**Resolved.** Exactly two semesters per year, no summer term; year label `2026/2027` (CR-10).
Students and profiles	5	Attribute set, status values and graduated-student read-only access all confirmed. The admission-forfeiture rule is specified (CR-09).
Historical import design	6	Screen, origin flag, three-state status, provisional marking and progress report all specified. Owner confirmed: the Admin office, gradually, after go-live (CR-11).
GPA / CGPA engine	7	**Resolved — the largest change in this revision.** Nine-point scale with two-decimal grade points; three-decimal GPA, half-up; decimal score entry with a defined banding rule; passing grade D; mandatory-repeat rules; 132-credit graduation total; Incomplete deadline; academic standing bands. 47 fixtures (Appendix B).
Offerings and scheduling	8	No open items.
Course planning	9	**Resolved.** 21-credit ceiling, no meaningful floor, mandatory-repeat obligations surfaced as warnings rather than hard blocks.
Grade management	10	**Resolved.** Numeric entry with decimals; Super Admin may approve individually *and* as a batch; students see published grades only (CR-06, CR-07).
Backup, export, go-live	11	**Resolved.** Automatic weekly database backup adopted, in addition to the semester-end backup and CSV export (CR-12).

37.2 NOT READY

NONE

No stage is blocked. Every one of the nine blocking decisions from version 1.0 has been answered, and each answer is implemented in this revision rather than merely recorded. Stages 1 through 11 may proceed in the order given in Section 24.

37.3 OPEN — academic policy

NONE

Every rule that affects a student's academic record is now specified. The two items outstanding at v2.0 — the Incomplete resolution deadline and academic standing thresholds — were settled on 25 August 2026 and are implemented in this revision (CR-13, CR-14). Nothing in Sections 12 to 23 depends on an answer the College has not given.

37.3.1 Remaining confirmations — configuration, not design

ID	Item	When it is needed
DEC-12	Prerequisite override window and expiry date	Set at go-live, not at build time. Section 17.8 explains why one semester may be too short in year one.
DEC-14	Add / drop after plan approval	Design is specified; confirm the workflow suits the office at UAT.
DEC-26	Who adjudicates a conflicting paper record	Before the Admin office meets its first conflict — days after import starts, not before the build.
DEC-29	Synthetic internal identity acknowledgement	Before Stage 2 ships. A fallback is documented if declined.
DEC-30 / 31	Hosting region and production database tier	Before production data exists.
ASM-20 / 21	Definition of "major course"; counting of "two inactive semesters"	Before Stages 7 and 5 respectively. Small fixes if caught early.

37.4 Two things the College should do alongside the build

#	Action	Why it matters now rather than later
1	**Make one real lecturer grade sheet and one real historical student file available** before Stages 10 and 6 respectively	These are the two highest-volume screens in the system. Validating them against real paper before they are built costs an hour; discovering the mismatch at UAT costs a stage, and discovering it in production costs the import (risk R-17).
2	**Decide how the College will explain provisional figures to students**	Because historical entry happens gradually after go-live (CR-11), nearly every continuing student will see a provisional CGPA for months. The software makes that honest and unmissable; only the College can make it *understood*. A short note in the portal and at registration would prevent a great deal of avoidable anxiety.

37.5 Overall verdict

VERDICT — CLEARED TO PROCEED

Every academic-policy decision is closed and Phase 1 development may begin. Stage 1 depends on nothing outstanding and should start now. Stage 7 — the GPA engine, previously the hard stop — is fully specified, and its gate condition has changed from "obtain the rules" to "pass all forty-seven fixtures exactly", which is a condition the team controls.

What remains is configuration and confirmation: hosting tier and region, two assumption checks with the Registrar, sample documents for screen validation, and UAT sign-off. None of it is a design decision, and none of it blocks a stage from starting.

The order still holds: plan correctly → get approval → then code. The first two are now done.

38. Pre-Development Approval Checklist

Items marked ✔ were answered by the College on 25 August 2026 and are implemented in this revision. Items marked ☐ remain outstanding; **none is an academic-policy decision and none blocks a stage.**

38.1 Before any coding begins

✓	Item	Approver	Reference
✔	Phase 1 scope confirmed; announcements and reports confirmed as Phase 2	VPAA	Section 5; DEC-17, DEC-18
✔	Technology stack and hosting confirmed	Project Owner	Section 7
✔	Role and permission model confirmed, including every explicit denial	VPAA	Section 11
☐	This document (v2.1) countersigned as the Phase 1 engineering baseline	VPAA + Registrar + Project Owner	Section 1.5
☐	Confirmation that Admin and Super Admin will be held by different people	VPAA	ASM-13; risk R-16

38.2 Stage 2 — Identity

✓	Item	Answer received	Reference
✔	**Student ID format and reuse**	Digits only; first four = admission year; 6–8 digits; existing IDs reused	CR-08, DEC-02
✔	Graduated students retain read-only access to their academic history	Yes	DEC-22
☐	Synthetic internal identity approach for Student-ID login acknowledged	Recommended; awaiting acknowledgement	DEC-29

38.3 Stages 4 and 5 — Calendar and students

✓	Item	Answer received	Reference
✔	Semesters per academic year	Exactly two; no summer or vacation school	CR-10, DEC-11
✔	Academic year label format	`2026/2027`	CR-10
✔	Admission forfeiture rule	Two consecutive unregistered semesters forfeits the admission	CR-09
✔	Student status values and their effects	Active, Inactive, Suspended, Graduated, Admission Forfeited	DEC-16

38.4 Stage 6 — Historical import

✓	Item	Answer received	Reference
✔	**Who enters historical records, and when**	The Admin office, gradually, after the system is live	CR-11, DEC-23
☐	Who adjudicates a missing, unreadable or contradictory paper record	Recommended: a named Registrar-office role	DEC-26 — needed before the first conflict, not before the build
☐	A real historical student file made available for screen validation	—	Section 37.4 item 1

38.5 Stage 7 — GPA / CGPA engine

✓	Item	Answer received	Reference
✔	**The grading scale**	Nine letters, A+ 4.00 to F 0.00, 5-mark bands from 60 up	CR-01
✔	**Minimum passing grade**	D — that is, D− (0.70) and above	CR-05, DEC-05
✔	**GPA rounding**	Three decimal places, half-up	CR-03, DEC-07
✔	**Non-standard grades**	Incomplete (I) in use, on grade sheet and transcript. Withdrawal, audit and transfer credit not used	CR-02, DEC-08
✔	**Grade entry format**	Numeric marks, decimals permitted; system derives the letter	CR-07, DEC-04, DEC-06
✔	Repeat rules	F always repeated; any D in a major course repeated; D elsewhere stands	CR-05
✔	Graduation requirement	132 credit hours for a BSc	CR-04
✔	**Incomplete resolution deadline**	One semester. Expiry produces a worklist; closure runs through the correction workflow	CR-13, DEC-38
✔	**Academic standing thresholds**	Probation below 2.000; good standing 2.000–3.499; honours 3.500+. Label only, suppressed while provisional	CR-14, DEC-27
☐	Registrar glances at fixture F-12 and confirms "most recent attempt" is intended even when the retake is worse	Carried from v4.0 §10.1	ASM-19 — non-blocking

38.6 Stages 9 and 10 — Planning and grades

✓	Item	Answer received	Reference
✔	**Credit-hour limits**	Maximum 21 per semester; no meaningful minimum	CR-04, DEC-03

✓	Item	Answer received	Reference
✔	**Grade approval granularity**	Both — individual grades and whole-class batch	CR-06, DEC-20
✔	Grade visibility	Published grades only; never drafts	DEC-19
☐	Prerequisite override window and expiry date	Recommended: end of the first live semester	DEC-12 — set at go-live
☐	Add, drop and withdrawal after plan approval	Recommended: Admin-performed, audited, with reason	DEC-14
☐	A real lecturer grade sheet made available for screen validation	—	ASM-14; risk R-17

38.7 Before go-live

✓	Item	Answer received	Reference
✔	**Backup frequency**	Automatic weekly database backup, plus the semester-end backup and CSV export	CR-12, DEC-28
✔	Transcript signatories and issuance (Phase 2, recorded)	Hand-signed and sealed by the VPAA with the Dean of Admission and Records; no electronic signature; no fee or request form	CR-12, DEC-24, DEC-25
☐	Production database tier confirmed with point-in-time recovery	—	DEC-31; risk R-22
☐	Hosting region acknowledged	—	DEC-30
☐	UAT signed off by the Admin office and the Super Admin	—	Section 23.9
☐	Students informed that figures will read provisional while historical entry proceeds	—	Section 37.4 item 2

38.8 Summary

Category	Count	Note
Blocking decisions outstanding	**0**	All nine closed on 25 August 2026.
Decisions closed across v2.0 and v2.1	**32**	Nine blocking plus twenty-three non-blocking.
Academic-policy items still open	**0**	The Incomplete deadline and academic standing were the last two; both are settled.
Administrative confirmations outstanding	**9**	Signatures, hosting tier, sample documents, UAT — all normal project hygiene, none architectural.
Stages cleared to begin	**11 of 11**	The full Phase 1 sequence is unblocked.

39. Conclusion

The Liberia Christian College E-Portal is a modest system with an unusually high accuracy requirement. It has few users, few screens and no exotic technology. What it does have is a set of numbers — a GPA, a CGPA, a prerequisite decision, a credit total measured against 132 — that must be right, that people will act on, and that were previously produced by hand from paper. Getting those right is the whole project.

Version 1.0 of this plan could not specify those numbers, because eight grading rules were undecided. Version 2.1 can. The nine-point scale, the three-decimal precision, the D pass mark, the 21-credit ceiling, the 132-credit graduation total, the mandatory-repeat rules, the one-semester Incomplete deadline and the standing bands are all now exact, and every one of them appears in Appendix B as a fixture with an expected value. **The project's central risk has moved from "we do not know the rules" to "the rules are written down and the tests hold them" — which is the best position a system of record can be in before its first line of code.**

Four concentrations of effort survive from version 1.0, and the College's answers have reinforced rather than weakened each:

Concentration	Reason, restated for version 2.0
One academic record, with frozen values	Now carrying frozen credit hours, frozen grade points at two decimal places, *and* a frozen major-course flag — because a student changing department must not gain or lose a repeat obligation retroactively. Four features still read this one table.
A pure, policy-driven GPA engine with fixtures written before the code	The scale is unusual enough — nine letters, 5-mark bands, two-decimal points — that an AI coding agent's default assumptions will be wrong in at least three places. The fixtures are what catch that.
Provisional marking that propagates everywhere	**This is now the most important paragraph in the plan.** The College has chosen to enter historical records gradually after go-live. That means year one is a year of provisional figures for nearly every continuing student. The marker is not a corner case to be tidied away; it is the normal state of the system, and it has to be unmissable and honest on every screen it touches.
Segregation of duties enforced in three independent layers	Now covering per-grade approval as well as batch approval, which adds paths without adding exceptions: entry and approval stay in different hands whichever route a Super Admin takes.

The two things the source document said would decide this project were the accuracy of the grading rules and the completion of the historical record entry. The first is settled. The second is now an operational commitment with a named owner and a known shape: the Admin office, working gradually, live. The plan's job there is to make the work sustainable — a keyboard-first entry screen, honest progress reporting, and figures that never overstate what is known.

Nothing has been built. No repository exists, no database has been created, no code has been written. But unlike version 1.0, there is now nothing left to wait for:

> **STATUS**
>
> **PLAN CORRECTLY → GET APPROVAL → THEN CODE.**
>
> The first two are complete. All nine blocking decisions are closed, every answer is implemented in this revision, and all eleven stages are cleared. **Stage 1 begins on countersignature of this document.**

Appendix A — Glossary

Term	Meaning in this project
Academic Record	One completed course result for one student in one semester. The single source of academic truth, whatever its origin.
Academic Year	The College's yearly cycle, labelled `2026/2027`, containing exactly two semesters.
Admin	The operational role. Runs day-to-day academic operations. Cannot approve grades.
Admission Forfeited	A student status. Applied when an admitted student has failed to register for two consecutive semesters (CR-09).
CGPA	Cumulative Grade Point Average across all semesters, with the repeat carry-over rule applied. Reported to three decimal places.
College	The faculty-level organisational unit, e.g. College of Christian Education. The College's own term, used verbatim.
Course	A reusable catalogue definition — code, title, credit hours, prerequisites. Not a class.
Course Offering / Class	A course actually taught in one semester as one section, with meeting times and an instructor.
Course Plan	A student's proposed set of courses for one semester, submitted for Admin approval. Capped at 21 credit hours.
Course Registration	Enrolling a student in a course offering. Created when a plan is approved. *Distinct from Student Enrolment.*
Credits attempted	Credit hours of records that count toward the attempted total. Excludes Incompletes.
Credits earned	Credit hours with a grade point of 0.70 or better. A repeated course counts once. 132 are required for a BSc.
Department	The programme-level unit within a College, e.g. Department of Theology. The College's own term.
Draft grade	A grade entered by an Admin and not yet submitted. Never visible to students.
GPA	Grade Point Average for one semester. Reported to three decimal places.
Grade Point	The numeric value of a letter grade, from A+ 4.00 down to F 0.00, held to two decimal places.
Academic standing	A label derived from CGPA at read time: Probation below 2.000, Good standing 2.000–3.499, Honours 3.500 and above. Suppressed entirely while the CGPA is provisional. Gates nothing.
I (Incomplete)	A real grade-sheet and transcript entry carrying no academic weight until resolved. **Must be resolved within one semester**; at the deadline it appears on a worklist, and closure runs through the ordinary two-key correction workflow.
Idempotency key	A client-generated token making a repeated request safe — the answer to "no duplicate submissions".
Imported record	An academic record transcribed from paper, flagged `origin = IMPORTED`.
Import status	Per student: Not started, In progress, Complete. Drives the provisional marker.
Locked grade	A published grade, changeable only through an approved correction request.
Major course	A course owned by the student's own department, frozen onto the record at the moment the result is written. Determines whether a D triggers a mandatory repeat.
Mandatory repeat	An outstanding obligation to retake a course, created by an F anywhere or by any D in a major course. Advisory and surfaced, never a silent block.
Provisional	A derived figure computed from an incomplete historical record. Badged wherever it appears. In year one, the normal state.
Quality points	Grade point × credit hours for one record. The numerator of every GPA.
R (Repeated)	A display marker on an earlier attempt dropped from CGPA. Not a grade.
RLS	Row-Level Security. Database policies restricting which rows a session may see or change.
Segregation of duties	The rule that grade entry and grade approval are performed by different actors.
Semester	One of exactly two terms within an academic year, carrying the six-state machine.
Student Enrolment	Creating a student's record and account. *Distinct from Course Registration.*

Term	Meaning in this project
Super Admin	The governance role: approves grades individually or in batch, approves corrections, manages Admin accounts, reads the audit log, controls backward semester transitions. Not a superset of Admin.

Appendix B — GPA / CGPA Test Fixtures

These fixtures are the executable specification of Section 16, recomputed in full against the confirmed nine-point scale. They are written before the engine exists and become the automated suite for Stage 7. **Every expected value is compared as an exact string**, so a floating-point artefact fails the test rather than passing quietly.

Scale in force: A+ 95–100 = 4.00 · A− 90–94 = 3.70 · B+ 85–89 = 3.30 · B− 80–84 = 2.70 · C+ 75–79 = 2.30 · C− 70–74 = 1.70 · D+ 65–69 = 1.30 · D− 60–64 = 0.70 · F below 60 = 0.00 · I excluded. **Rounding:** GPA and CGPA half-up to three decimal places. Scores rounded half-up to a whole number before band lookup. **Passing:** 0.70 and above. **Graduation:** 132 credit hours.

B.1 Score-to-letter derivation

ID	Entered score	Rounds to	Expected letter	Grade point	Purpose
F-01a	96	96	**A+**	4.00	The College's own worked example.
F-01b	94.4	94	**A−**	3.70	Rounds down, stays below the A+ band.
F-01c	94.5	95	**A+**	4.00	Half-up carries the mark into A+.
F-01d	59.4	59	**F**	0.00	The pass boundary, from below.
F-01e	59.5	60	**D−**	0.70	Half-up turns a fail into a pass. The single most consequential rounding in the system.
F-01f	100	100	**A+**	4.00	Upper bound accepted.
F-01g	0	0	**F**	0.00	Lower bound accepted.
F-01h	100.1 / −1	—	**Rejected**		Out of range. Validation error, nothing written.
F-01i	87.5 stored	88	**B+**	3.30	The entered mark is stored as 87.5; only the band lookup uses 88.

B.2 Semester GPA

ID	Input records	Quality pts	Credits	Expected GPA	Also asserts
F-02	CSC 101 3cr 96 (A+) · THE 110 3cr 88 (B+) · MAT 105 4cr 73 (C−) · ENG 101 2cr 82 (B−)	34.10	12	**"2.842"**	attempted 12; earned 12. 2.8416666… rounds up.
F-03	CSC 201 3cr 84 (B−) · THE 210 3cr 55 (F) · MAT 201 3cr 91 (A−)	19.20	9	**"2.133"**	attempted 9; earned 6. 2.1333333… rounds down. The F stays in the denominator.
F-04	Three 3cr courses at 96, 98, 100 (all A+)	36.00	9	**"4.000"**	Trailing zeros preserved. "4" or "4.0" is a failure.
F-05	MAT 101 3cr 45 (F) · PHY 101 3cr 58 (F)	0.00	6	**"0.000"**	attempted 6; earned 0. **Not null** — the student did attempt. Two repeat obligations created.
F-06	THE 200 3cr **I** only	—	0	**null**	Zero eligible credits returns null, never 0.000 and never a division error.
F-07	CSC 401 3cr A+ · THE 401 3cr B+ · ENG 401 2cr C−	25.30	8	**"3.163"**	25.30 ÷ 8 = 3.1625 exactly. Half-up gives 3.163; truncation gives 3.162. **The rounding-direction test.**
F-08	BIB 300 3cr B+ · HIS 300 3cr C+	16.80	6	**"2.800"**	A value that terminates before three places still prints three.
F-09	Single 1cr course at 72 (C−)	1.70	1	**"1.700"**	Minimal case; no division artefacts.

ID	Input records	Quality pts	Credits	Expected GPA	Also asserts
F-10	**Maximum load:** seven 3cr courses at A+, A−, B+, B−, C+, C−, D+	57.00	21	**"2.714"**	2.7142857… rounds down. Also asserts V2 accepts a plan of exactly 21 credit hours and rejects 22.
F-11	Student in Theology: THE 210 3cr 62 (D−, major) · MAT 105 3cr 66 (D+, non-major) · ENG 101 3cr 78 (C+)	12.90	9	**"1.433"**	earned **9** — D grades earn credit. Exactly **one** repeat obligation, against THE 210.

B.3 Repeats and carry-over

ID	Input across semesters	Expected CGPA	Also asserts
F-12	2025/26 S1: CSC 101 3cr 52 (F), THE 110 3cr 87 (B+) 2025/26 S2: CSC 101 3cr 83 (B−), MAT 105 3cr 92 (A−)	**"3.233"**	S1 GPA stays "1.650", S2 GPA "3.200"; attempt 1 marked R and repeat-dropped; earned 9; cumulative attempted 12; the F obligation is cleared. **The core carry-over fixture.**
F-13	2025/26 S1: BIB 101 3cr 97 (A+) 2026/27 S1: BIB 101 3cr 71 (C−)	**"1.700"**	Most-recent policy per v4.0 §10.1 — the retake **lowered** the CGPA from 4.000. Under a best-attempt switch the same fixture must yield "4.000", proving the policy is genuinely configurable (ASM-19).
F-14	S1: PHY 101 3cr 45 (F) · S2: PHY 101 3cr 62 (D−) · S3: PHY 101 3cr 86 (B+)	**"3.300"**	Three attempts; only the third counts; two marked R; credit counted once (earned 3); cumulative attempted 9.
F-15	2024/25 S2: ENG 101 3cr 72 (C−) 2025/26 S1: ENG 101 3cr 93 (A−)	**"3.700"**	Chronology resolved by academic year then semester sequence — **not** by insertion order or record id.
F-16	S1 **imported**: CSC 101 3cr F S2 **system, published**: CSC 101 3cr 96 (A+)	**"4.000"**	The carry-over rule works across origins. A repeat spanning a paper record and a system record behaves identically.
F-17	S1: MAT 101 3cr 82 (B−), ENG 101 3cr 82 (B−) S2: MAT 101 3cr 88 (B+) flagged as a retake	**"3.000"**	Repeating a passed course is permitted when flagged; earned **6**, not 9 — credit is not earned twice.

B.4 Mandatory repeat obligations

ID	Scenario	Expected
F-18	F in a non-major course	Obligation created. F always requires a repeat regardless of major status.
F-19	D− (62) in a course owned by the student's own department	Obligation created. Credit is still earned and the grade still counts in GPA.
F-20	D+ (68) in a course owned by the student's own department	Obligation created. **Both** D grades trigger the rule, not only D−.
F-21	D+ (68) in a course owned by another department	**No obligation.** The grade stands and the credit is earned.
F-22	Major-course D− later retaken and passed with C− (71)	Obligation cleared. The D− attempt is marked R and dropped from CGPA.
F-23	Major-course D− later retaken and graded D+ (67)	**Obligation persists.** The retake is itself a D in a major course, so it creates its own obligation. The engine evaluates the kept attempt, not the history.
F-24	Student earns D− in a Theology course, then transfers to the Department of Education	**Obligation persists unchanged.** `was_major_at_record` was frozen at the time the result was written; a department change never rewrites obligations in either direction.
F-25	A student with two outstanding obligations submits a plan including neither	Plan is **not blocked**. The Admin sees a warning naming both courses; approving anyway is permitted and the warning plus the decision are both audited.

B.5 Incomplete

ID	Scenario	Expected
F-26	CSC 301 3cr 96 (A+) · THE 301 3cr **I** · MAT 301 3cr 78 (C+)	GPA **"3.150"** (18.90 ÷ 6); attempted 6; earned 6. The I appears on the grade sheet and would appear on a transcript.
F-27	The same I is later resolved to 85 (B+) through the correction workflow	GPA recomputes to (12.00 + 9.90 + 6.90) ÷ 9 = 28.80 ÷ 9 = **"3.200"**; attempted 9; earned 9; audit records old value I, new value B+, both actors and the reason.
F-28	An entire semester of Incompletes	Semester GPA **null**, not 0.000. CGPA unaffected by that semester.
F-29	Any attempt to enter a Withdrawal (W), audit or transfer-credit grade	**Rejected.** These letters do not exist in the scale (CR-02) and must not be selectable, enterable or importable.

B.6 Provisional marking and integrity

ID	Scenario	Expected
F-30	Import status **In progress**; two of four expected semesters entered	All figures computed from the entered semesters; every one returned with `is_provisional = true`; the badge appears on every screen showing them, including the credits-to-graduation figure.
F-31	Import status moves to **Complete**	Same figures, `is_provisional = false` everywhere, in one transaction; the change audited with the actor.
F-32	Import status **Not started**; current-semester grades only	Figures computed and flagged provisional. **Not suppressed** — a hidden number sends the office back to manual calculation. This is the ordinary case in year one.
F-33	A draft grade and a submitted grade exist in the semester	Neither changes any figure. REQ-C02.
F-34	Course credit hours edited from 3 to 4 after publication	No historical figure moves. The record's frozen credit hours are used (DER-07).
F-35	Grade scale version 2 created with different points	Records carrying policy version 1 keep their figures. New records use version 2.
F-36	Any computed figure serialised to JSON and back	Exact decimal string preserved. No `3.6999999999999997` anywhere — grade points of 3.70 and 2.30 are precisely the values that expose binary floating point.
F-37	A published grade corrected from F (55) to B− (83)	Grade, academic record, both summaries and the repeat obligation all update in one transaction; the CGPA moves by exactly the expected amount; the obligation is cleared; audit captures old value, new value, both actors and the reason.
F-38	Student with 118 credits earned	Record shows 118 of 132, 14 remaining. If the import is incomplete, **the remaining figure is badged provisional too** — it is the number most likely to be mistaken for graduation clearance.
F-39	Student with 135 credits earned	Shows "requirement met". **No workflow fires.** Graduation clearance is not a Phase 1 feature.
F-40	Reconciliation after 100 randomised operations	All three queries in Section 22.4 return zero rows: summaries match the engine, published grades all have records, repeat resolution is coherent.

B.7 Academic standing and Incomplete expiry

ID	Scenario	Expected
F-41	CGPA exactly 3.500, import Complete	**Honours.** The boundary is inclusive. A CGPA of 3.499 returns **Good standing**.
F-42	CGPA exactly 2.000, import Complete	**Good standing.** The boundary is inclusive. A CGPA of 1.999 returns **Probation**.
F-43	Full-precision CGPA of 1.99962, which displays as "2.000"	**Good standing** — the comparison uses the displayed three-decimal value, not full precision. **The label must never contradict the number printed beside it.** This is the fixture that protects that rule.

ID	Scenario	Expected
F-44	A student passing every course at D−: CGPA "0.700"	**Probation**, while having failed nothing. Correct under the conventional bands the College selected, and recorded as deliberate intent in Section 16.7 rather than as a defect.
F-44b	Any student whose CGPA is provisional	**No standing label at all** — not "Probation", not "Unknown", not a blank cell. The interface reads "not yet available". In year one this is the ordinary case.
F-44c	Student with no academic records; CGPA null	No standing label. No division error.
F-45	An I awarded in First Semester 2026/2027	Deadline computed as the **end of Second Semester 2026/2027** and displayed on the student's record beside the grade: "I — must be resolved by end of Second Semester 2026/2027".
F-46	That I is still unresolved when Second Semester 2026/2027 closes	It appears on the semester-close review worklist (A-21). **The grade does not change. The GPA does not change. No status changes.** The system reports; a human acts.
F-47	The overdue I is converted to F through the correction workflow	Admin requests with a reason; **Super Admin approves** — the same two-key path as any locked grade, with no Admin-only shortcut. On approval, in one transaction: grade becomes F; academic record updates; semester GPA, CGPA, credits attempted and credits earned recompute; **a mandatory repeat obligation is created**; academic standing is re-derived. Audit captures old value I, new value F, both actors and the reason.
F-47b	An Admin attempts to convert an overdue I without Super Admin approval	**Refused.** No such service path exists, no permission row grants it, and the check constraint would refuse it. An expired Incomplete is a locked grade like any other.

HOW TO USE THIS APPENDIX

Transcribe these forty-seven fixtures into a test file **before** writing a line of the GPA engine. They are the acceptance criterion for Gate G7, and they are the primary defence against a plausible-but-wrong implementation — an AI coding agent's priors will default to a four-point five-letter scale, two-decimal GPA and "best attempt" repeats, and all three of those defaults are wrong here. Where a fixture names a policy switch, the alternative outcome becomes a test of the switch rather than a deleted case.

End of document. Liberia Christian College E-Portal System — Phase 1 Development Planning Document, version 2.1, 25 August 2026. Development baseline; no implementation has been performed.